SoloTrace

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN

Version 0.10.1-wip (built 2026-11-10 14:30)

Matthew Armstrong · PolyForm Noncommercial 1.0.0 (source-available)

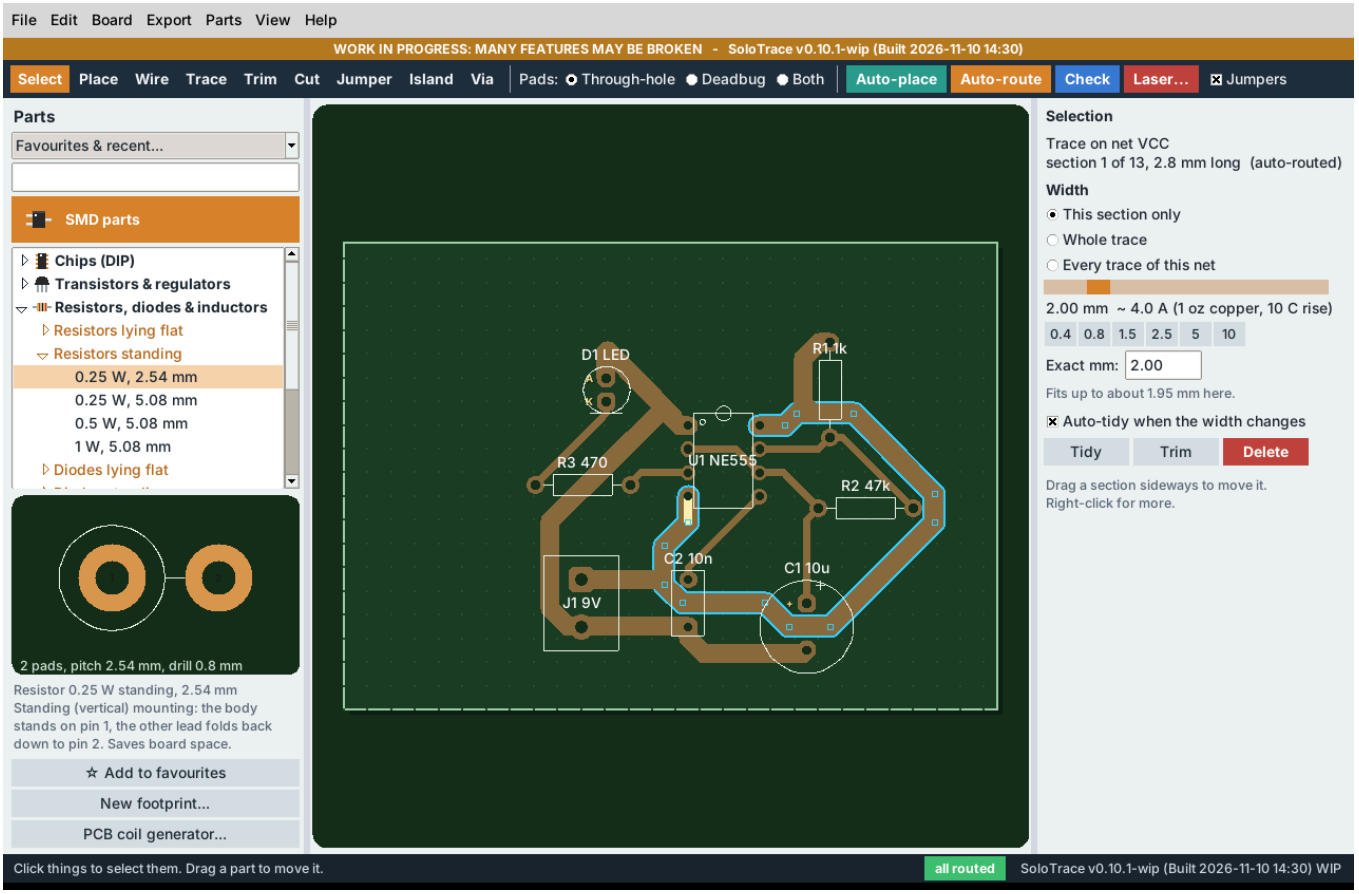
github.com/The-Dorkknight/solotrace

SoloTrace

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN · version `0.10.1-wip` · source-available, non-commercial

An easy PCB designer for **single-sided, through-hole, quick-prototype boards**. A small library of real full-size parts, traces up to 10 mm wide set with a slider, **every corner chamfered** (no pure 90° traces), an autorouter that avoids the "tangled string" look, and output for printing (toner transfer), Gerber fabs and a **gantry diode laser (Elegoo Phecda) with a negative option and adjustable trace gap**.

Open SoloTrace in your browser: nothing to install, nothing uploaded (the editor's Python runs inside the page). Or download it and double-click a launcher (below).



Screenshots are of the real program running on a Linux test machine with the built-in example boards.

> **Licence and no warranty.** Copyright Matthew Armstrong. Licensed under the PolyForm Noncommercial License 1.0.0: free to use, change and share for non-commercial purposes; commercial use needs separate permission. It is **source-available, not open source**. GitHub shows the licence as "Other". This is not legal advice. The software is provided **as is, with no warranty and no liability**. Third-party parts keep their own licences (THIRD_PARTY.md).
>
> **Safety.** Etching chemicals burn skin and eyes. A diode laser can blind you and start fires, and paint, resist and etchant fumes are harmful: wear laser safety glasses rated for your module,

> ventilate, never leave a running laser alone. The laser G-code has **never been run on a real machine**. Check every print against your real parts before etching.

What exists and what does not

Area	Status
Place, wire, trace, trim and jumper tools, undo/redo, pan/zoom, flip view	Works in tests and the screenshots; lightly tested by hand
Right-click menus (trim, split, tidy, width, rotate, duplicate, wire from a pin, route one net…)	Works in tests and the screenshots
Rigid wires (Shift + Wire): a direct power trunk that the rest of the net branches off	Works in tests and on the demo; first suggested from real use
Auto-route retries (best of up to 6 net orders) plus a polish pass, jumpers optional globally or per net; the same board always routes the same way (0.8.0)	Works on the test boards; compared with every older version in DESIGN_NOTES.md
Cut tool: one click removes a piece of trace from joint to joint (Shift: between two points)	Works in tests
Favourites & recently placed parts; Shift + click turns a part while placing	Works in tests
Branches stay joined when a trunk is widened, tidied or dragged; DRC warns about hanging trace ends	Works in tests and on rebuilds of real boards
Board > Keep traces out from under parts	Works on the test boards; can add jumpers on cramped boards
Pin labels (B C E, G D S, K A, IN GND OUT, optocoupler and DIP layouts) and part values on the board	Works; layouts are the common ones for each shape, **check your datasheet**
Dragging a trace section sideways (neighbours follow, 45° rules kept)	Works in tests; lightly tested by hand
90 through-hole parts (plus 11 hole strips for breakouts and 15 antenna and keep-out parts) in a form-factor tree with a live preview, footprint wizard, KiCad `.kicad_mod` import	Works; some parts excluded on purpose (see below)
Trace width slider 0.4–10 mm (segment, trace or net) + current readout + "fits up to" hint	Works
Wide traces leave pads straight and end on the pad centre (auto-tidy)	Works on the test boards; may leave a trace as it was when the tidy shape does not fit
Deadbug (surface-solder, no-drill) pads, per part or per pad	Works
Copper islands (Island tool): round or rectangular pads you drop, drag and resize; pins on an island join it	Works in tests and the screenshots; new in 0.4.0, not yet used on a real board
Ground fill for RF / Manhattan boards: the board is ground copper with a gap round every other net, floating fill removed	Works in tests, the examples and an independent Gerber reader; new in 0.4.0, **never etched yet**
Thermal reliefs on ground pads (ring + spokes, sized by hand or for a current; heavy / solid per part)	Works in tests and the examples; new in 0.5.0, **never soldered yet**
Back-side ground plane for a double-clad board: vias (Via tool), clearance holes, stitching grid, top-copper Gerber, back prints and laser job, countersink list	Works in tests and the examples; new in 0.5.0, **never built yet**
SMD parts (the big **SMD parts / Through-hole parts** button swaps the list): 41 common SMD footprints on the copper side	Pad sizes from makers' drawings, checked for 0.3 mm gaps; not yet soldered on a real board

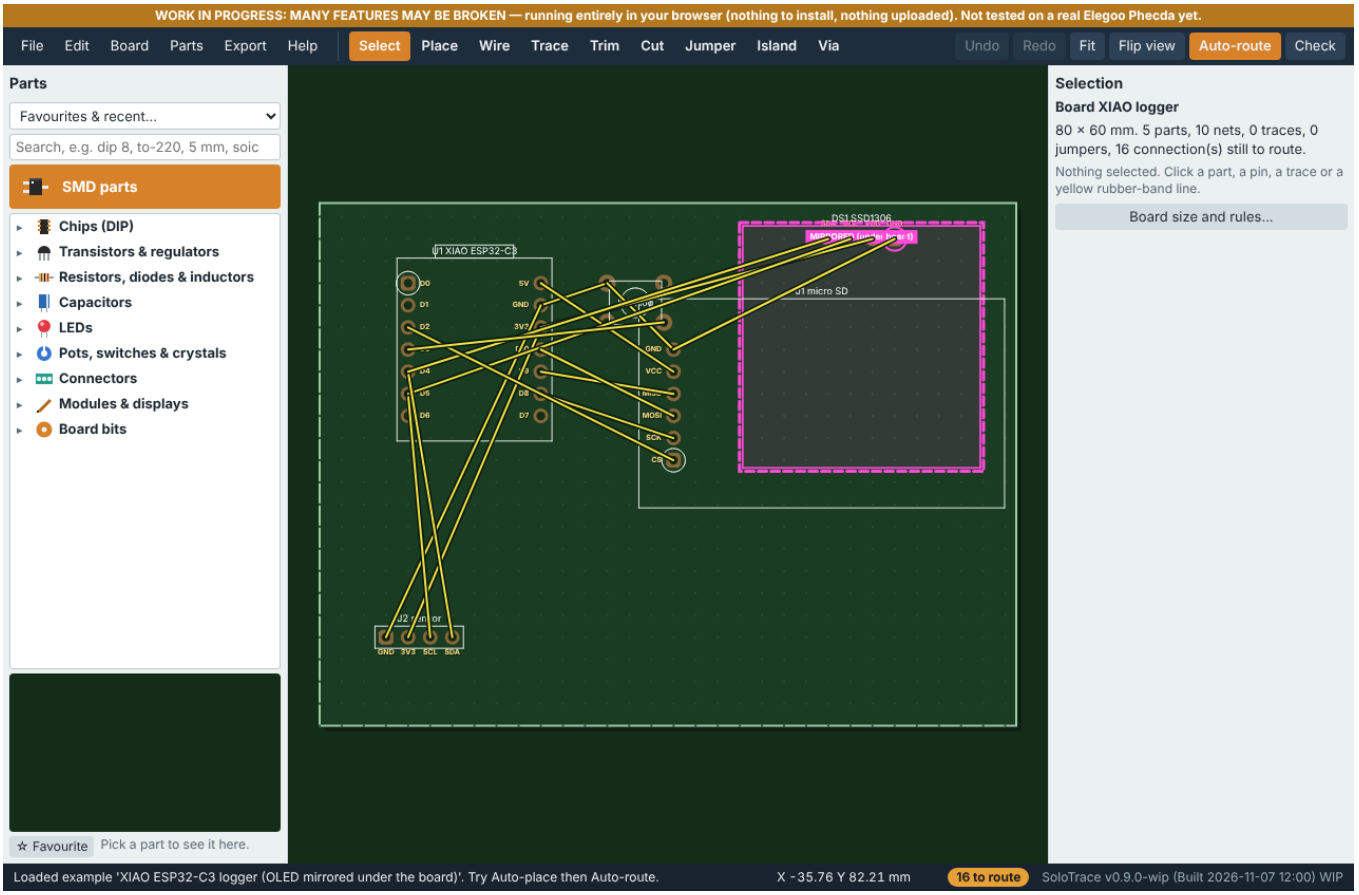
Area	Status
(0603 to 2512, SOT-23/223, DPAK, D2PAK, SOIC, SMA/SMB/SMC, tantalum, electrolytic)	
Modules & displays (new in 0.7.0): 23 microcontroller boards, OLEDs and SD card modules on 2.54 mm headers, pins already labelled, through-hole or deadbug	Pin orders, row spacings and sizes from makers' KiCad/Eagle files and datasheets; a few are single-source (each part's note says which); none plugged into a real etched board yet
Green lock circle when wiring (new in 0.9.0): with the Wire tool, a solid green circle jumps onto the pin the click would land on, so you can see before you click that the wire will connect; a grey dashed ring means it is the pin you started from	Both UIs; tested (the pin lock, the hover reply, the radius at different zooms); **not yet used by a person**
PCB antennas with keep-outs (new in 0.10.0): meander monopoles for 315 / 433.92 / 868 / 915 / 2450 MHz, wire whips and bare keep-out boxes; copper, fill and plane stay out of the box	Formula starting points (quarter wave on FR4, +10 % to trim); tests and examples only, **never etched or measured**
PCB coil generator (new in 0.10.0): outer size plus inductance wanted (finds the turns) or plus turns wanted (works out the inductance)	Estimate by a straight-bar method, within 3 % of the Mohan et al. formula on three test coils; **never measured**
Breakout boards (new in 0.9.0): three ready-made boards (XIAO ESP32-S3/C3, ESP32-C3 SuperMini, ESP32-S3-Zero) with a hole strip on every pin (4 extra holes, through-hole and deadbug in one), three power rails (3V3, GND, 5V) and a 2-pin screw terminal or header for power in	Routed with no jumpers, DRC clean, Gerber and laser files in `examples/`; **never etched or built**
Mirror a part (Shift+M) to mount it under the board, shown with a pink dashed frame and a MIRRORED tag	New in 0.7.0; tested in both UIs
Web UI: the same editor in your browser, the default since 0.8.0 (`python3 src/main.py`; `desktop` for the tkinter window)	New in 0.6.0; every tool, panel, menu, dialog and export; tested in headless Chromium and by the unit tests; **not yet used by a person on a real design**
In-browser version, nothing to install (open it): the same code run by Pyodide in the page	New in 0.8.0; tested in headless Chromium with a local copy of Pyodide 0.29.5; **loading Pyodide from the jsDelivr CDN has not been tried** (the build machine cannot reach it)
Collisions after moving a part over laid traces: red marks while dragging, a message on drop, part body over traces when mounted under the board	New in 0.8.0; Check compared with a brute-force check on 64 random moves
Auto-route algorithm: Auto (negotiated congestion, then classic), Negotiated, Classic	Measured on 11 boards (DESIGN_NOTES.md); new in 0.5.0
Auto-place algorithm: Annealing, Routability (experimental)	Routability measured **no better** than Annealing; off by default
Chamfered corners everywhere + DRC	Works
Auto-place and auto-route with jumper fallback	Works on the three demo boards and random test boards; **pure Python, can be slow** on big boards
Gerber + drill zip, 1:1 PDF/SVG, BOM CSV, netlist text	Written and re-read by an independent Gerber library in tests; **not opened in a fab's own viewer**
Laser G-code (negative / isolation / positive, test cards, frame file)	Geometry checked in tests; **never run on a real Elegoo Phecda**
Windows and macOS launchers, GitHub workflows	**Not run** (written but untested on those systems)
Floating slider pop-up beside the cursor	**Not built**: the slider is in the right-hand panel

Area	Status
Topological (rubber-band) router, C++ speed-up	**Not built** (see DESIGN_NOTES.md)
Soldermask layer, multi-layer boards, fine-pitch SMD	Out of scope

Install and run

0. Nothing to install: use it in your browser

Open **https://the-dorkknight.github.io/solotrace/app/** (needs the project's GitHub Pages switched on). The whole editor runs inside the page: Pyodide (Python compiled to WebAssembly, about 12 MB downloaded once from the jsDelivr CDN and then cached) runs the same SoloTrace code as the download, in a background worker. Your board is never uploaded: *Save* downloads the `.solotrace` file, *Open* reads one from your computer, exports download. Settings, favourites, your own parts and the autosave stay in the browser's own storage, so they survive a reload. It is slower than the download (Python in WebAssembly; auto-route takes roughly two to three times as long) and auto-route has no Cancel button. Tested in Chromium only; see Known limitations.

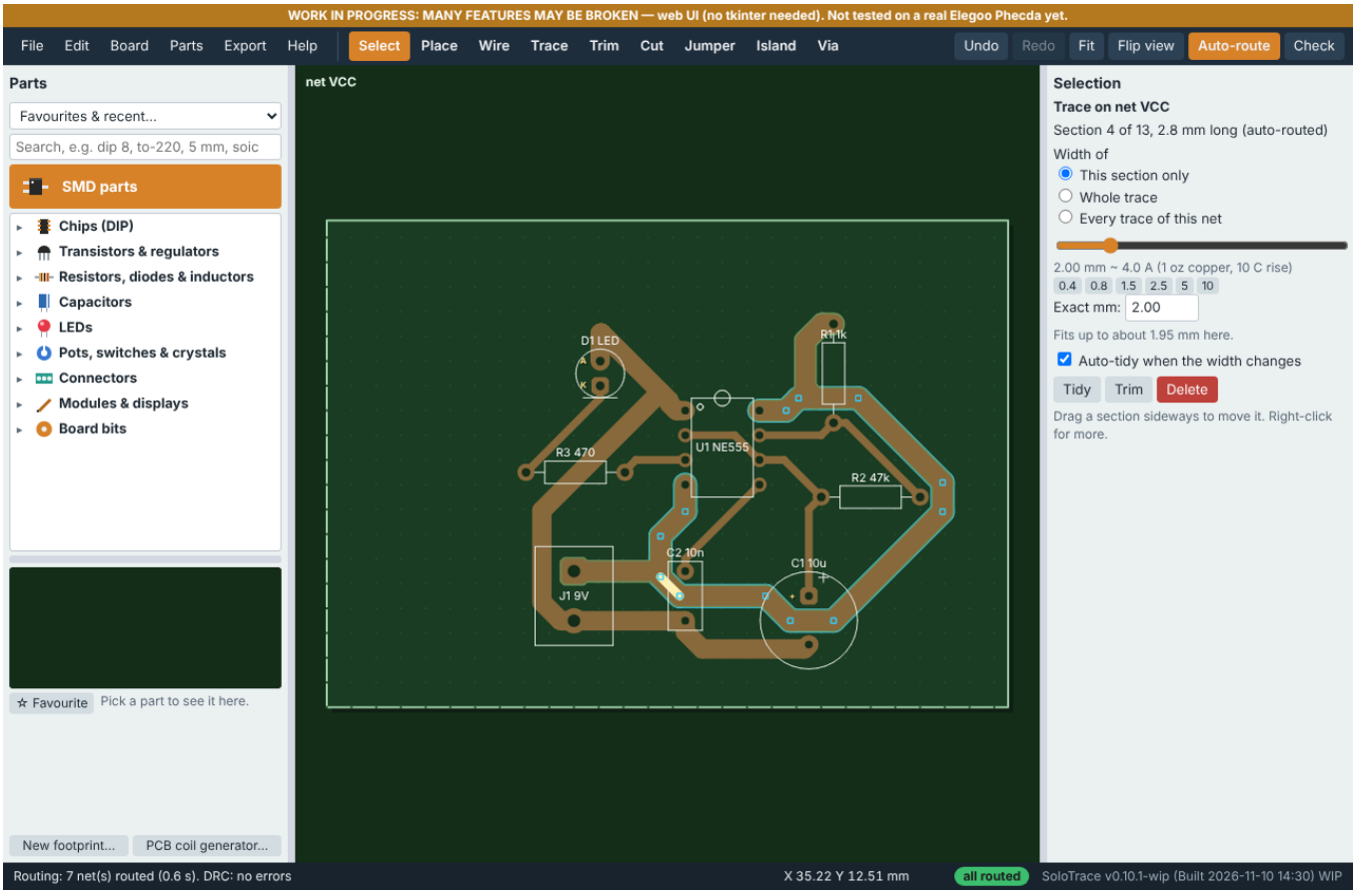


1. Double-click this

System	Double-click	What it needs
Linux	`SoloTrace-Linux.sh` (if asked, choose **Run**)	`python3` (3.8+)
Windows	`SoloTrace-Windows.bat`	Python 3.8+ from python.org
macOS	`SoloTrace-Mac.command` (first time: right-click > Open)	Python 3.8+

If Python is missing, the launcher shows a message with the exact fix (and the address of the browser version). Nothing else is needed: SoloTrace uses only the Python standard library and sends nothing over the network.

It opens in your web browser; tkinter is no longer needed. Since 0.8.0 the launcher (and `python3 src/main.py`) always starts the **web UI**: a tiny local server on `127.0.0.1` (never on the network) plus the same editor as a web page, which opens in your default browser. Every tool, the right-click menus, the panels, the dialogs, the ground fill, the back-side plane, DRC and all the exports are there; *Save* downloads the `.solotrace` file and *Open* reads one from your computer. The old desktop window (tkinter) is still there for those who like it: `python3 src/main.py desktop` (if tkinter is missing it says so and opens the browser instead). *File* > *Quit* stops the server; it also stops by itself about a minute and a half after the page is closed.



Linux Mint / Ubuntu / Debian: `sudo apt install python3` (plus `python3-tk` only if you want the desktop window). If the double-click does nothing, right-click the file > Properties > Permissions > "Allow executing as program", or run it from a terminal.

2. From a terminal

```
python3 src/main.py                 # open the editor in your browser (local server on
127.0.0.1)
python3 src/main.py desktop         # the tkinter desktop window instead
python3 src/main.py web board.solotrace --port 8321 --no-browser   # a fixed port, open the
address yourself
python3 src/main.py --version       # version line
python3 src/main.py --demo          # open the 555 example
python3 src/main.py demo "5 V regulator" board.solotrace
python3 src/main.py route board.solotrace routed.solotrace
python3 src/main.py islands board.solotrace rf.solotrace --deadbug   # ground fill on (--
off turns it off)
python3 src/main.py route board.solotrace routed.solotrace --algorithm negotiated
```

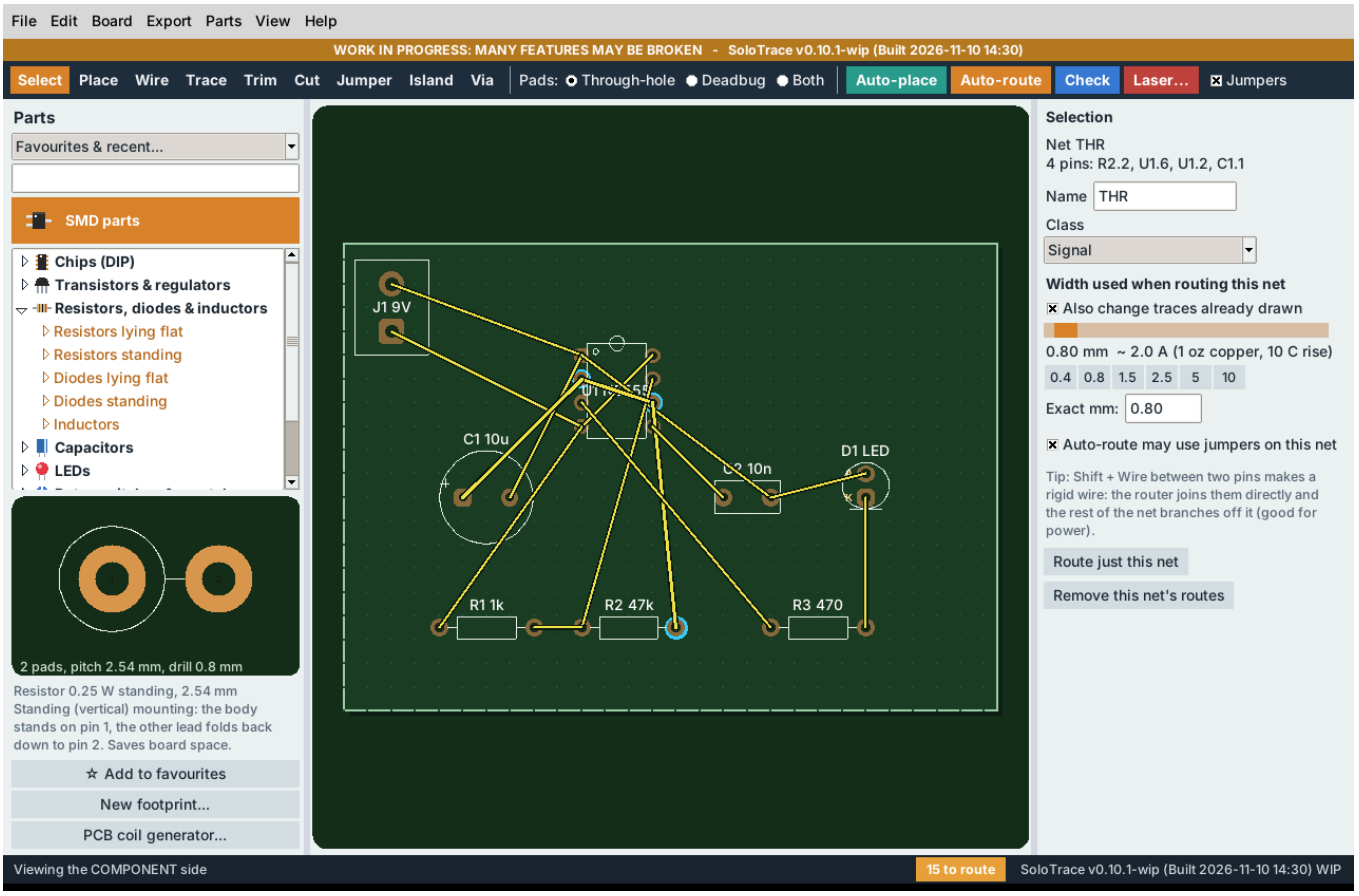
```
python3 src/main.py place board.solotrace placed.solotrace --placer routability
python3 src/main.py export routed.solotrace --gerber out.zip --pdf out.pdf --laser
out.gcode --mode negative
python3 src/main.py selftest         # run the tests
```

The first time it starts it shows a safety and work-in-progress notice; tick the box and click **I agree**. Closing that window quits.

Quick tour: your first board in 5 minutes

1. **File > Open example > 555 LED flasher** (or **File > New board** and set a size).
2. **Place parts:** pick from *Favourites & recent* at the top of the parts panel, or open a group in the parts panel on the left (the little arrows show the sub-types, such as *Resistors standing*), pick a part, check the drawing under the list, then click on the board. Or type in the search box (`dip 8`, `to-220`, `standing`). `R` or Shift + click turns the part, `Esc` stops placing. Star parts you use a lot with the button under the preview.
3. **Wire:** press `W`, click one pin, then another (or drag from pin to pin). A **green circle** snaps onto the pin the click would hit (the lock reaches about 14 screen pixels beyond the pad edge, never less than 1.4 mm or more than 3 mm on the board), so you do not have to line the cursor up exactly. A yellow rubber-band line shows the connection. Hold **Shift** for a **rigid wire** (see below). Click a rubber-band line to name its net or give it a class (Signal 0.8 mm, Power 2 mm, High current 5 mm; change them in Board > Board size and rules).
4. **Auto-place, then Auto-route** (toolbar). A progress bar with **Cancel** shows while it works. Routes use 0°, 45° and 90° moves, and a 90° turn is always built as two 45° turns (a chamfer). If a trace can't get through, a **wire-link jumper** (blue dashed) is added on the component side.
5. **Label:** select a part and type its *Value / type* (NE555, BC547, 10k...), and pick its *Pin labels* (B C E, G D S, IN GND OUT...).
6. **Tweak:** drag a trace section sideways to move it, use **Trim** (`X`) to cut a section out, and **right-click** anything for its menu.
7. **Check**, then **Export**.

Quick and dirty RF (Manhattan / deadbug): *File > Open example > Manhattan RF amplifier (copper islands)*. Press `I` and click to drop copper islands, sit the parts' pins on them, and tick *Board > Ground fill*. No traces needed.

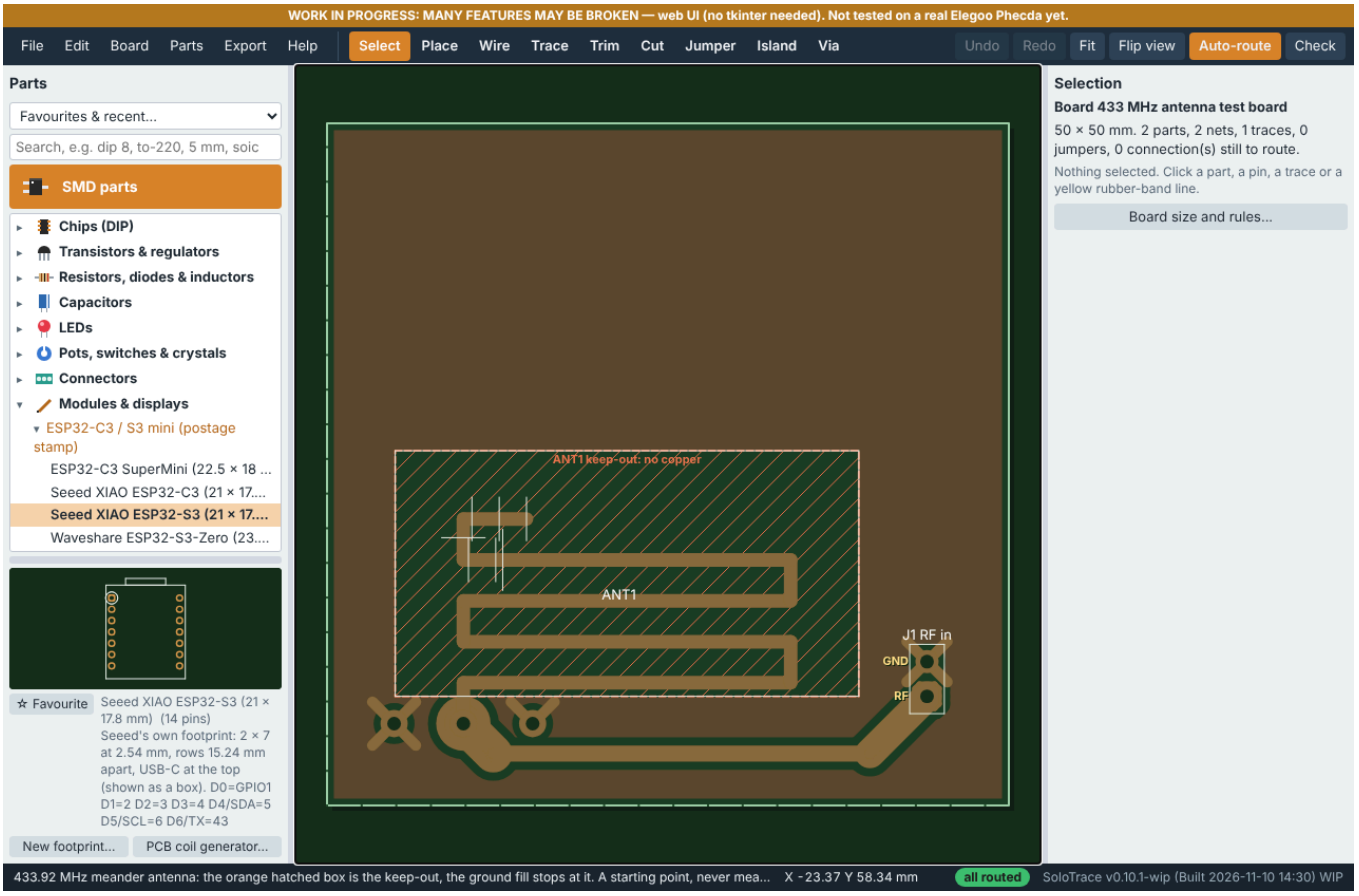


Resizing the panels (new in 0.10.1)

The parts panel and the selection panel can be dragged wider or narrower with the grey handle beside them, and the web UI also has a handle between the parts list and its preview (drag it down to give the list more room, up to read a long description). Double-click a handle to put it back.

Antennas and the coil generator (new in 0.10.0)

Antennas. In the parts list, *Board bits > Antennas and keep-outs* holds a meander antenna for each of 315, 433.92, 868, 915 and 2450 MHz, a matching **wire whip** (cut a wire of the length in its name) and five bare **keep-out boxes**. Place the antenna at the edge of the board, wire pin 1 to the radio's RF pin (through any matching parts) and pins 2 and 3 to ground, and keep the orange hatched box free: copper, ground fill and the back-side plane are all kept out of it, the autorouter goes round it and Check reports anything inside it. Open *File > Open example > 433 MHz meander antenna* to see one.

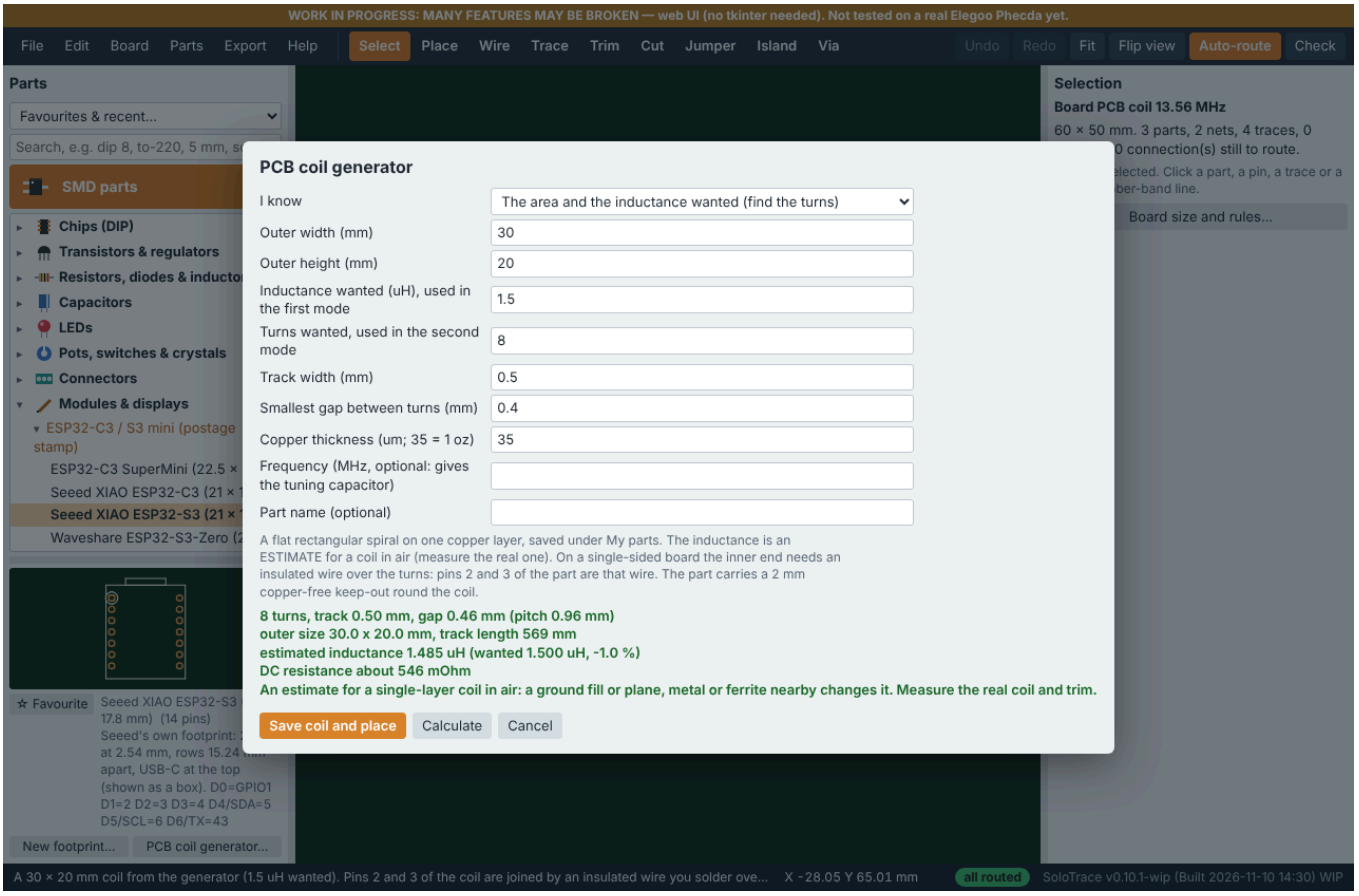


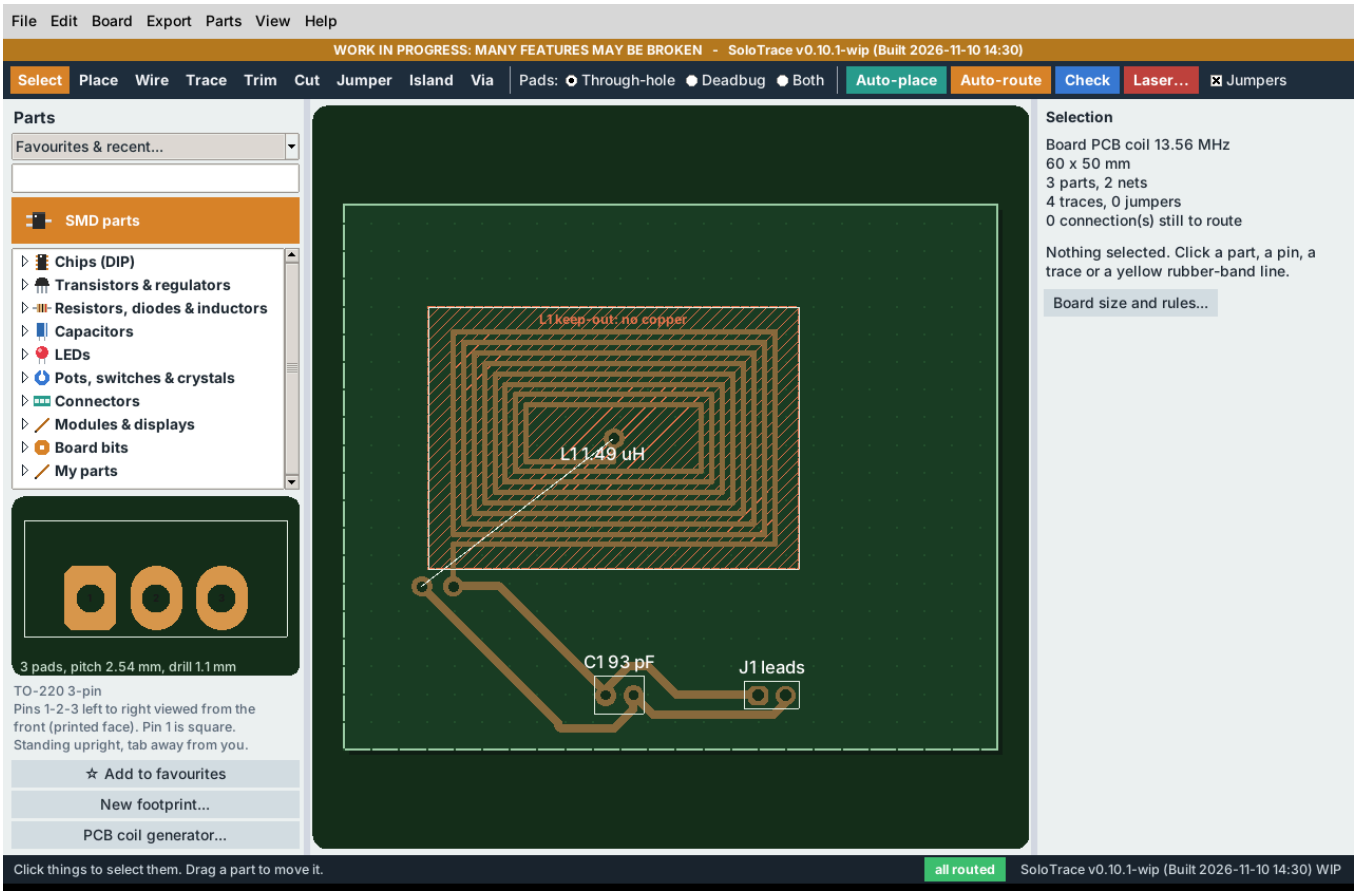
These are **starting points, not tested designs**. The copper length is the quarter wave on FR4 by a rule of thumb plus 10 % to trim: measure with a NanoVNA and scratch copper away from the tip (silk ticks every 2 mm; the longer tick is the formula length). A real antenna usually also needs a small matching network, which SoloTrace does not provide. Notes, sources and what was left out: docs/research/pcb-antennas.md.

Coil generator. The *PCB coil generator...* button under the parts list (also in the *Parts* menu). Type the outer width and height, then pick one mode:

1. **Area + inductance**: say how many uH you want and it finds the number of turns and the track spacing that come closest.
2. **Area + turns**: say how many turns and it works out the inductance, spreading the turns over the area.

Calculate shows the turns, gap, track length, estimated inductance, DC resistance and (if you gave a frequency) the parallel capacitor that would tune it. *Save coil and place* stores it under *My parts* and arms it for placing. On a one-sided board the inner end of the coil cannot come out without crossing the turns, so solder an **insulated wire from pin 2 to pin 3** of the part; SoloTrace treats those two pins as already joined. Wire your circuit to pins 1 and 3. Open *File > Open example > PCB coil from the generator* to see one.



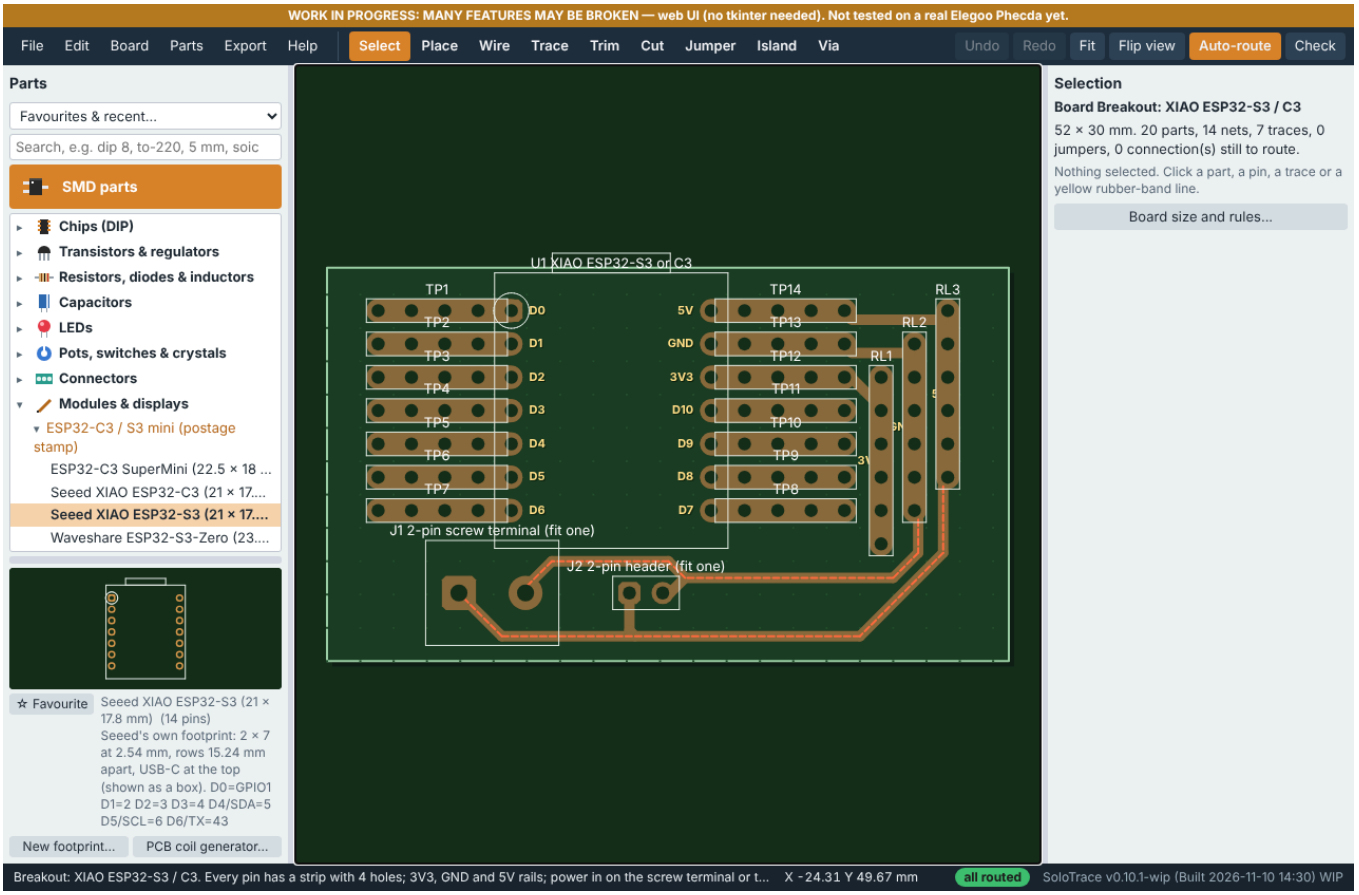


The inductance is an **estimate for a coil in air**; a ground fill or plane, ferrite or metal nearby changes it, so measure the real coil. Method and cross-check: [docs/research/pcb-antennas.md](). From a terminal: `python3 src/main.py coil --width 30 --height 20 --uh 1.5 --freq 13.56` (or `--turns 8`; add `--save` to put the part under My parts).

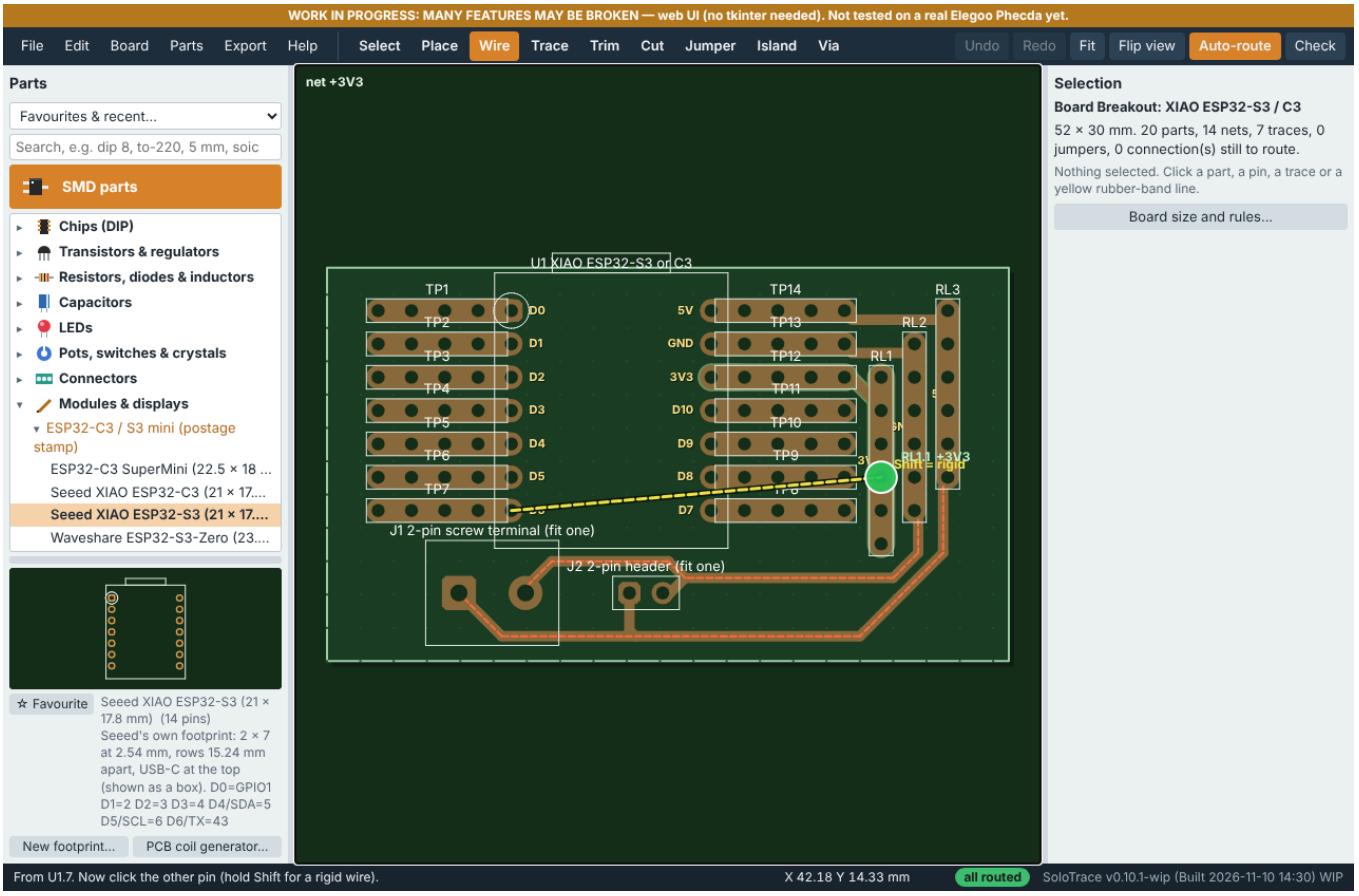
Breakout boards (new in 0.9.0)

File > Open example now has three breakouts for the "postage stamp" ESP32 boards: **XIAO ESP32-S3 / C3** (one board serves both, same footprint), **ESP32-C3 SuperMini** and **ESP32-S3-Zero**. Each is generated by the app from the module's own pad table, routed by the autorouter (no jumpers) and checked (no DRC problems).

- **Every module pin has a hole strip**: one bar of copper with four holes, touching the module's pad. Pick any hole for a pin header, a jumper wire or a component lead, or solder a wire or an SMD part flat on the bar (deadbug). With the module's own pad that is **four extra connections per GPIO**, all the same net.
- **Three power rails** of six holes each: 3V3, GND and 5V.
- **Power in** on a 2-pin **screw terminal** and on a 2-pin **header**, both on 5V (pin 1) and GND (pin 2). Fit one.
- **No copper under the module**, because many of these modules have pads on the back.
- Sizes: 52 × 30 mm (XIAO), 52 × 31.5 mm (SuperMini), 52 × 32.5 mm (S3-Zero).



The green circle while wiring (here from a module pin towards the 3V3 rail; the cursor was a little off the hole):



Build your own: *Board bits > Hole strips (breakout)* has strips of 2 to 16 holes. A strip is always through-hole and flat copper (the *Pads of this part* choice does not apply), is one net, and is not in the BOM. The "with lead-in" strip (`BRK_STRIP4_LEAD`) starts 0.3 mm past a module pad so that the two touch and need no wire. Details and the open points: docs/research/breakout-boards.md.

Check before you power one: nothing here has been built. The 5V pad of the XIAO is called VBUS in Seeed's symbol; whether it is safe to feed it from the screw terminal while USB is plugged in was **not** looked up. Read your module's documentation first.

Feature by feature

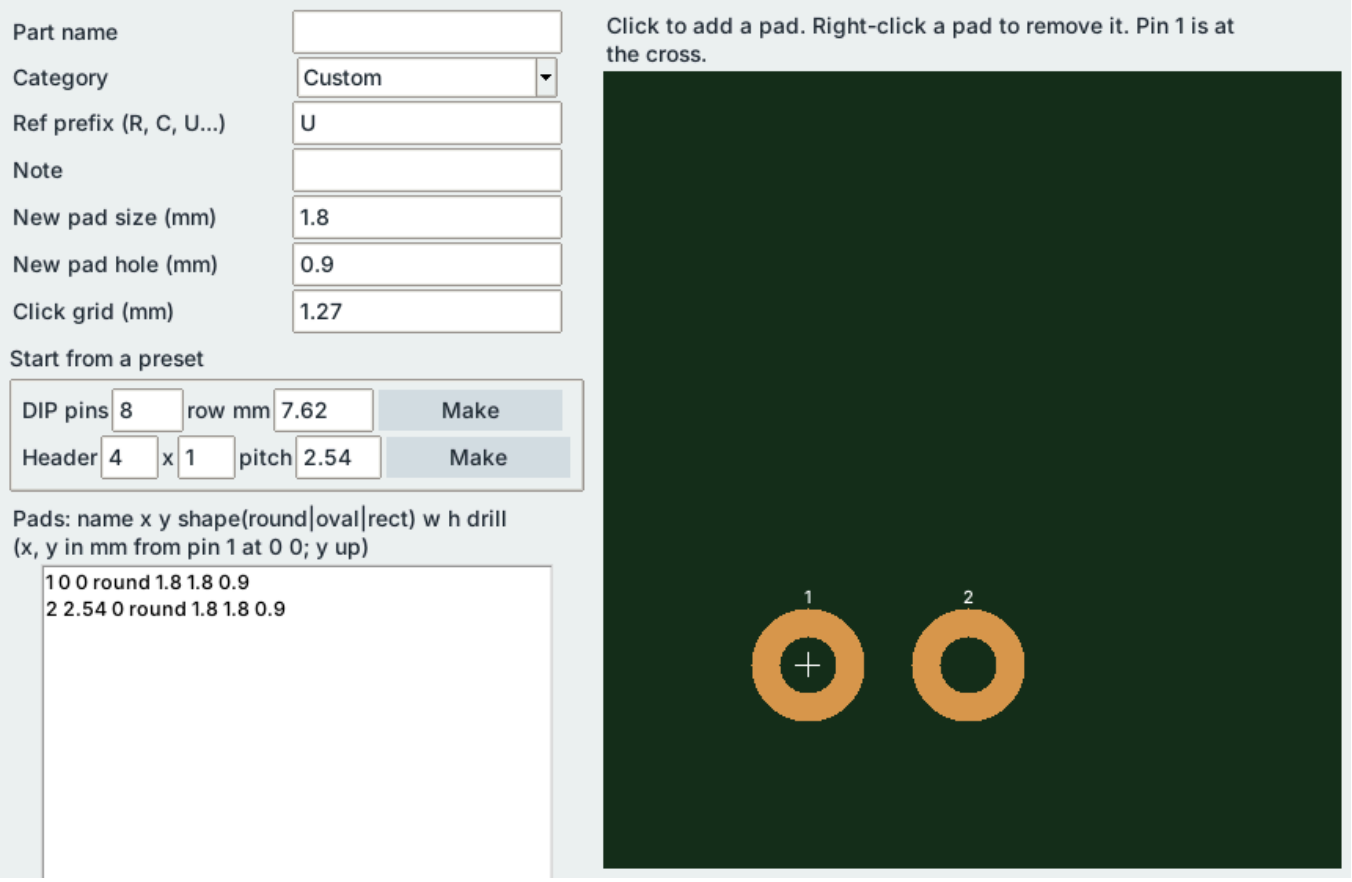
Library and parts. 90 through-hole parts plus 11 hole strips, 23 modules and 41 SMD parts, sorted by shape (form factor) rather than by maker code. Each group has an arrow that opens its sub-types:

Group	Sub-types
Chips (DIP)	Narrow (7.62 mm rows) DIP-4 to DIP-28, wide (15.24 mm rows) DIP-28 and DIP-40
Transistors & regulators	TO-92, TO-220 (2-pin, 3-pin, with heatsink outline), TO-247 / TO-3P
Resistors, diodes & inductors	Resistors lying flat (1/4 W to 5 W), **resistors standing (1/4 W at 2.54 and 5.08 mm, 1/2 W and 1 W at 5.08 mm)**, diodes lying flat and standing, axial and radial inductors
Capacitors	Ceramic disc, film / box, electrolytic
LEDs	3 mm and 5 mm
Pots, switches & crystals	Inline pots (2.54 and 5.08 mm), triangle trimmer, 6 mm tactile switch, HC-49 crystal
Connectors	Pin headers 1-row and 2-row, screw terminals, JST-XH

Group	Sub-types
Board bits	Wire links, mounting holes, solder and surface pads, **hole strips** (1 × 2 to 1 × 16 holes in one bar of copper, for breakouts)
My parts	Parts you made or imported

Click a part and a drawing of it (copper, holes, outline, pin numbers, pitch and drill) appears under the list, in the sidebar, with no pop-up. Names say what the part is ("Electrolytic cap, 5 mm pitch"), not an obscure model number. Left out on purpose because their sizes vary too much between makers: relay, USB-B, barrel jack, buzzer, fuse holder, panel pots with lugs, TO-126, bridge rectifiers. Make those with the wizard. Body outlines, the inductor, pot and trimmer layouts, and the standing-part pitches are generic: check them against your real part.

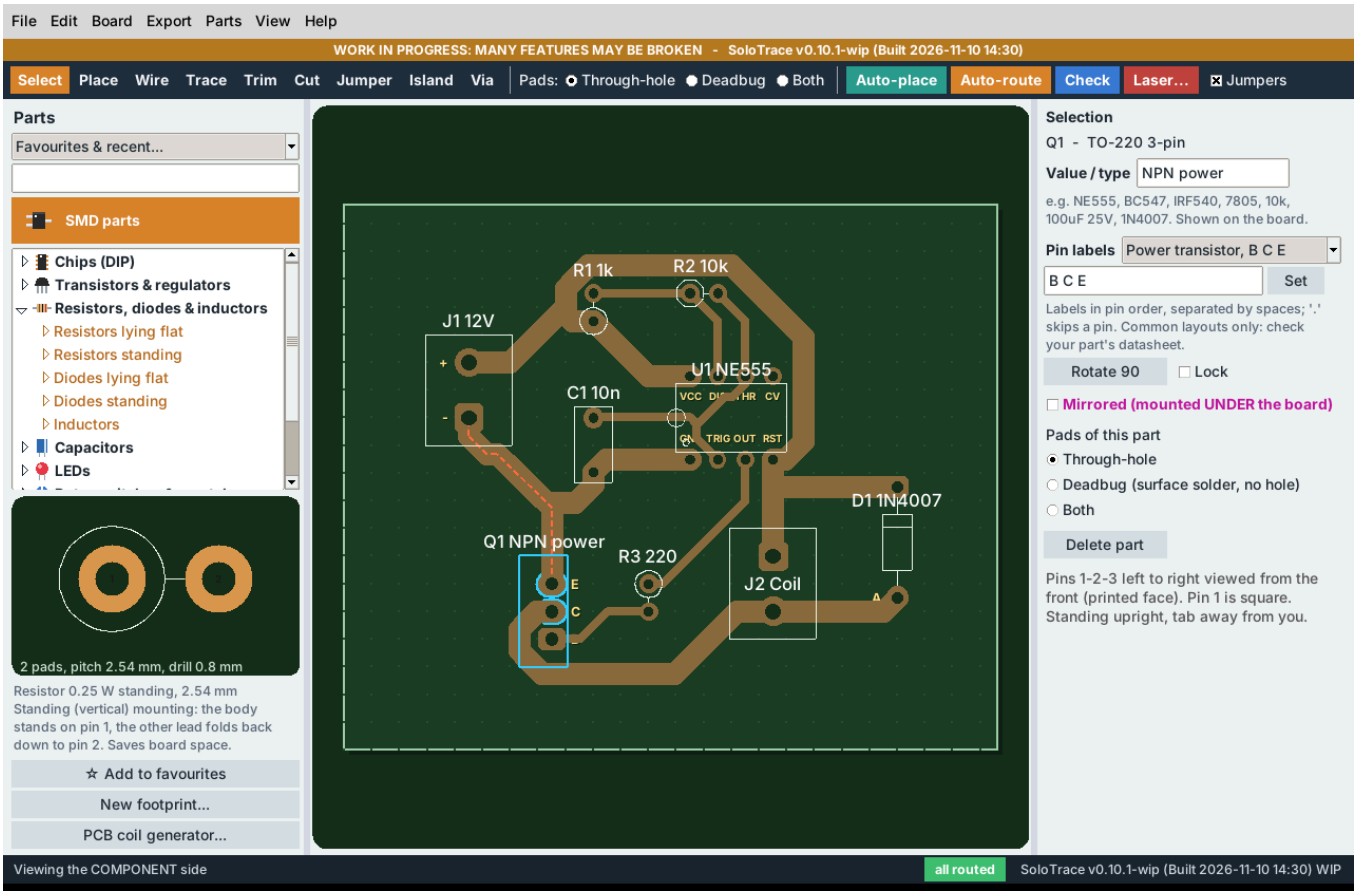
Your own parts. *Parts > New footprint...* opens the wizard: type a pad table (name, x, y, pad size, drill), or click on the canvas to drop pads, or start from a DIP or header preset, then save. Saved parts live in `~/.solotrace/parts` (*Parts > Open my parts folder*). *Parts > Import KiCad .kicad_mod...* reads the text format of a KiCad footprint (through-hole pads and silkscreen lines only).



Trace width. Press `T` and click along the board to draw a trace (`Enter`, a double-click or *Finish trace here* on the right-click menu finishes it; corners are chamfered for you). To change a width, click a **trace section** on the drawing, or click a **yellow rubber-band line** to set the width of a whole net, then drag the **slider** in the right panel (0.4–10 mm; preset buttons 0.4, 0.8, 1.5, 2.5, 5, 10). Choose whether it applies to the section, the whole trace or the whole net. The panel shows roughly how many amps that width carries (IPC-2221 outer-layer formula; board settings give copper weight and temperature rise; a guide only) and how wide the trace can go where it is before it breaks the clearance rule.

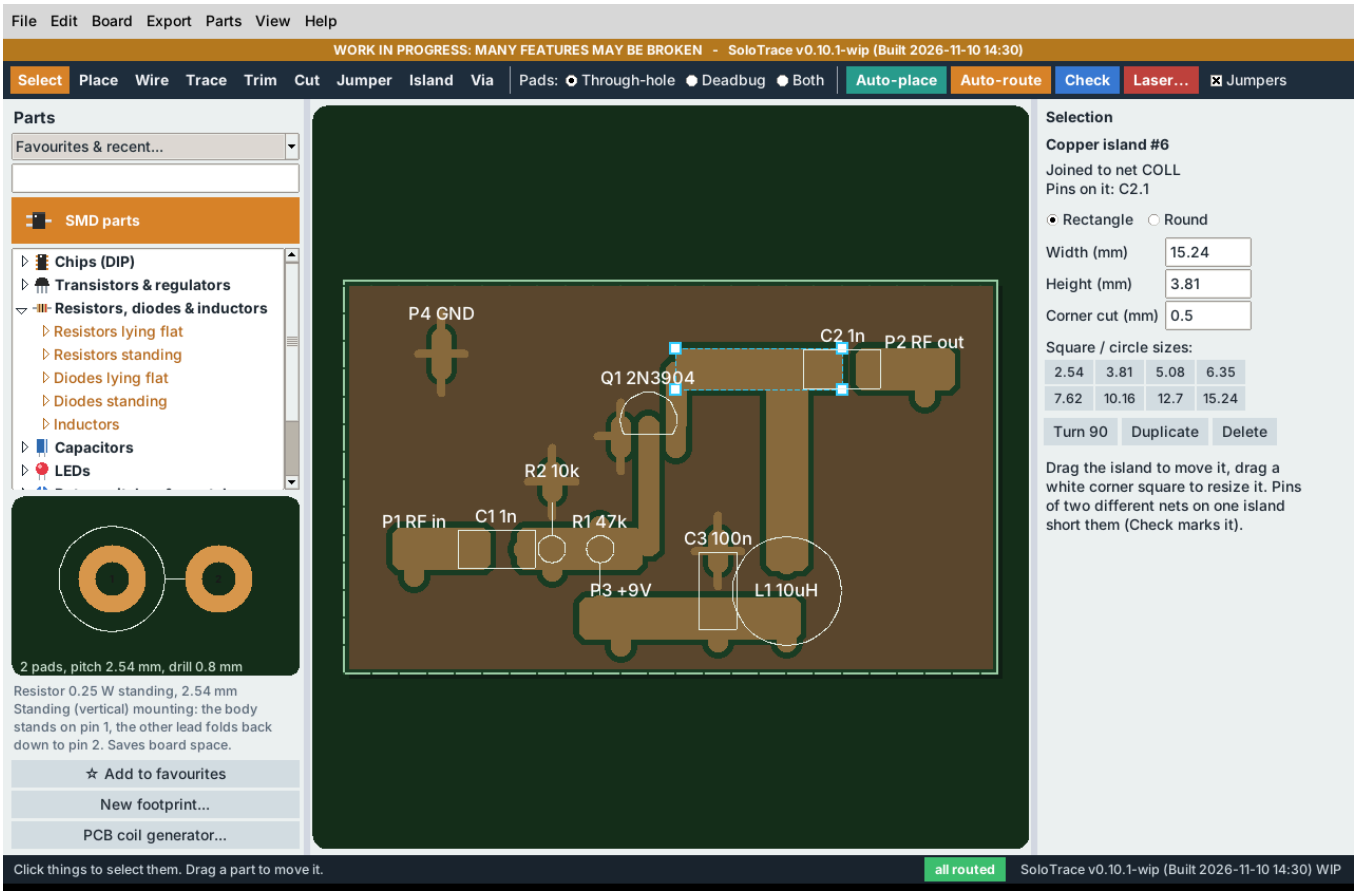
Rigid wires (power trunks). A net with more than two pins is normally routed by joining the nearest pins first, so a ground net can end up as one long chain (transistor to capacitor to terminal). If you

want a direct trace between two particular pins, hold **Shift** while wiring them (click or drag), or right-click a pin > *Rigid wire from this pin*, or right-click an existing rubber band > *Make ... rigid*. The router routes every rigid pair first, as its own pin-to-pin trace, then connects the rest of the net to the nearest point of that trunk, so a capacitor becomes a short branch. "Rigid" fixes *which* two pins are joined, not the path: the trunk bends round other parts like any trace, can be moved by a re-route, and you can drag or cut it. Rigid rubber bands are thicker and orange. Once routed, the trunk is drawn in the same copper colour as everything else, so joints look like the one piece of copper they will be, with a thin dashed orange line down its middle to show it is a rigid wire. Branches stop inside the trunk instead of poking out of its far side, and when you drag a trunk sideways the branches joined to it move with it and stay joined. The net panel lists the rigid pairs, each with a *make normal* button.



Copper islands (Island tool, `I`). Free-standing pads for Manhattan / deadbug building. Click on the board to drop one (5.08 mm square to start), or drag to draw one any size; Shift + click switches new islands between rectangular and round. With *Select*, drag an island to move it (0.635 mm steps, so it can sit under pins) and drag one of its white corner squares to resize it while the opposite corner stays put. The right panel takes an exact width, height and corner cut (rectangles get 45 degree corners like everything else; a round island with a different width and height is an oval), offers square/circle sizes from 2.54 to 15.24 mm, and has *Turn 90*, *Duplicate* (`Ctrl+D`) and *Delete*; arrow keys nudge it. Right-click an island for the same choices, or right-click empty board > *Add a copper island here*. An island hidden under a part's body: click the part, then click it again to pick the island.

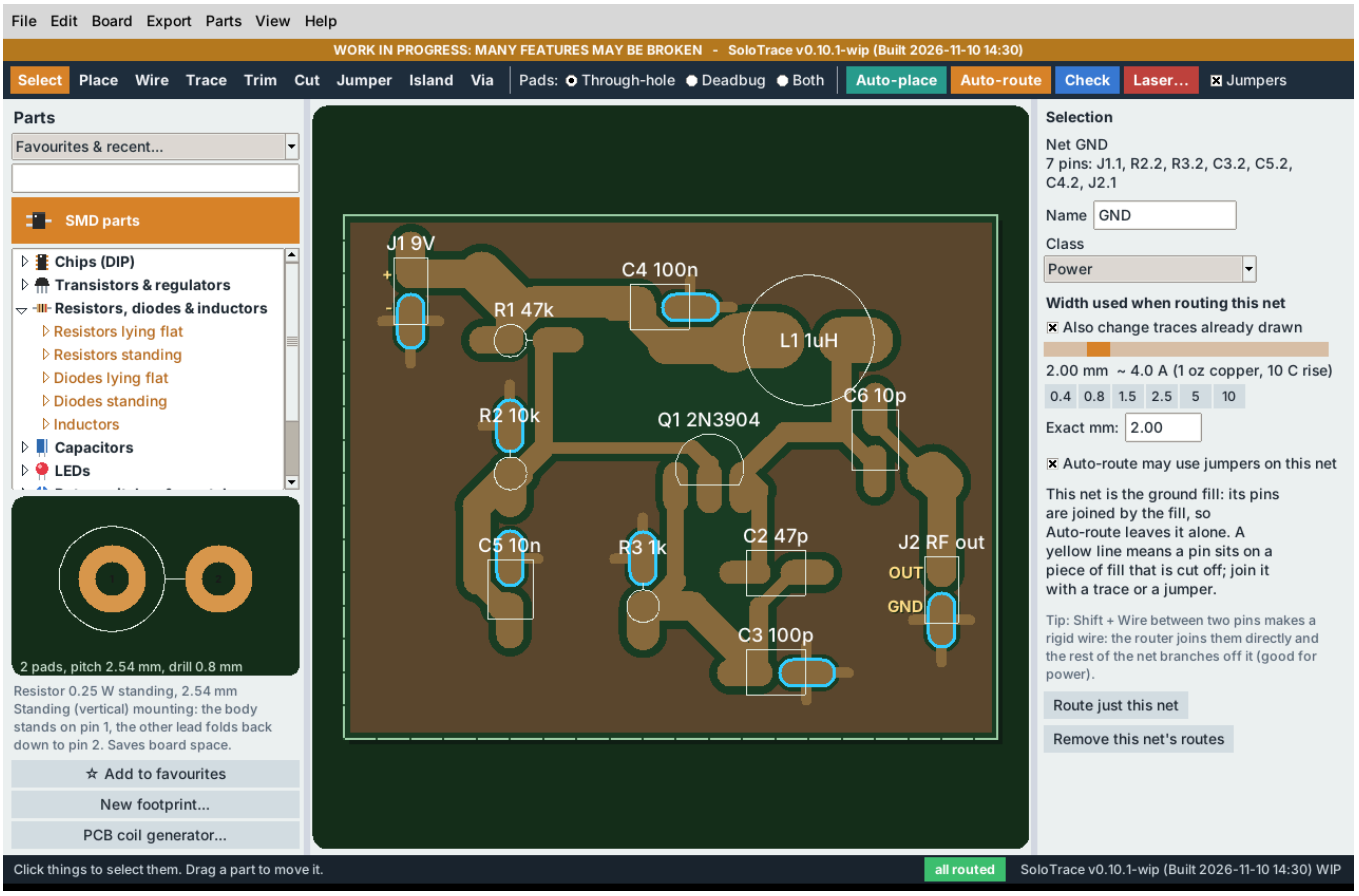
An island has no net of its own: it **joins the net of whatever touches it**, a pin, a trace or a jumper pad, and islands that touch each other share a net. So sit a part's pins on islands (deadbug pads make that easy) and the connection is made, with no trace. The panel says which net an island is on and which pins sit on it. If pins of two different nets end up on one island, that is a short and *Check* reports it. The router treats islands as copper and routes round them, and they appear in every export.

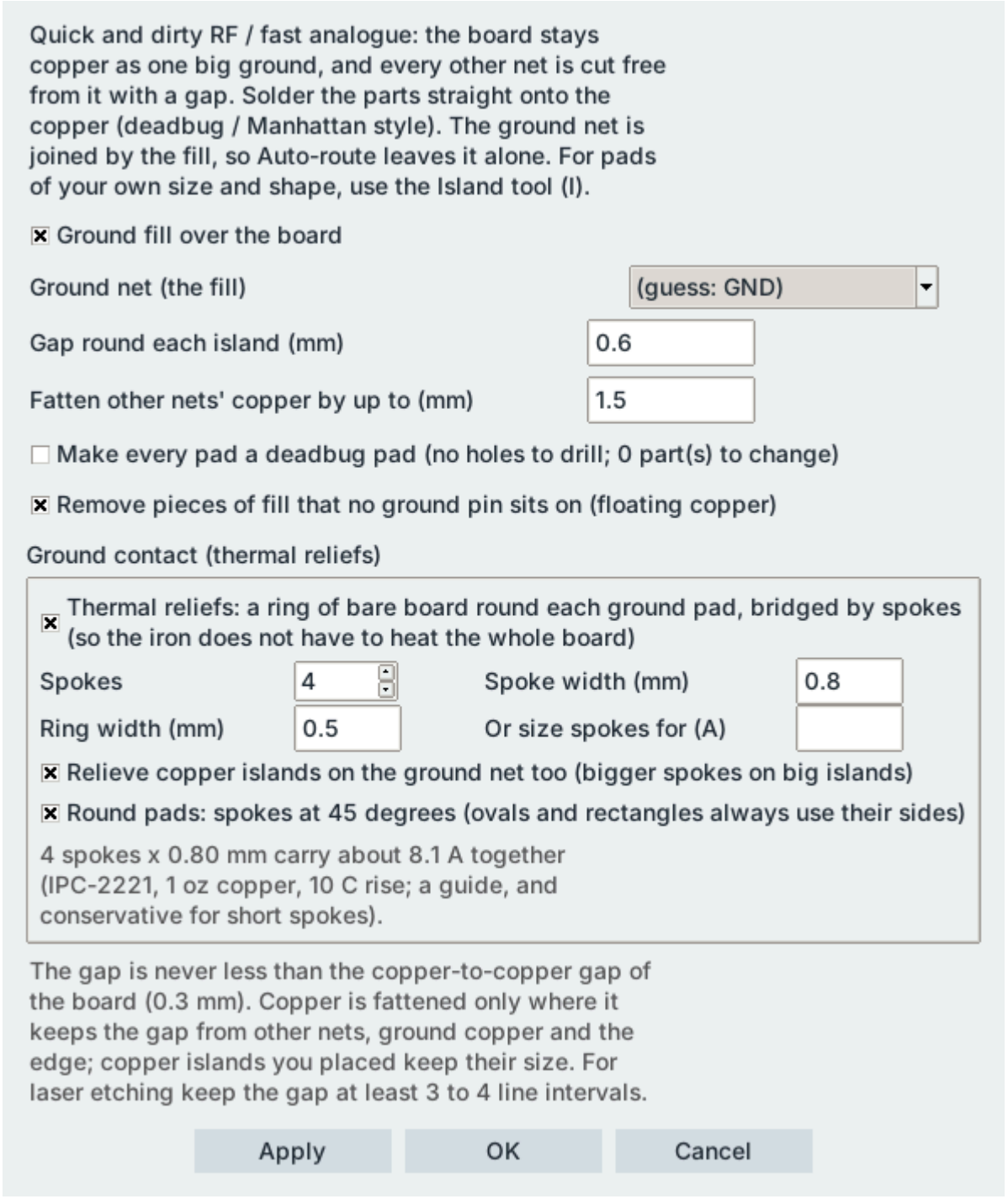


Ground fill (Board > Ground fill). For quick and dirty RF and fast analogue: the whole board becomes copper, used as ground, and every other net is cut free from it with a gap. Settings are under *Board > Ground fill settings...*:

- **Ground net:** guessed (a net called GND, 0V, VSS, ground... otherwise the net with the most pins), or pick one; the net panel also has *Use this net as the ground fill*. The ground net is joined by the fill, so Auto-route does not route it, and its old automatic traces are removed when you turn the fill on.
- **Gap** round everything else (default 0.6 mm, never less than the board's copper-to-copper gap). For laser etching keep it at least 3 to 4 line intervals.
- **Fatten other nets' copper** by up to this much (default 1 mm): routed traces and pads grow into wide, low-inductance patches, but only where that keeps the gap from other nets, ground copper and the board edge. Copper islands you placed keep their size.
- **Remove floating fill** (on by default): a piece of fill that no ground pin sits on, boxed in by other nets, is cut away rather than left as floating copper. A ground pin that sits on a piece of fill cut off from the rest gets a yellow rubber band; join it with a trace or a jumper.
- Turning the fill on offers to make every pad a **deadbug pad**, since such boards are usually built with the parts soldered straight onto the copper.

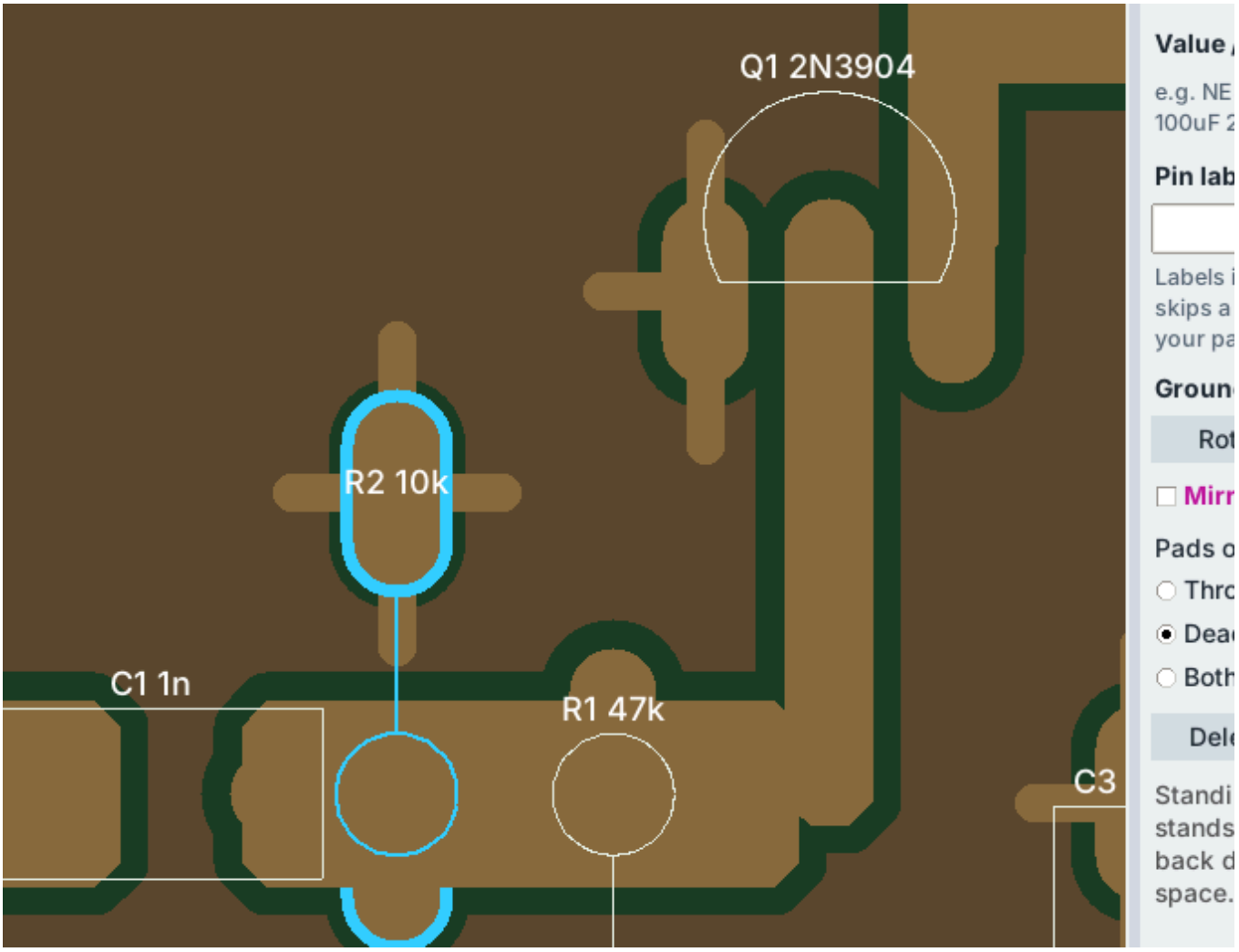
The fill is drawn in the editor and is in the Gerber, the PDF/SVG prints and the laser G-code. In negative laser mode the laser then burns only the gaps (much faster); isolation mode burns the same thing on such a board.


File Edit Board Export Parts View Help
WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN - SoloTrace v0.10.1-wip (Built 2026-11-10 14:30)
Select Place Wire Trace Trim Cut Jumper Island Via
Pads: Through-hole Deadbug Both
Auto-place Auto-route Check Laser... Jumpers
Parts
Favourites & recent...
SMD parts
Chips (DIP)
Transistors & regulators
Resistors, diodes & inductors
Resistors lying flat
Resistors standing
Diodes lying flat
Diodes standing
Inductors
Capacitors
LEDs
2 pads, pitch 2.54 mm, drill 0.8 mm
Resistor 0.25 W standing, 2.54 mm
Standing (vertical) mounting: the body stands on pin 1, the other lead folds back down to pin 2. Saves board space.
☆ Add to favourites
New footprint...
PCB coil generator...
J1 9V
C4 100n
R1 47k
L1 1uH
C6 10p
R2 10k
Q1 2N3904
C5 10n
R3 1k
C2 47p
J2 RF out
OUT
GND
C3 100p
Selection
Net GND
7 pins: J1.1, R2.2, R3.2, C3.2, C5.2, C4.2, J2.1
Name GND
Class
Power
Width used when routing this net
Also change traces already drawn
2.00 mm ~ 4.0 A (1 oz copper, 10 C rise)
0.4 0.8 1.5 2.5 5 10
Exact mm: 2.00
Auto-route may use jumpers on this net
This net is the ground fill: its pins are joined by the fill, so Auto-route leaves it alone. A yellow line means a pin sits on a piece of fill that is cut off; join it with a trace or a jumper.
Tip: Shift + Wire between two pins makes a rigid wire: the router joins them directly and the rest of the net branches off it (good for power).
Route just this net
Remove this net's routes
Click things to select them. Drag a part to move it.
all routed
SoloTrace v0.10.1-wip (Built 2026-11-10 14:30) WIP



Thermal reliefs (ground contact). A pad joined solidly to a big copper fill is hard to solder: the fill takes the heat away faster than a hobby iron supplies it. So every ground pad gets a **ring of bare board bridged by spokes** (a thermal relief), in the ground fill and in the back-side plane. *Board > Ground fill settings > Ground contact* sets the number of spokes (default 4), their width (default 0.8 mm) and the ring (0.5 mm), and shows the current the spokes carry together (IPC-2221 outer-layer formula, 1 oz copper, 10 °C rise: four 0.8 mm spokes are good for about 8 A). Ground usually carries the return current of the whole board, so there is also **"size spokes for N amps"**: type the current and the spoke width is worked out for you. Round pads get radial spokes at 45°; deadbug ovals, rectangular pads and copper islands get spokes off their flat sides and ends (never a corner); islands over 4 mm get spokes scaled to their size. In a part's panel, *Ground contact* can be set to **heavy** (spokes twice as wide) or **solid** (no relief: for the heavy-current pads and for decoupling capacitors and RF grounds, where the industry rule is a direct connection). *Check* warns when the ground net's trace class is good for more current than its reliefs carry. A spoke that would touch another net is

dropped, and a pad that would lose all its spokes stays solid, so a relief never cuts a pad off. Research and sources: docs/research/thermal-reliefs.md.

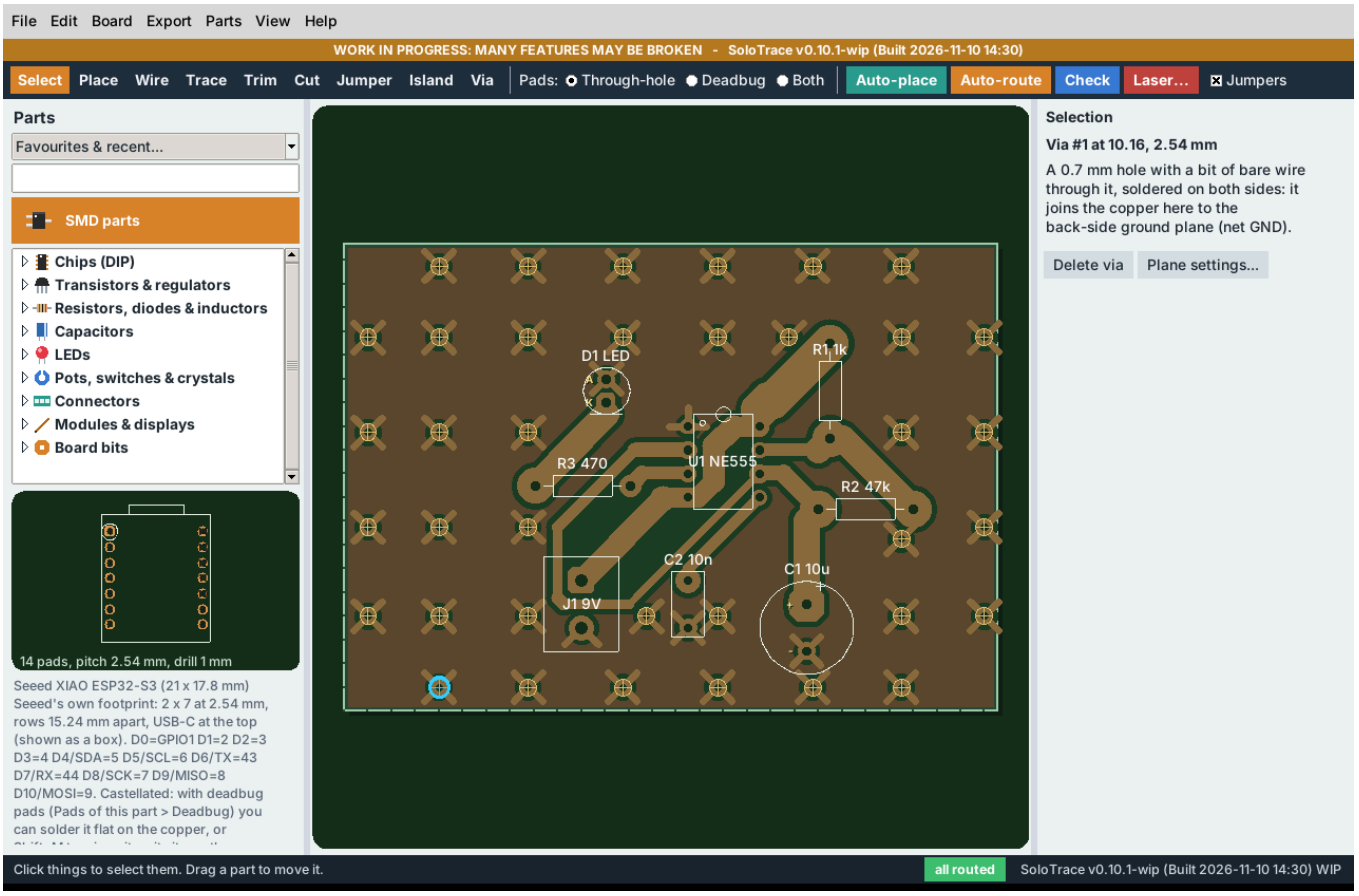


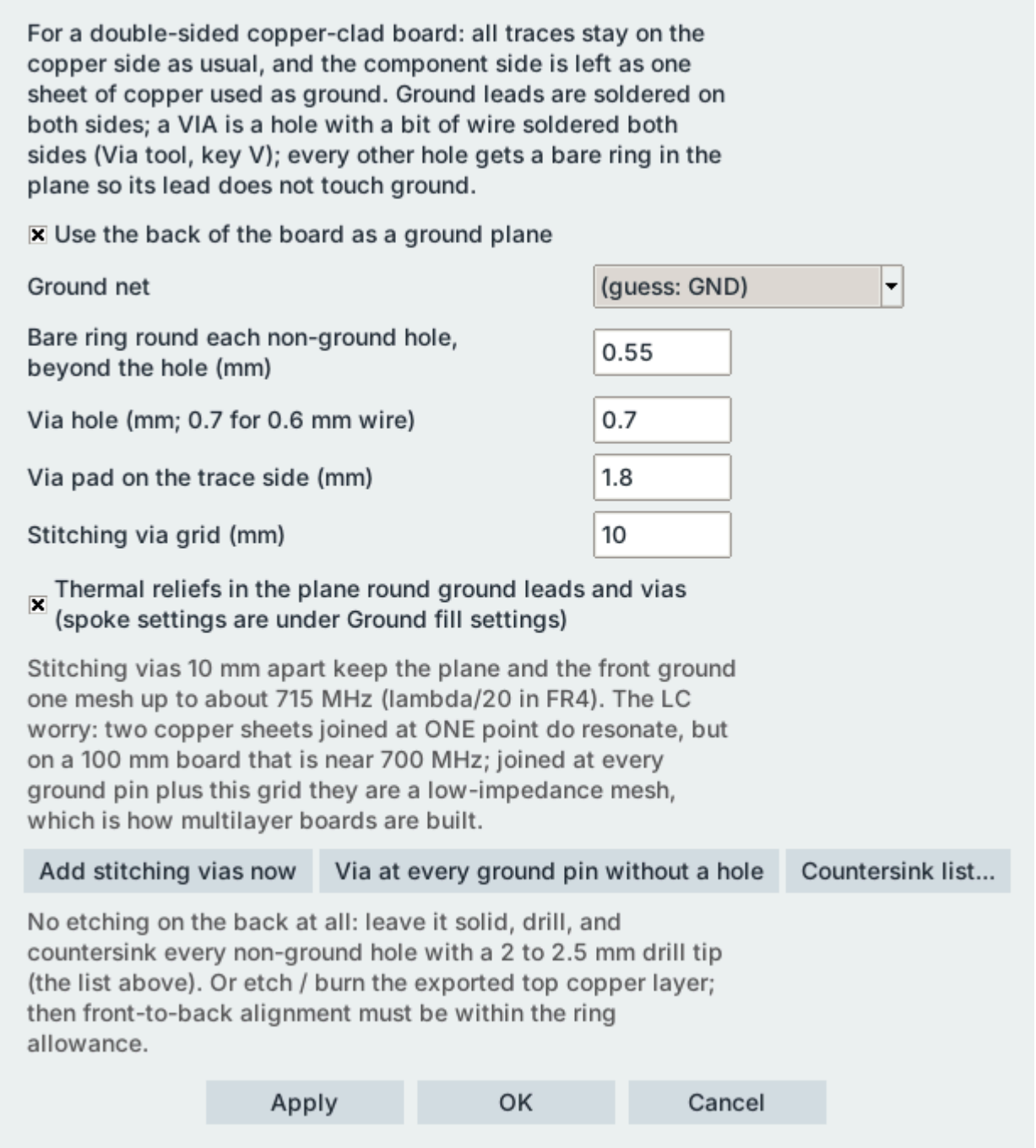
Back-side ground plane (double-clad board). *Board > Back-side ground plane* uses the component side of a double-sided copper-clad board as one solid ground plane while all traces stay on the copper side as usual. Every ground lead that passes through the board is soldered on both sides; a **via** (Via tool, `V`; a hole with a bit of bare wire through it, soldered both sides) joins deadbug or SMD ground pins, ground islands and the ground fill to the plane; every other hole gets a bare ring in the plane (default hole + 1.1 mm: 0.25 mm of electrical clearance plus 0.3 mm for front-to-back misalignment). The dialog has *Via at every ground pin without a hole* and *Add stitching vias* (a grid, default 10 mm), and says up to what frequency that spacing keeps the plane and the front ground one low-impedance mesh (lambda/20 in FR4: 10 mm is good to about 700 MHz).

The "big LC circuit" worry: two copper sheets joined at one point do form a resonant cavity, but on a 100 mm FR4 board its first resonance is near 700 MHz, far above most hobby circuits, and joining the sheets at every ground pin plus a loose grid of vias turns the cavity into a mesh, which is exactly how commercial multilayer boards are built. What you gain even at audio and low MHz: every trace has its return path directly underneath, so loops, crosstalk and radiated noise shrink. Sources and the numbers: docs/research/back-side-ground-plane.md.

Two ways to make the back: (1) **etch or burn it**: *Export > Gerber* gains a top-copper layer, *Export > Back-side ground plane* gives a 1:1 PDF/SVG (seen from the component side, so not mirrored), and the laser dialog has a *Back-side ground plane* choice (burns only the rings and reliefs); front-to-back

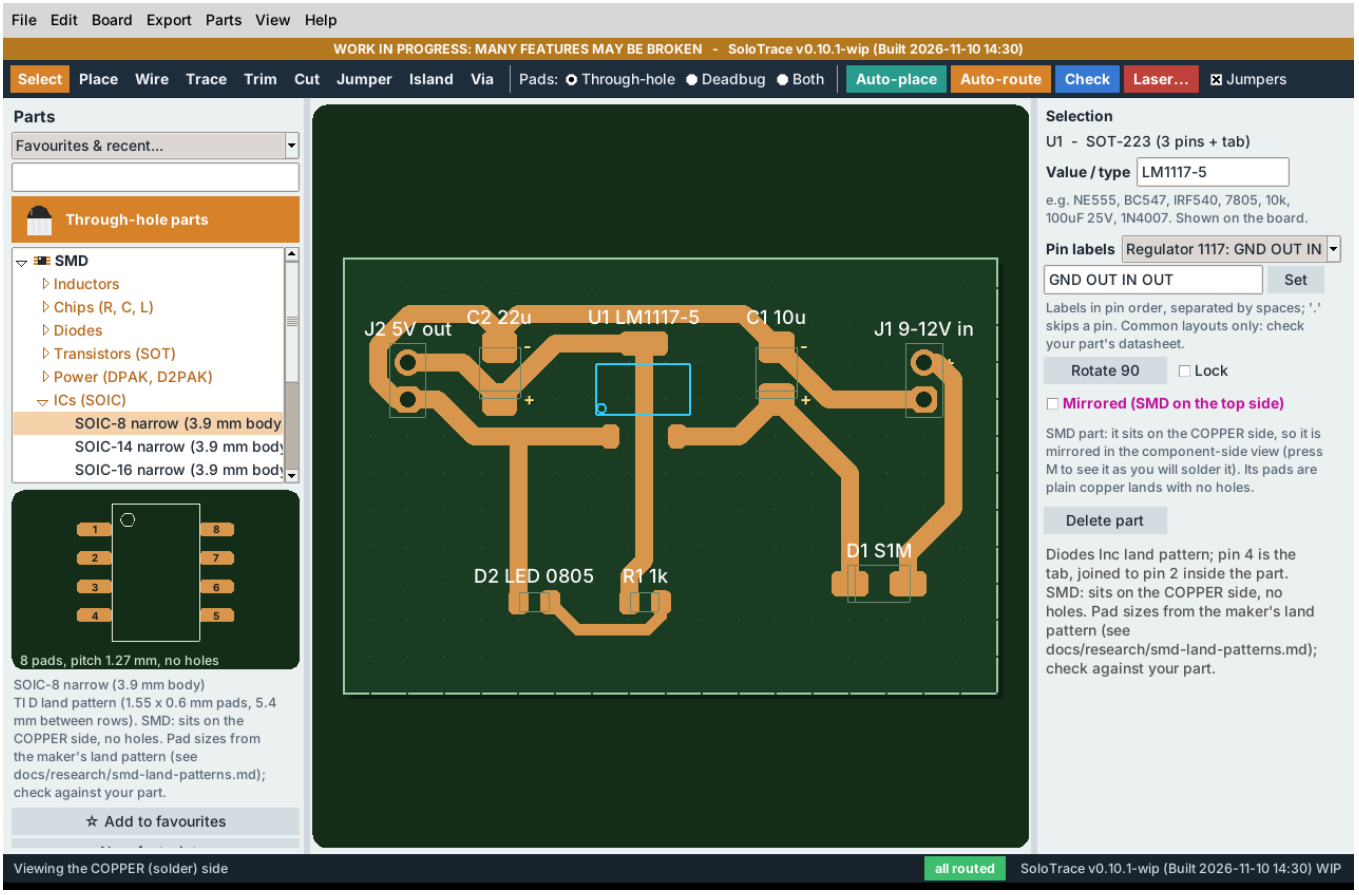
registration must then be within the ring allowance. (2) **Leave it solid**, the ham way: drill every hole, then clear the copper round each non-ground hole with a 2 to 2.5 mm drill tip or countersink; *Export > Countersink list* lists those holes, and nothing needs aligning. Ground leads and vias get thermal reliefs in the plane (same spoke settings) so they can be soldered. *Check* reports a via whose back-side connection a clearance ring would cut, a ground lead trapped inside another hole's ring, a plane nothing joins, and a note on how far the stitching is good for.

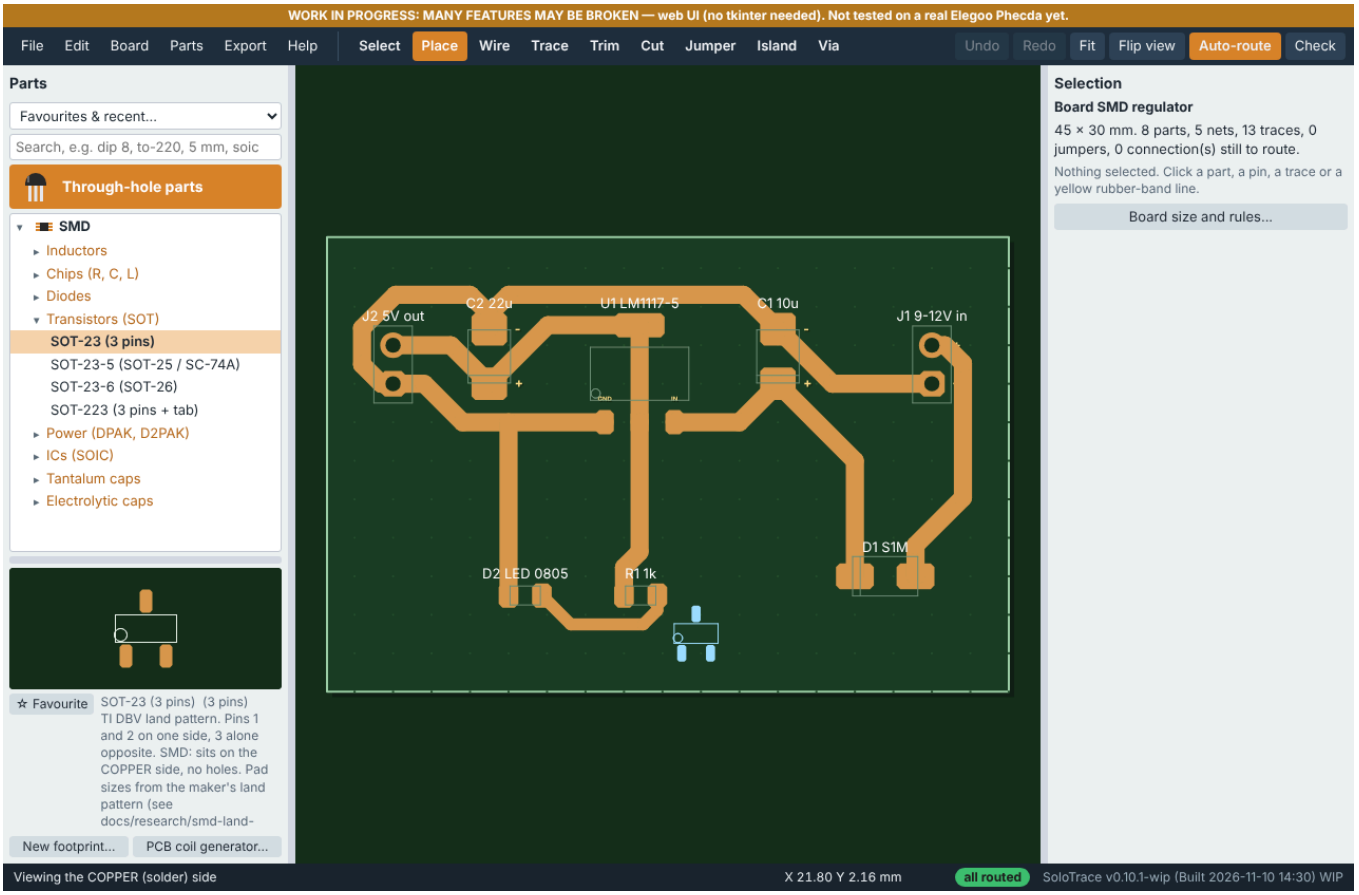

File Edit Board Export Parts View Help
WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN - SoloTrace v0.10.1-wip (Built 2026-11-10 14:30)
Select Place Wire Trace Trim Cut Jumper Island Via
Pads: Through-hole Deadbug Both
Auto-place Auto-route Check Laser... Jumpers
Parts
Favourites & recent...
SMD parts
Chips (DIP)
Transistors & regulators
Resistors, diodes & inductors
Capacitors
LEDs
Pots, switches & crystals
Connectors
Modules & displays
Board bits
14 pads, pitch 2.54 mm, drill 1 mm
Seeed XIAO ESP32-S3 (21 x 17.8 mm) Seeed's own footprint: 2 x 7 at 2.54 mm, rows 15.24 mm apart, USB-C at the top (shown as a box). D0=GPIO1 D1=2 D2=3 D3=4 D4/SDA=5 D5/SCL=6 D6/TX=43 D7/RX=44 D8/SCK=7 D9/MISO=8 D10/MOSI=9. Castellated: with deadbug pads (Pads of this part > Deadbug) you can solder it flat on the copper, or
D1 LED
R1 1k
R3 470
U1 NE555
R2 47k
C2 10n
C1 10u
J1 9V
Selection
Via #1 at 10.16, 2.54 mm
A 0.7 mm hole with a bit of bare wire through it, soldered on both sides: it joins the copper here to the back-side ground plane (net GND).
Delete via
Plane settings...
Click things to select them. Drag a part to move it.
all routed
SoloTrace v0.10.1-wip (Built 2026-11-10 14:30) WIP



SMD parts. The big **SMD parts** button under the parts search (with the little SOT-23 picture) swaps the whole list for the SMD parts; it then reads **Through-hole parts** (with a TO-92 picture) and clicking it again brings the normal list back. The SMD list holds the common, large, hand-solderable surface-mount packages (chips 0603 to 2512; SMA, SMB, SMC, SOD-123, MELF diodes; SOT-23 with 3, 5 and 6 pins; SOT-223; DPAK and D2PAK; SOIC-8/14/16 narrow, SOIC-16 to 28 wide, SOP-8 5.3 mm; tantalum cases A to D; SMD electrolytics 4 to 10 mm; two shielded inductors). Every pad size is taken from a maker's land-pattern drawing (TI, Diodes Inc, Vishay, KEMET, Nichicon, Sumida), using the roomier wave-solder or "level A" pattern where there was a choice, and every gap between pads is at least 0.3 mm, so the parts can be soldered without a solder mask; finer packages (SOT-363, TSSOP, TQFP) are left out on purpose. SMD parts sit on the **copper side**: in the component-side view they appear mirrored, press `M` to see the board as you will solder it; their outlines go to a bottom silkscreen Gerber and they have no holes. Pin-label presets cover SOT-23 transistors and MOSFETs, SOT-223 and DPAK regulators and transistors, and the SOIC equivalents of the DIP layouts. SoloTrace stays a through-hole program first: it starts in through-hole mode and remembers which list you had open.

Where the numbers came from, and what was left out and why: docs/research/smd-land-patterns.md.



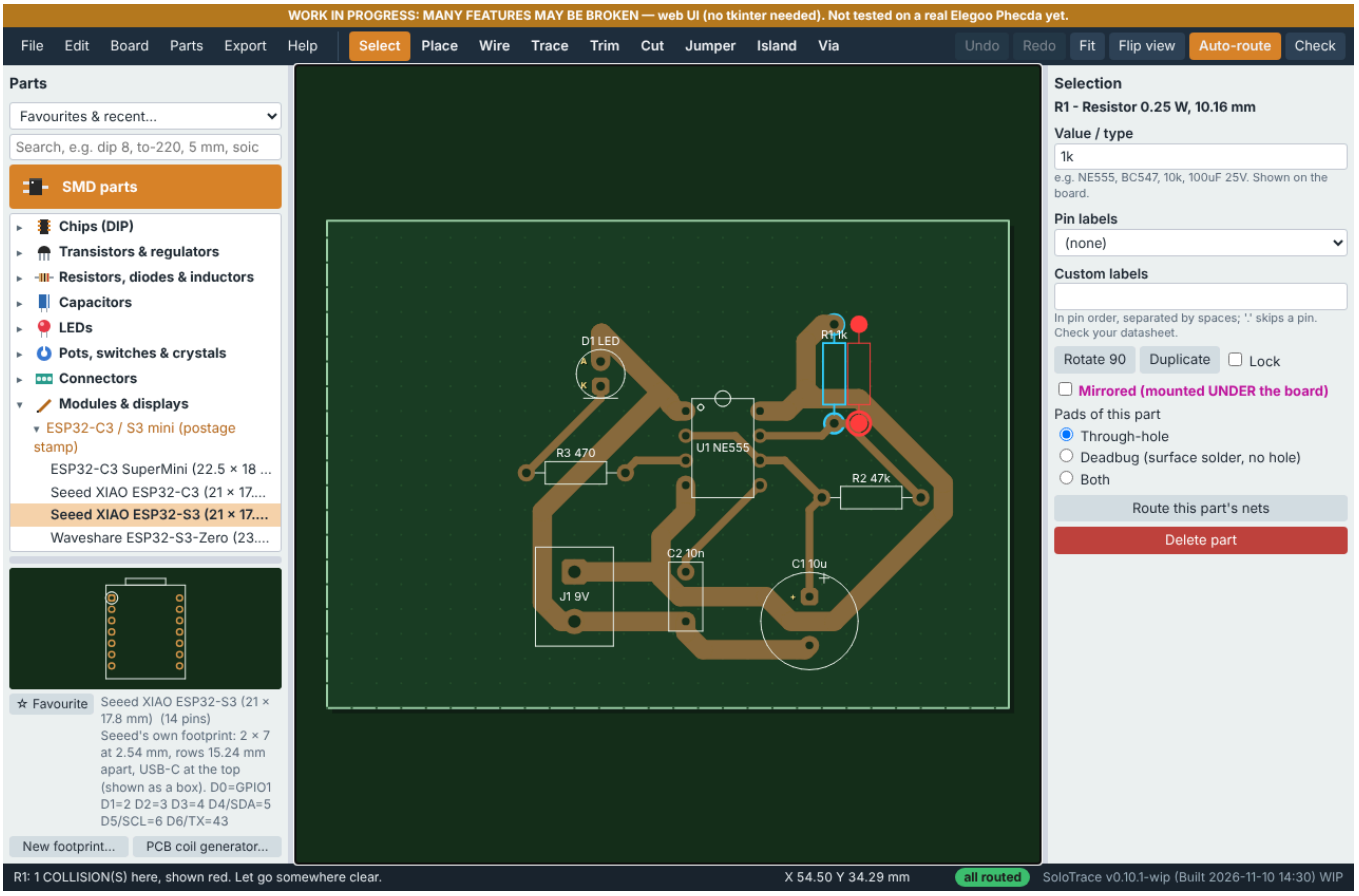


Modules & displays. A new group in the through-hole list holds the boards people actually build with, each on 2.54 mm headers with every pin already labelled as it is printed on the module:

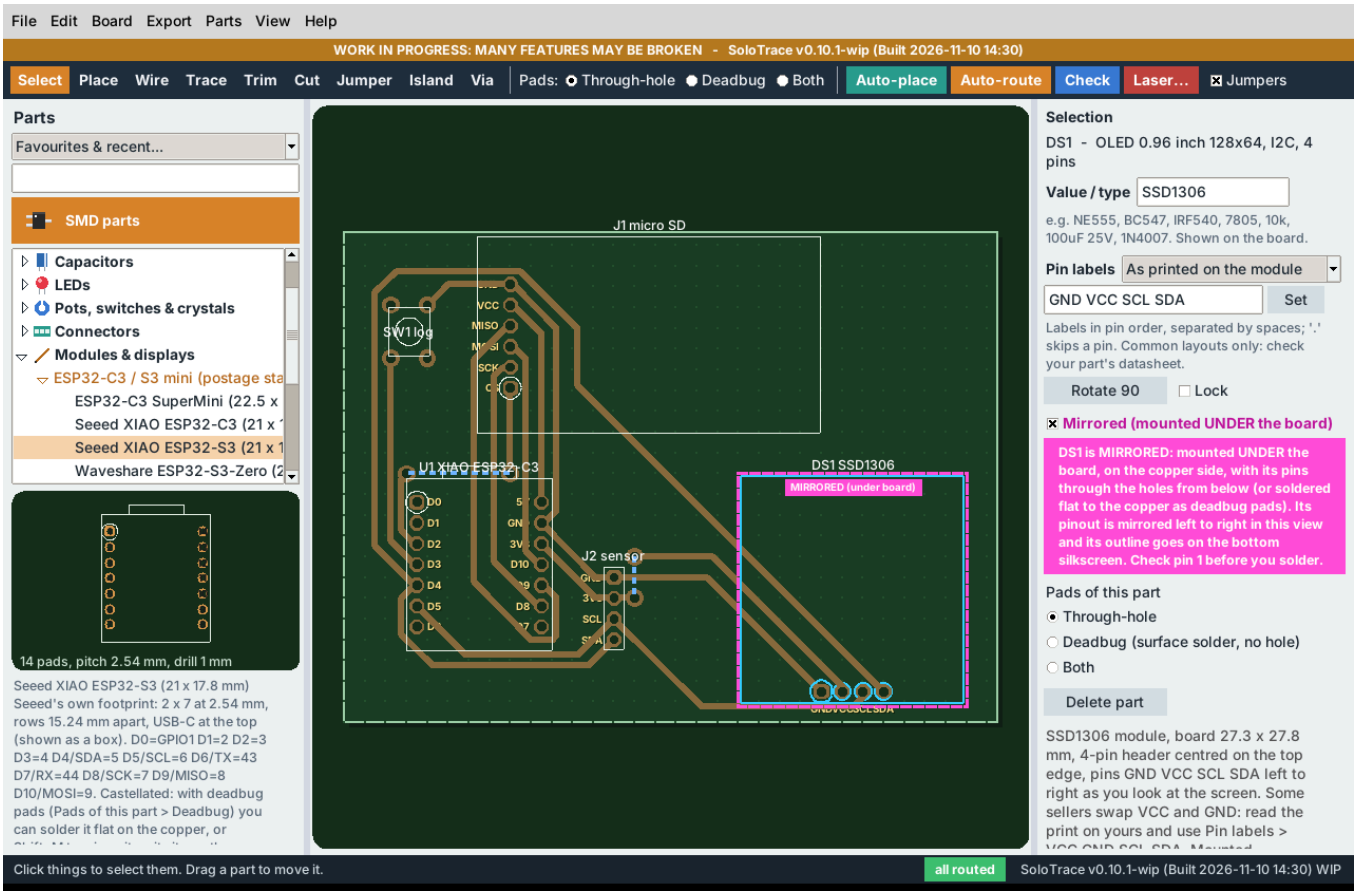
Sub-group	Boards
ESP32-C3 / S3 mini (postage stamp)	Seeed XIAO ESP32-C3 and ESP32-S3 (21 × 17.8 mm, 2 × 7), ESP32-C3 SuperMini (22.5 x 18 mm, 2 × 8), Waveshare ESP32-S3-Zero (23.5 × 18 mm, 2 × 9)
RP2040	Raspberry Pi Pico and Pico W (2 × 20, 17.78 mm rows), Waveshare RP2040-Zero (9 + 9 + 5 along the bottom), Seeed XIAO RP2040
ESP32 dev boards	DOIT DevKit V1 30-pin, Espressif DevKitC 38-pin (both 25.4 mm rows)
ESP8266	NodeMCU v2 Amica (22.86 mm rows), NodeMCU v3 LoLin (27.94 mm rows), WEMOS / LOLIN D1 mini, ESP-01
Arduino	Nano, Pro Mini (with or without the FTDI header), Pro Micro
OLED displays	0.96 inch 128×64 I2C (4 pins), 0.91 inch 128×32 I2C (4 pins), 0.96 inch 128×64 SPI (7 pins)
SD card	the blue 5 V micro SD SPI module (42 × 24 mm) and the small 3.3 V one

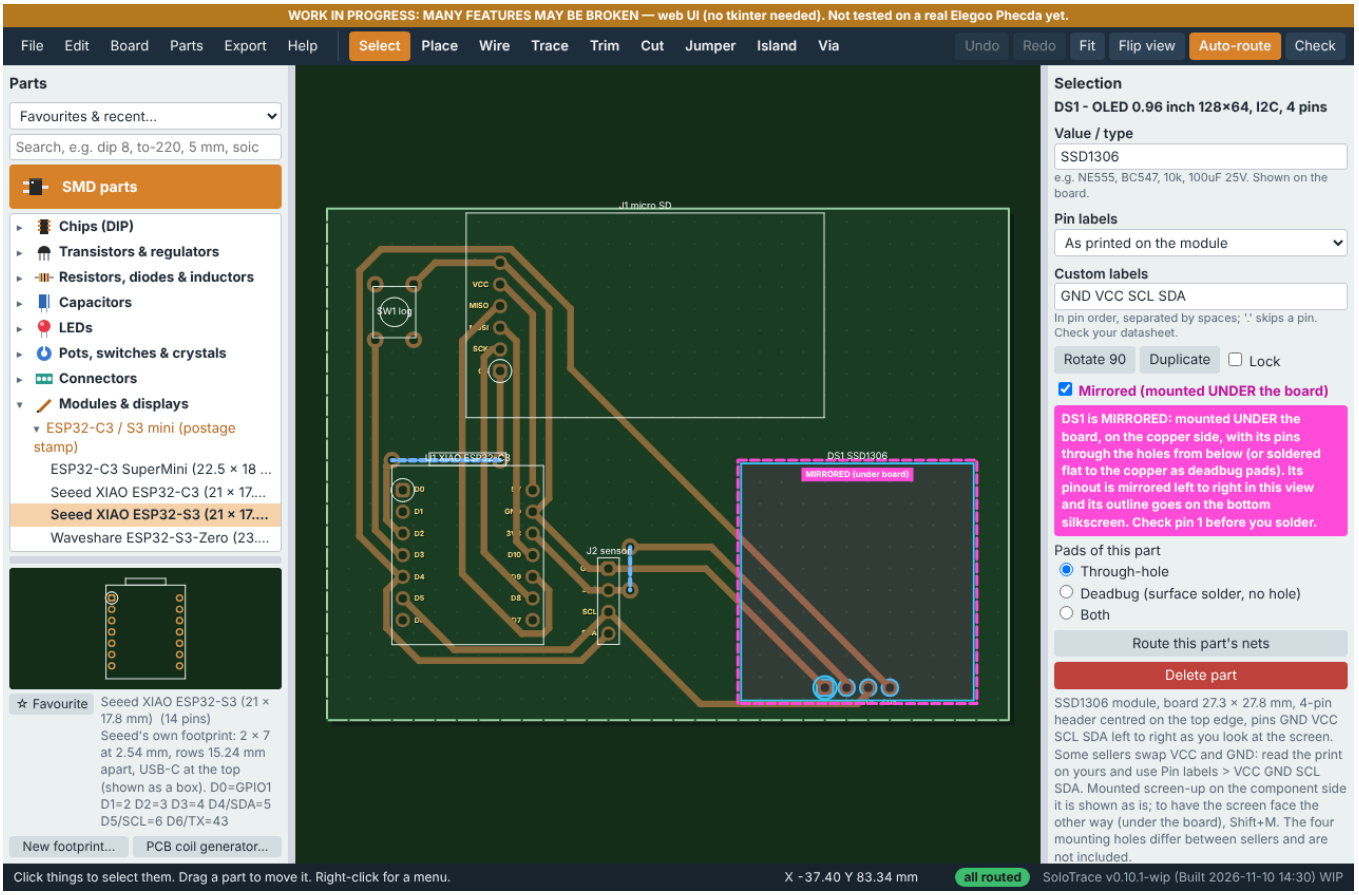
Every board works both ways: through-hole (solder header pins and plug the module in, or solder it straight down), or **deadbug** (*Pads of this part > Deadbug*): no holes, longer pads, so a castellated "postage stamp" board (XIAO, SuperMini, Zero, Pico) can be soldered flat onto the copper, or header pins bent flat. Each part's note gives the GPIO number behind every pin name and says where the data came from; where the sources were thin or disagreed (the SuperMini's exact position along the board, the ESP32-S3-Zero and RP2040-Zero outlines, the DOIT board outline, the SPI OLED's header position, the SD modules' pin order), the note says so. Some sellers swap VCC and GND on the OLEDs: *Pin labels* has *VCC GND SCL SDA* for those. Always check the print on your module before etching. The research, with a source and a confidence word for every number: docs/research/mcu-modules.md.

Moving a part after routing. Traces do not follow a part. When you drag a part over laid copper, every pad and trace it would touch is ringed in red as you move (in the browser the ghost turns red), and when you let go the status bar says how many collisions there are: drag it clear, or right-click it > *Route this part's nets*. The same happens when you turn, nudge, mirror, duplicate or place a part. *Check* also warns when a module mounted under the board and soldered flat would sit on another net's trace.



Mirrored parts (under the board). Shift+M (or right-click > *Mirrored*, or the tick box in the part's panel) mounts a part on the other side: a display that should look out of the copper side, a module plugged in from below so the board sits flat in a box. Its pins swap left to right in the component-side view (the part flips about its pin 1), its outline moves to the bottom silkscreen, the BOM says *copper side (MIRRORED)*, and *Check* lists it as a reminder. While placing, Shift+M places the next part mirrored. A mirrored part is drawn so it can't be missed: a thick dashed **pink** frame, a pink *MIRRORED (under board)* band, pink outline, and a pink note in the panel. (SMD parts already live on the copper side, so they are only shown pink if you mirror one onto the component side.) Example: *File > Examples > XIAO ESP32-C3 logger*, with its OLED mirrored under the board.



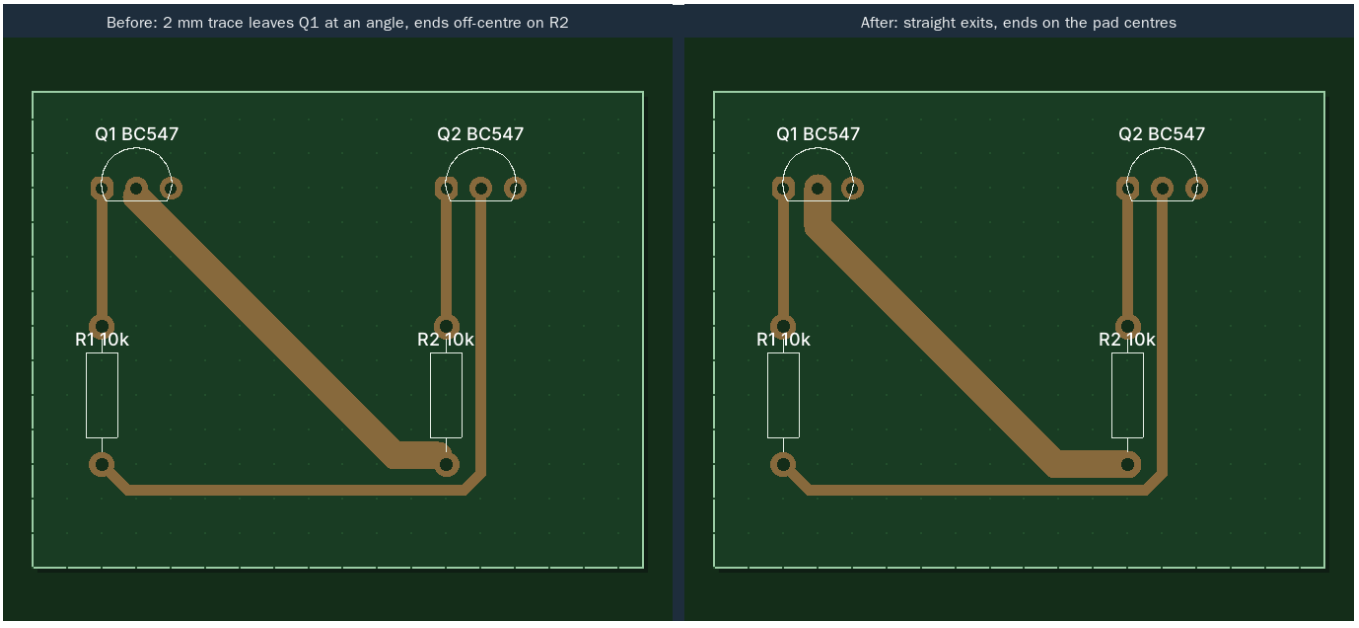


Values and pin labels. Select a part: *Value / type* at the top of the right panel takes anything (NE555, BC547, IRF540, 7805, 10k, 100uF 25V) and is shown next to the part name on the board; double-click a part to jump to it. *Pin labels* offers the common layouts for that shape, not specific part numbers:

Shape	Layouts offered
TO-92	E B C, C B E, B C E, D G S, G D S, S G D, small regulator OUT GND IN / IN GND OUT
TO-220 3-pin	power transistor B C E, MOSFET G D S, IGBT G C E, regulator IN GND OUT, adjustable regulator ADJ OUT IN, negative regulator GND IN OUT, triac MT1 MT2 G, dual diode A K A
TO-220 2-pin	diode K A / A K
TO-247	G D S, G C E, B C E
DIP-4 / DIP-6	optocoupler A K E C; optocoupler with base pin; opto-triac
DIP-8	555-style timer, dual op-amp, single op-amp, dual optocoupler
DIP-14 / 16 / 20	logic GND and VCC pins; quad op-amp (DIP-14)
Pots, connectors	1 W 3, CCW W CW; + -, - +, AC AC, IN OUT, V+ GND V-, IN GND OUT

Or type your own in pin order, separated by spaces (`.` skips a pin). Diodes and LEDs show K A, electrolytics + -, without asking. Pin orders differ between makers, so **always check the datasheet** of the part you have. Labels are shown in the editor; they are not yet written into the Gerber silkscreen.

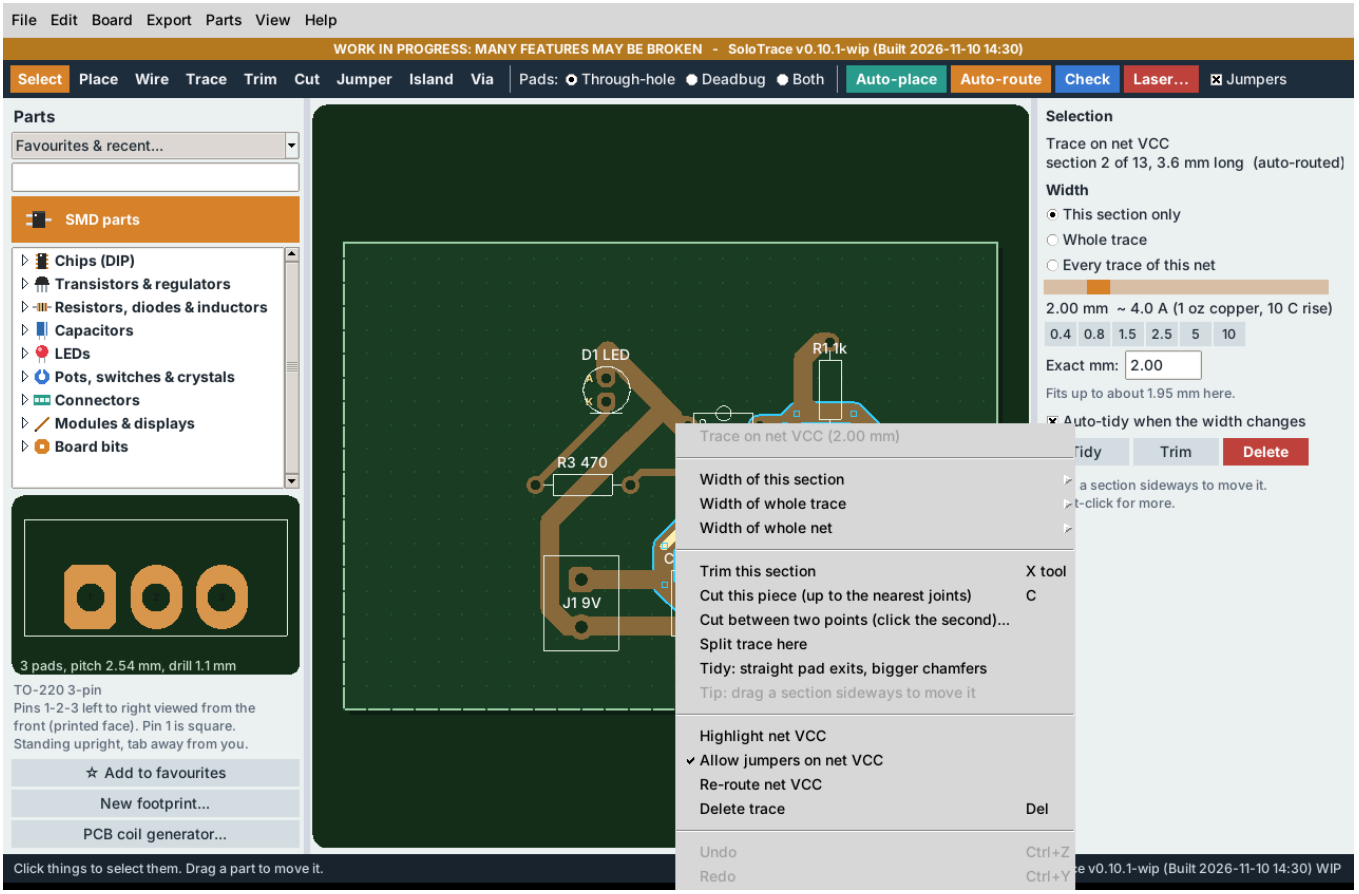
Wide traces are tidied. When a trace is 1.2 mm or wider, SoloTrace makes it leave each pad with a straight vertical or horizontal run (about the pad radius plus the trace width) before it turns, ends it exactly on the pad centre, and makes the chamfers bigger where there is room. This happens after auto-routing and, with *Auto-tidy when the width changes* ticked (the default), whenever you change a width. If the tidier shape would touch other copper the trace is left as it was, and the clearance problem is marked in red. *Tidy* (right panel or right-click) does it on demand; *Edit > Tidy all traces* does the whole board.



Cutting a piece out of a trace. Press `C` and click a trace: the piece under the mouse goes, from the joint before it to the joint after it (a pad, a place where another trace meets it, a jumper pad). The rest of the trace and of the net stays. While you hover, the piece that would go is shown in red. Right-clicking a trace only opens its menu; rest the mouse on *Cut this piece* to see the piece in red, and choose it to remove it. To cut between two points of your own choosing, Shift + click the first point and Shift + click the second (or right-click > *Cut between two points*); `Esc` cancels.

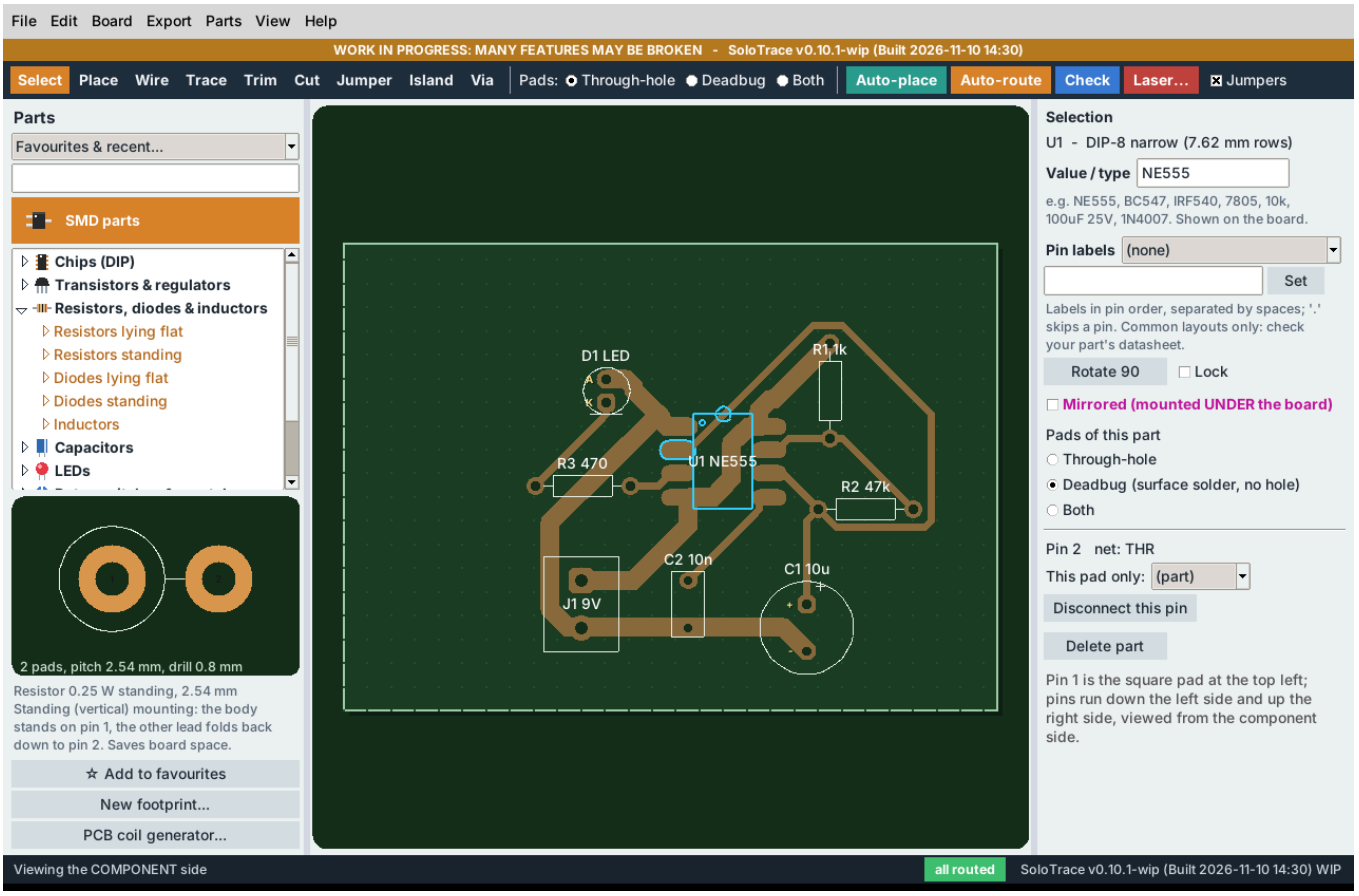
Moving, trimming and splitting traces. With *Select*, drag a trace section sideways: it moves parallel to itself and the sections either side stretch to follow at their own angle, so no 90° corner appears. Ends on pads stay put (a 45° link is added). **Trim** (`X`, or right-click > *Trim this section*) cuts one section out and leaves the rest as separate pieces; *Split trace here* breaks a trace in two without removing copper. A trace you have moved or trimmed counts as hand-drawn, so Auto-route keeps it. `Ctrl+Z` undoes any of this.

Right-click menus. Right-click (without dragging) on a trace, pin, part, rubber-band line, jumper or empty board for the actions that fit: widths, trim, split, tidy, highlight or re-route the net, rotate either way, duplicate, lock, pad styles, wire or trace from a pin, disconnect, delete, and Undo/Redo. Right-drag still pans.



Small helpers. Hover over copper to light up its whole net (the net name shows in the corner). Arrow keys nudge the selected part by 1.27 mm (`Shift`: 2.54 mm), `Ctrl+D` duplicates it, `Shift+R` turns it back, `Shift+M` mirrors it (mounts it under the board). *File > Open recent* lists your last boards. While a board has unsaved changes it is saved every 2 minutes to `~/.solotrace/autosave.solotrace`; *File > Recover autosave* opens it (the file is removed when you quit normally). Hover over a toolbar button for a hint. The status bar shows the cursor position in mm and a green *all routed* or orange *N to route* badge.

Pads: through-hole or deadbug. Pick *Through-hole*, *Deadbug* or *Both* in the toolbar for new parts, or change it for a placed part in the right panel, or for one pad ("This pad only"). Deadbug pads have **no hole** and are drawn as a long oval that sticks out from the part so you can bend the lead flat and solder it on the copper side. "Both" keeps the hole *and* the extra copper.



Auto-place. Simulated annealing that moves, rotates and swaps unlocked parts to shorten the ratsnest and cut crossings. Tick **Lock** on a part to keep it where it is. *Board > Auto-place algorithm* also offers **Routability** (experimental): the same moves plus an estimate of where copper will not fit (RUDY congestion) and a penalty for pads boxed in by other parts, with VPR's adaptive cooling. In measurements it was **no better** than Annealing (DESIGN_NOTES.md), so Annealing stays the default.

Auto-route. A grid router that remembers its heading, so it prefers straight runs and clean 45° bends instead of staircases, and prefers not to run under part bodies. Three ways to drive it (*Board > Auto-route algorithm*):

- **Auto** (default): negotiated congestion first; if that leaves a connection unrouted or needs a jumper, the classic tries follow within the time budget, and the best result is kept.
- **Negotiated congestion**: the PathFinder method FPGA tools use. Every net is routed at once, overlaps allowed; then the price of every contested cell rises pass after pass (and a history cost remembers where fights happened) until the nets have sorted themselves out, usually in 3 to 15 passes. A jumper is just another priced resource, dear until contested copper is dearer. It does not depend on the net order, so it needs no retries. On eleven test boards: everything routed, the same number of jumpers as classic, 7 to 9 % less copper, about two-thirds of the time.
- **Classic**: one net at a time against hard obstacles, wide nets first; a blocked net tears up the nets in the way and retries; a jumper is the last resort (unless **Jumpers** is unticked in the tool bar or for that net). The job is tried with up to six net orders (about 12 seconds at most, plus the try in progress) and the best result is kept.

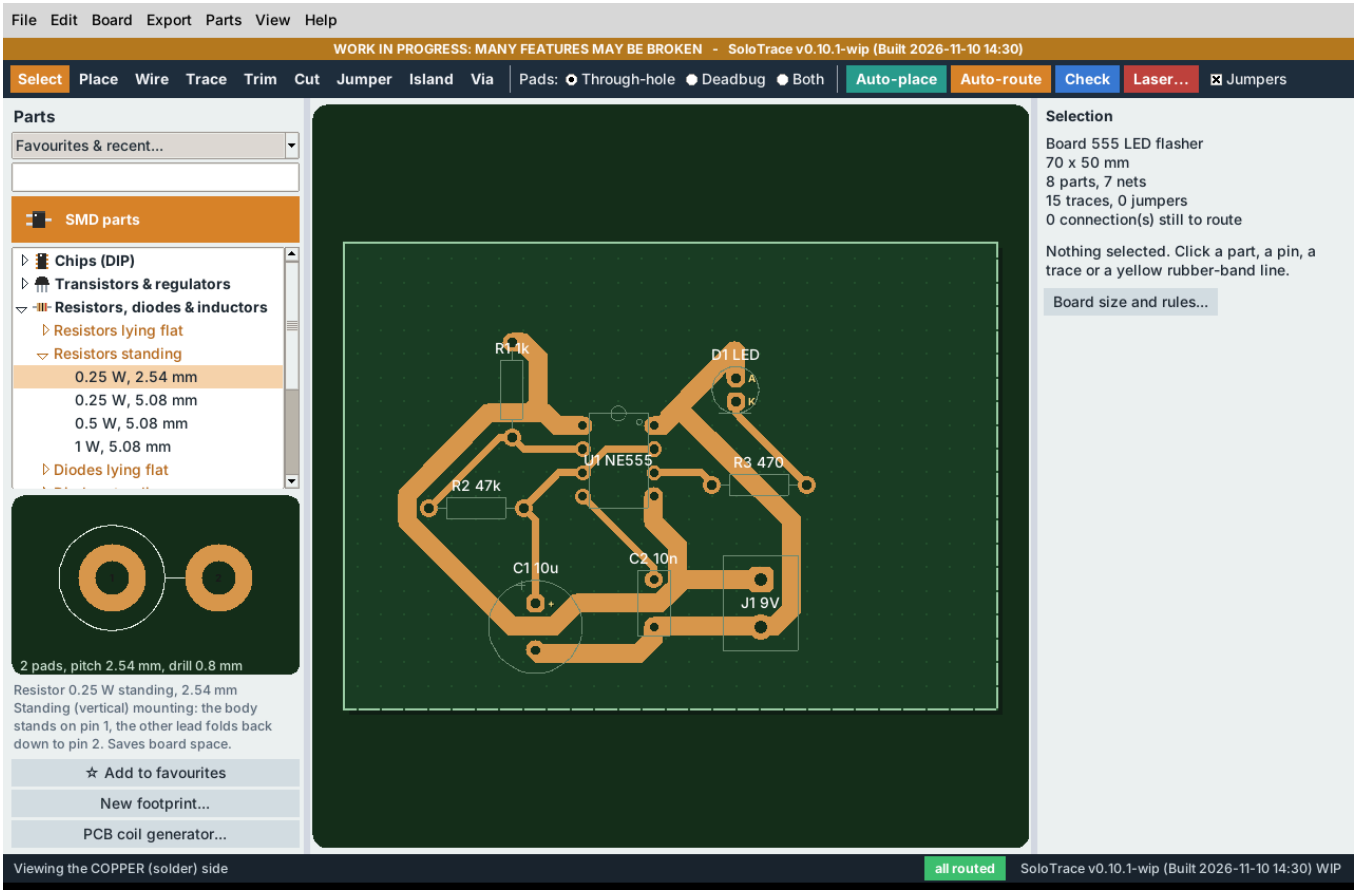
After the best try is chosen, a **polish** step routes each net once more against the finished board and keeps the new copper only if it has fewer jumpers or is shorter, so it never makes a board worse. The same board always routes the same way (before 0.8.0 it could differ from run to run). How it

compares with the first versions: DESIGN_NOTES.md (fewer jumpers than 0.3.4 on the test boards, the same total copper; sometimes a longer path where it saves a jumper).

Traces you drew by hand are kept by all three. *Board > Remove automatic routes* undoes only the automatic ones.

Jumpers. Press `J` to add one by hand. They are plain wire links on the component side, in 2.54 mm steps from 5.08 mm. They are listed in the DRC window.

View. `M` flips between the **component side** (how you place parts) and the **copper side** (how the etched board looks from below). Wheel zooms, right or middle drag pans, right-click opens the menu, `F` fits the board.



DRC. *Check* lists trace ends that are not joined to anything ("dangling"), copper too close to other copper or the board edge, parts sticking out of the board, traces thinner than the minimum, turns sharper than 45°, drilled pads with too little copper ring, jumpers that are the wrong length or cross each other, and pins that are not connected yet. Click a row to jump to it on the board.

Exports (Export menu).

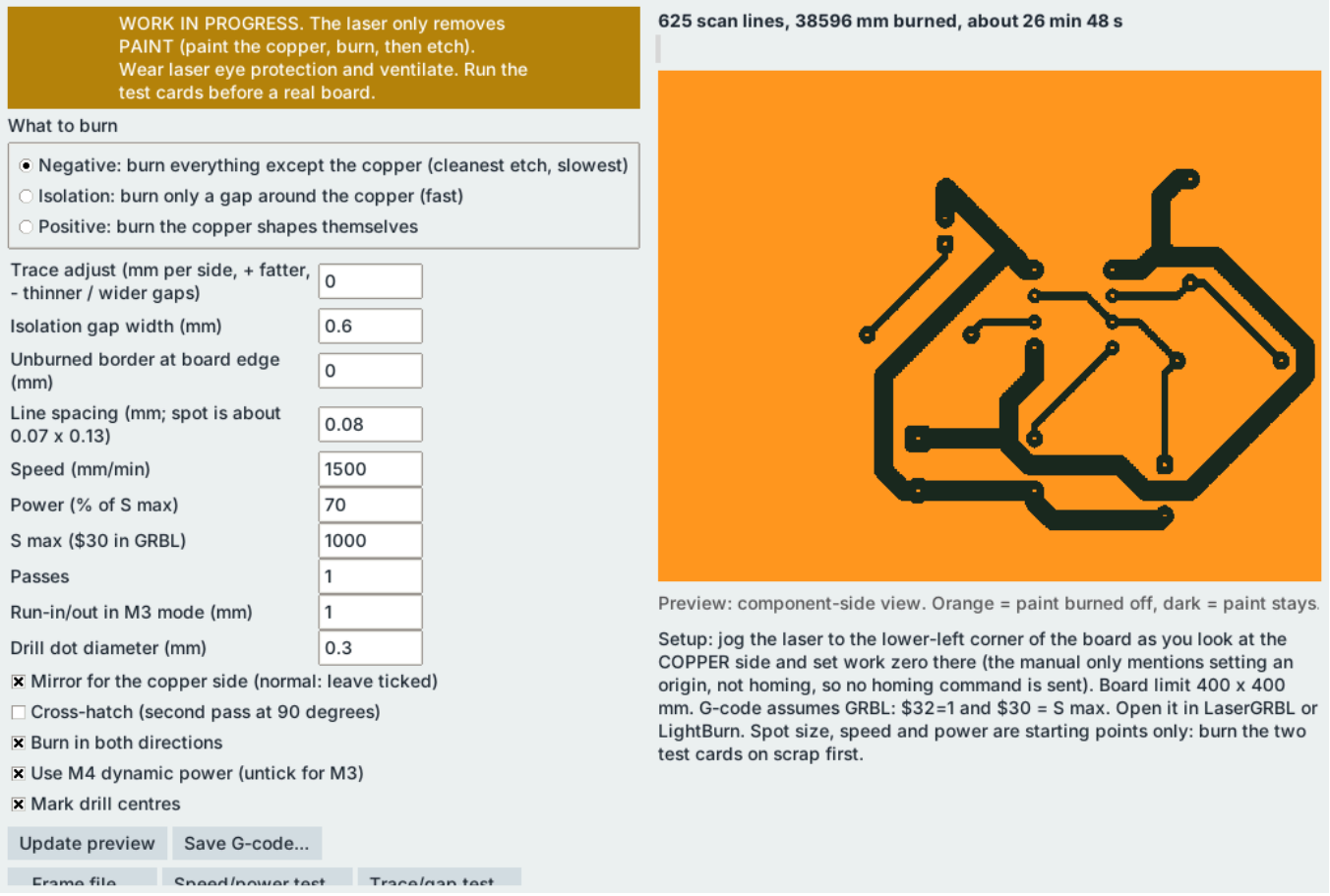
Output	Use
Gerber + drill (zip)	Copper (bottom), board edge, silkscreen, Excellon drill. Single layer; **no soldermask layer**
1:1 PDF / SVG, mirrored	Toner transfer or photo-resist film: the copper side as you will see it. Always print at 100 % and **measure a known distance** before etching
1:1 PDF, design view	Component-side print
Bill of materials (CSV)	Parts grouped by name and value
Netlist text	Round-trip text format for a quick board from a text editor

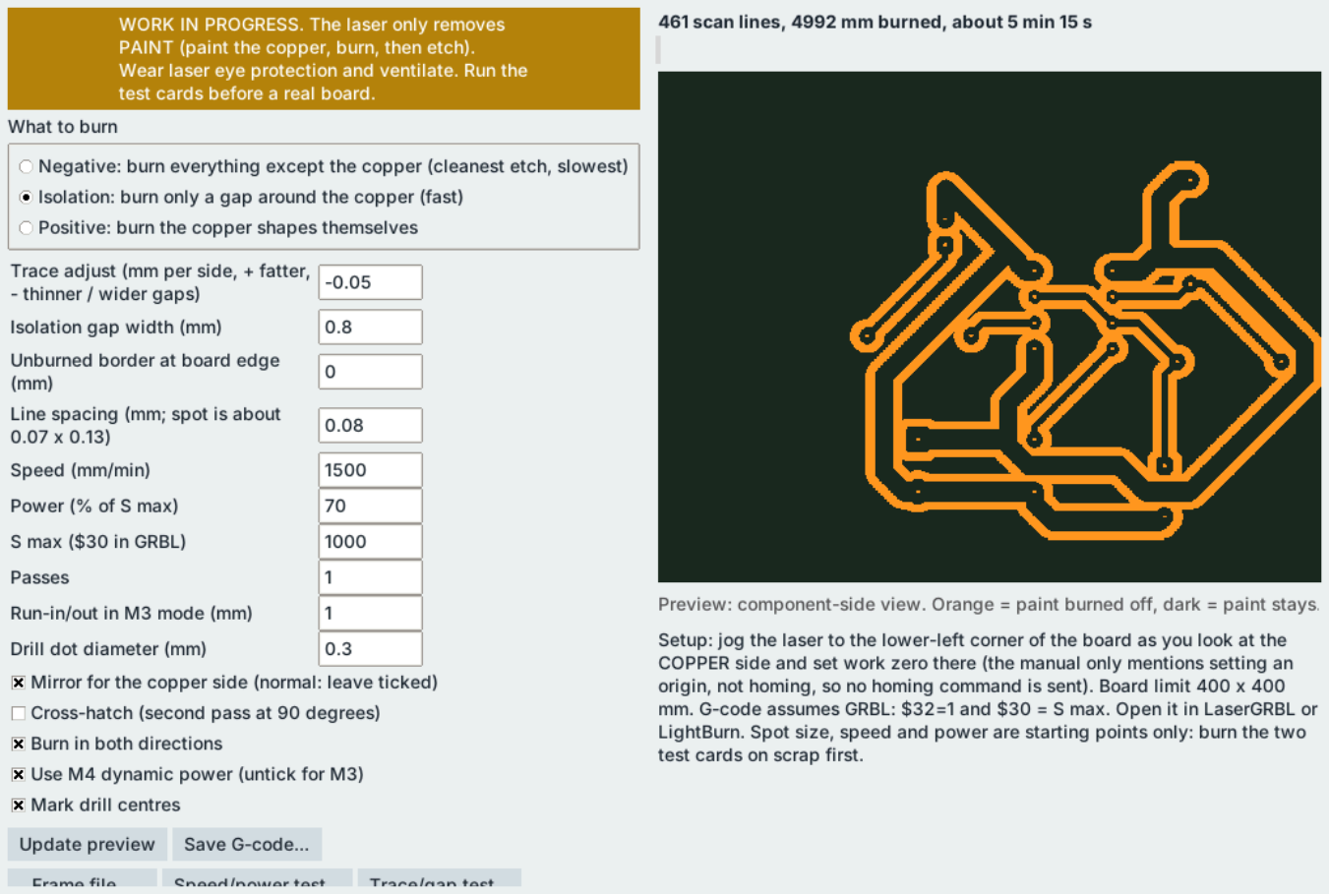
Output	Use
Laser G-code (Elegoo Phecda)	See below

Laser etching with the Elegoo Phecda (gantry diode laser)

Read this first. A diode laser cannot remove bare copper. The method is: **paint** the copper, let the laser **burn the paint away** where copper must go, **etch**, then clean the paint off. Nobody has run these files on a Phecda yet. Treat every default as a starting guess and use the test cards.

1. Finish the board and run DRC. Open **Export > Laser G-code (Elegoo Phecda)...**
2. **Mode** - **Negative** (default): burns everything that is *not* copper, so paint is left only on the traces and pads. Slowest, cleanest. - **Isolation**: burns only a band of `gap` mm around the copper. Fast; leftover paint between bands must be cleared by hand or by the etchant. - **Positive**: burns the copper shapes themselves (for a negative-working resist or marking).
3. **Adjust** (all live-previewed): - *Trace adjust* (mm per side): positive grows the copper, negative thins it and widens every gap between traces. Use this to correct for the laser's spot size and your etch. - *Gap* (isolation band width) and *Border* (keep paint at the board edge). - *Line interval* (default 0.08 mm; the module's spot is about 0.06–0.13 mm depending on model), *feed* (default 1500 mm/min), *power* (default 70 % of S max), *passes*, *crosshatch* (second pass at 90°), *bidirectional* scanning, *M4 dynamic power* (default) or *M3 with overscan*, and *drill dots* (a tiny burn at every hole centre as a drilling guide).
4. Click **Save G-code...**. Also use **Frame file...** (traces the board outline at 1 % power so you can line up the paint) and, on first use, **Speed/power test...** and **Trace/gap test...** (a grid of squares and a ladder of fine traces and gaps).
5. In LaserGRBL or LightBurn: jog the laser to the **lower-left corner of the board as seen from the copper side**, set work zero there, open the file, check the preview, run the frame, then burn. The exporter mirrors X for you because the design view is the component side.
6. Controller assumptions: GRBL-style, `$30` equal to the S max in the dialog (default 1000) and `$32=1` (laser mode). The Elegoo manual names LaserGRBL and LightBurn but does not say "GRBL"; see docs/research/phecda-laser-notes.md.





Examples to look at without installing anything: examples/ has eleven boards (the 555 flasher, 5 V regulator, 555 power oscillator, logic mix, RF oscillator with ground fill, Manhattan RF amplifier, SMD regulator, a XIAO ESP32-C3 logger with its OLED mirrored under the board, the 555 flasher with

ground fill *and* back-side plane, a 433.92 MHz meander antenna board, and a 1.5 uH coil from the generator), each with its `.solotrace` file, Gerber zip, 1:1 copper-side SVG and PDF, BOM and laser G-code in both modes; the back-plane one also has the back-plane PDF/SVG, its laser job and the countersink list. Plus test cards.

Troubleshooting

Problem	Try
Double-click does nothing on Linux	Right-click > Properties > Permissions > allow executing; or `bash SoloTrace-Linux.sh`
"tkinter is missing"	Only `main.py desktop` needs tkinter; everything else runs in your browser. For the desktop window: `sudo apt install python3-tk` (Linux), reinstall Python with "tcl/tk" ticked (Windows), install Python from python.org (macOS)
The web UI page says the server did not answer	The local server stopped (Ctrl+C in the terminal, the computer slept for a long time, or the page was closed for more than a minute and a half). Double-click the launcher again; an unsaved board is autosaved every 2 minutes (*File* > *Recover autosave*)
The in-browser version stays on "Downloading Python"	It needs to reach `cdn.jsdelivr.net` once (a firewall, an ad blocker or a school network may block it). Use the download and a launcher instead
A part I moved now sits on a trace	Traces do not follow a part. The red marks and the status bar say which pads collide; drag it clear, or right-click it > *Route this part's nets*
The browser blocks the download when you Save	Allow downloads from `127.0.0.1` in the browser's site settings; the file goes to your downloads folder
macOS says the file is from an unidentified developer	Right-click the `.command` file > Open > Open (first time only)
Window looks cut off	Maximise it or make it larger; the panels were laid out on a 1280 × 800 window
A wide trace still leaves its pad at an angle	The straight exit did not fit next to the neighbouring pads (common on 2.54 mm pitch parts above about 2.5 mm width). Make it narrower near the pad (*Width of this section*) or move the part, then *Tidy*
Dragging a trace does nothing	Use the *Select* tool and press on the trace itself (not on a pad); drag sideways, the section only moves at right angles to itself
Right-click pans instead of opening the menu	Click without moving the mouse; a right-drag pans
A trace runs straight through the middle of a chip	Tick *Board* > *Keep traces out from under parts* and route again (it may need a jumper instead)
Check says "pad ... touches island #n"	That island has pins of two nets on it (a short). Make it smaller, move it, or split it into two islands with a gap
An island says "not joined to anything yet"	No pin, trace or jumper pad touches it. Drag it under a pin, or make it bigger
Ground fill: a yellow line from a ground pin	That pin sits on a piece of fill cut off by other nets. Join it with a trace or a jumper, or move parts so the fill can reach it
Ground fill: big bare areas	Fill that no ground pin sat on was cut away (floating copper). Untick *Remove pieces of fill...* in *Board* > *Ground fill settings* to keep it
A short stub hangs off a trace	*Check* lists it as "dangling"; use Cut or Trim to remove it, or re-route the net

Problem	Try
Auto-route adds jumpers you do not want	Untick **Jumpers** in the tool bar (whole board) or in the net panel (one net), then route again; finish what is left by hand
Auto-route leaves nets unrouted	Make the board bigger, lock fewer parts, run Auto-place again, lower the width of wide nets, or finish by hand with the Trace tool; unrouted nets stay as yellow lines
Drawing feels slow	Update to 0.2.1 or later (only the moving layer is redrawn now). Turn off *View > Show grid* on very big boards
Auto-route is slow	It is pure Python; boards over about 40 parts can take tens of seconds on a slow CPU. Press Cancel, or route a few nets at a time
Print is the wrong size	Print at 100 % / "actual size", no "fit to page"; check a known distance with calipers
Printed pinout is mirrored	Use the **mirrored** PDF for toner transfer; the design view is the component side
Laser burns the wrong place or mirrored	Work zero must be the lower-left of the board as seen from the **copper** side; check the preview in your laser software
Laser power looks wrong	Make `$30` match the S max in the dialog and set `$32=1`
Fab rejects a Gerber	Tell us with an issue and attach the zip; only an independent Python Gerber reader has been used so far

Known limitations

- **0.10.0:** the antennas and the coil have never been etched or measured; their lengths and inductance come from formulas (rules of thumb for the antennas). There are no matching-network parts, no inverted-F, patch, chip or loop antennas, and no module-specific keep-outs (use a keep-out box sized from the datasheet). The custom-footprint text editor drops the strokes, keep-outs and ties of a part it re-saves. The coil's wire between pins 2 and 3 must be insulated and is not drawn on the board.
- **0.9.0:** the three breakout boards and the hole strips have never been etched, soldered or used with a real module. The green wire-lock circle has been tested by unit tests and scripted browser events, not used by a person. A pad's extra holes (`holes`) are kept in files and built-in parts but are dropped if you edit that footprint in the custom-footprint text editor.
- **Work in progress.** Expect bugs. Back up your boards (`.solotrace` files are plain JSON).
- Laser output is untested on hardware; speeds, powers and spot sizes are assumptions. See DESIGN_NOTES.md.
- The router is pure Python. The logic demo needs 5 jumpers (negotiated) to 7 (classic) depending on the placement; a 30-part random board about 14 to 16. Auto-route can take 15 to 30 seconds on a cramped 20 to 30 part board on the development machine, more on a slow laptop (**not measured on a 7th-gen Core i3**); press Cancel to keep the best result so far. The negotiated router has no history cost between separate runs and no topological (rubber-band) stage; see DESIGN_NOTES.md.
- Deadbug pads and body outlines are approximations. Check clearances with your real parts.
- No soldermask output and no text on copper.
- The width slider is in the right-hand panel, not a floating pop-up.
- Dragging moves one section at a time; there is no push-and-shove of neighbouring traces, so a dragged trace can end up too close to others (it is marked in red).
- The tidy step only reshapes the first few sections at each end of a trace; it does not re-route.

- A trace cannot be thinner near a pad than elsewhere when auto-routed ("neck-down"), so a power net wider than about 2.4 mm cannot leave the middle pin of a TO-220 or TO-92 (2.54 mm pitch) with the 0.3 mm clearance: the net stays unrouted. Use 2 mm for such nets, or narrow that one section by hand.
- Thermal reliefs, the back-side ground plane and the SMD parts are new in 0.5.0 and have not been etched, soldered or built on a real board. Home-made vias and the plane's clearance rings assume the registration and hole sizes in the research notes; measure yours.
- SMD land patterns are the makers' recommended ones but have not been checked against real parts here; the two inductors and the SMD electrolytics in particular vary between makers (the research note says which numbers were read off drawings without labels). There is no exposed-pad (PowerPAD) footprint: a pad under the part cannot be soldered from the copper side.
- The back-side plane is always a full ground plane; nothing else can be routed on the back.
- Copper islands and the ground fill are new in 0.4.0 and have not been etched or lasered on a real board. Islands are only rectangles (with cut corners), circles and ovals; there are no other shapes yet, and no text on copper.
- With the ground fill on, the editor shows the fill at once but removes floating copper about 0.4 s after you stop editing (on a 160 × 100 mm, 30-part test board that check took about 1 s on the development machine; expect longer on a slow laptop). Check and exports always wait for it.
- Auto-place does not move copper islands, so on a board built from islands it puts pins off their islands; it asks before it starts.
- Pin labels and values are not yet part of the silkscreen in the Gerber, PDF or SVG exports.
- Only the tests and the screenshots have exercised the GUI; Windows and macOS have not been run at all.
- The module footprints have been checked against makers' files and the tests, not by plugging real modules into an etched board. The sources disagree on a few (see the part notes); measure yours. Mounting holes of the OLED and SD modules are not included (the sources disagree or have none).
- The in-browser version (0.8.0) runs Python in WebAssembly: everything is slower than the download, auto-route about two to three times slower and with no Cancel button (the page waits until it is done), and the first visit downloads about 12 MB from the jsDelivr CDN. Loading from the CDN itself has not been tried (only an identical local copy of the Pyodide 0.29.5 package); only Chromium has been tested, not Firefox or Safari. If the browser clears its site data, the autosave and your own parts go with it: save your boards as files.
- Moving a part never moves its traces: a part dragged onto another net's trace is a short until you re-route or redraw (0.8.0 marks it at once). There is no push-and-shove.
- The web UI is new in 0.6.0 and has only been driven by tests and screenshots in headless Chromium; no one has yet designed a real board in it. It draws a ghost while you drag and applies the move when you let go (the desktop window moves things live); the custom footprint editor is a text form rather than a click-to-add-pads canvas; there is no floating width pop-up; and an unsaved board lives in the server, so closing the browser tab for more than about a minute and a half loses anything after the last save or autosave.

More

- HOW_IT_WORKS.md: what it reads and writes, the coordinates, the router and the laser engine in plain English
- docs/research/breakout-boards.md: the breakout boards and hole strips

- DESIGN_NOTES.md: decisions, rejected ideas, measurements, what is untested
- docs/research/: the original research and plan, the Phecda notes, and the 0.5.0 notes on autorouters, thermal reliefs, the back-side ground plane and SMD land patterns
- CHANGELOG.md · CONTRIBUTING.md · SECURITY.md

Project page (after you enable GitHub Pages): `https://The-Dorkknight.github.io/solotrace/`

How SoloTrace works

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN. This page explains in plain English what the program reads, what it writes, and what it never does. For the reasoning behind the choices see DESIGN_NOTES.md.

What it reads and writes

It reads	When
`.solotrace` board files	File > Open, or a path given on the command line
Netlist text files	File > Import netlist text
KiCad `.kicad_mod` footprints	Parts > Import KiCad .kicad_mod
Your own parts in `~/.solotrace/parts/*.json`	At start
`~/.solotrace/config.json`	At start (notice accepted, last laser settings, recent files, favourites, which parts list was open)
`~/.solotrace/autosave.solotrace`	Only when you pick *File > Recover autosave*

It writes	When
`.solotrace` board files	File > Save (a board is plain, readable JSON that also embeds its part shapes, copper islands and ground-fill settings, so it opens anywhere)
Gerber zip, PDF, SVG, BOM CSV, netlist text, G-code files	Only when you pick an Export menu item and choose a file name
`~/.solotrace/parts/*.json`	When you save a footprint in the wizard
`~/.solotrace/config.json`	When you accept the first-start notice, export laser code, or open/save a board (recent files list)
`~/.solotrace/autosave.solotrace`	Every 2 minutes while the board has unsaved changes; deleted when you quit normally

It **never** connects to the internet, never talks to your laser or your printer (you open the G-code in your own laser software), never installs anything, and never changes files you did not ask it to save.

The web UI (the default since 0.8.0)

When you run `python3 src/main.py` or a launcher, SoloTrace starts a small HTTP server from the Python standard library on `127.0.0.1` (a random free port, or `--port N`) and opens `http://127.0.0.1:<port>/` in your browser. It only answers requests whose `Host` header is the local address, so nothing else on your network can reach it. The page (`src/solotrace/web/static/`) is plain HTML, CSS and JavaScript with no libraries; it asks the server for the picture of the board as data (every piece of copper, the outlines, the rubber bands) and draws it on a canvas, and sends every click, key and menu choice back as a command. The server holds the board, the undo stack and the tool state in `session.py`, which is the desktop editor with the window taken out: the same board, tidy, ground-fill and plane code answers both. Differences from the window: the page shows a ghost while you drag and the move is applied when you let go; *Save* is a download (the browser's downloads folder) and *Open* reads a file you pick; a board opened from a path on the command line can also be saved back

to it (*File > Save to the opened file*). The page pings the server every 10 s; 90 s without a ping (the tab was closed) and the server stops, as does *File > Quit*. The autosave file and `config.json` are the same ones the window uses.

The board

- Units are **millimetres**, Y points **up**. The screen normally shows the **component side, looking through the board**. Copper is on the other side.
- Parts sit at grid points (1.27 mm by default). Pins of DIPs and headers are 2.54 mm apart, and the router grid is 0.635 mm (a quarter of 2.54) so everything lines up.
- A **net** is a list of pins that must be connected, plus any **rigid pairs** (two pins of the net that must be joined directly). The wire tool adds pins to nets, and a yellow rubber-band line shows what is not yet routed.
- A **trace** is a list of points with one width per segment. Every segment is a straight capsule (a line with round ends), and a pad is a circle, oval or square polygon. Copper is simply the union of those shapes.
- **Chamfers:** whenever two segments meet at a corner, the corner is replaced by two shorter bends of 45° each. The chamfer leg is at least 0.9 mm or 1.5 × the trace width, whichever is larger, so a 10 mm trace gets a 15 mm chamfer. The route checker complains about any turn over 46°, so a hand-made 90° corner cannot survive.
- **SMD parts** (Pro) have rectangular copper lands with no hole and sit on the copper side: they are stored with *flip* on, so in the component-side view they appear mirrored, and their outlines go to the bottom silkscreen. Their pad sizes come from the makers' land-pattern drawings (docs/research/smd-land-patterns.md).
- **Pads:** *through-hole* has a drilled hole and a ring. *Deadbug* has no hole; the pad is stretched outwards by about 2.2 mm so you can bend the lead flat and solder it on the copper side. *Both* keeps the hole and the extra copper. The part definitions are checked so that pads stay at least 0.25 mm apart in all three styles.

Copper islands

An island is a round, oval or rectangular piece of copper with its corners cut at 45 degrees (rectangles only). It is saved in the board file with its centre, width, height, shape and corner cut, and nothing else: it has **no net of its own**. Whenever the copper is worked out, each island takes the net of whatever touches it (a pad, a trace, a jumper pad; islands that touch each other are taken together). If two real nets touch it, it takes one of them and the checker reports the other as a clash, which is the short it is. Sizes are kept between 1 and 400 mm whatever made the island (a file, the editor or a script). An island nothing touches is its own little net and is kept clear of everything else. Because islands are ordinary copper once their net is known, the router steers round them, connections through them count as made, and every export contains them.

Ground fill

With *Board > Ground fill* on, the board is copper except where it must not be. The program never cuts polygons; it describes the copper as a paint order that each output follows:

1. a rectangle covering the board, inset by the edge clearance, in copper;
2. every piece of copper that is not ground, grown by its own growth plus the gap, in bare board;

3. the fattened copper of those nets, then all the ordinary copper, in copper.

The growth of each shape is the wanted growth, cut down so that it keeps the gap from ground copper and the edge, and keeps the gap from any other net's shape even if that one grows by as much (half the free space each). Islands you placed do not grow.

To find floating fill, the fill is sampled along lines 0.05 mm apart; neighbouring intervals that overlap are joined into pieces, and each ground pin is looked up in its piece. Pieces with no ground pin are cut away (one polygon per run of rows), and the same is done once more across the other direction for slivers thinner than a row. Ground copper (pads, traces and islands of the ground net) counts as fill, so a thin ground trace through a narrow gap joins the pieces either side. Ground pins that sit on different pieces are shown as yellow rubber bands between the pieces. The editor draws the fill at once and does the floating-fill check about 0.4 s after the last change; Check and every export always use the finished fill.

Outputs: the editor and the PDF/SVG prints draw in the paint order; Gerber writes the fill as a dark region, switches to clear polarity (`%LPC*%`) for step 2 and back (`%LPD*%`) for step 3; the laser engine adds the fill to the copper intervals of each scan line (fill interval minus the cut intervals), so negative mode burns only the gaps.

Thermal reliefs

With the ground fill or the back-side plane on, each ground pad (part pads, jumper pads, via pads, and copper islands on the ground net if the setting says so) gets a ring of bare board cut round it and a few copper spokes drawn across the ring afterwards. The spoke width is the one typed in, or, if a current was typed, the width that carries that current divided by the spoke count by the IPC-2221 outer-layer formula (the same formula as the width slider's amps readout). Round pads get radial spokes; ovals and rectangles get spokes leaving their long sides at right angles and their ends along the long axis; an island over 4 mm gets spokes 10 % of its shorter side wide (0.6 to 1.5 mm, never thinner than the board's) and one per 5 mm of perimeter. Before a spoke is kept it is checked against the copper of every other net nearby; a spoke that would come within the clearance is left out, and if none are left the pad keeps its solid connection and no ring is cut. The spokes count as ground copper when the fill is split into pieces, so a relieved pad still joins the fill. A part marked *heavy* gets spokes twice as wide; *solid* gets no relief.

Back-side ground plane

The component side of the board is one copper sheet used as ground; nothing else is ever on it. Like the fill, it is a paint order: the plane rectangle (the board inset by the edge clearance), then bare rings round every hole whose lead is not ground (hole radius plus the clearance setting), then, for ground leads and vias, a relief ring and its spokes (or nothing for *solid* parts). A **via** is saved as a point; on the trace side it is a round pad of the via-pad size on the ground net, in the drill file a hole of the via-drill size. Connectivity: every ground pin whose pad has a hole, and every via, is on the plane, so they are one group; the fill pieces and copper groups join that group through any such pin or via. The router treats the plane group like already-joined copper, so it only draws traces from hole-less ground pins to the nearest via, ground pad or ground copper. Stitching vias are placed on a grid where a via pad keeps the clearance from other nets, is not under a part body, touches ground copper or the fill (and is not inside a cut of it), and is not inside another hole's clearance ring; "via at every ground pin" puts

one at the far end of each hole-less ground pad so the via pad overlaps it. The stitching guide is f = c / (20 · s · sqrt(4.4)) for a spacing s (lambda/20 in FR4).

Outputs: the Gerber copper layer of the trace side becomes *L2,Bot* and a *Copper,L1,Top* file holds the plane (dark region, clear rings, dark spokes); the back-plane PDF/SVG draw the same paint order seen from the component side, so they are not mirrored; the laser job for the back uses the plane as a fill (the rings and reliefs become the burn), forces mirror off and says so in its header. The countersink list is simply every hole whose lead is not ground.

Tidying wide traces

A trace that is 1.2 mm or wider gets three fixes, in this order, each checked against the real copper and the clearance rule:

1. If an end lies inside a pad of its own net but not on its centre (the router works on a 0.635 mm grid, so it often stops a little off-centre), the end is moved to the pad centre.
2. From the pad centre the trace first runs straight up, down, left or right for about the pad radius plus the trace width, then joins the rest of the trace at 45°. All four directions and a few ways of rejoining are tried; the shortest that fits wins, and running over the part's own body is avoided.
3. Corners are cut again with chamfers that may not shorten those straight exits, then lengthened where there is room.

A reshaped trace is only kept if everything of its own net that it touched (pads it ran over, branches ending on it, jumper pads) still touches it, with every branch end at least as deep inside it as before; otherwise only the end is moved onto the pad centre, or nothing changes. It must also be no closer to copper of any other net than it already was (thin traces only get step 1). This runs after auto-routing and, if *Auto-tidy* is ticked, a moment after the width slider stops. If a straight exit cannot fit (for example a 3 mm trace between 2.54 mm pitch pads), the trace keeps its old shape and the clash is marked in red.

T-joints

Where a branch ends on another trace of its net, the branch is stopped on that trace's centre line, or pulled back by half the difference in width when the branch is the wider one, so its round end does not stick out of the far side. The end always stays inside the other trace's copper. This runs after routing, after tidying and after a drag.

Dragging, trimming, splitting

- **Drag:** branches that end on the trace you drag move with it: each slides along its own last section until it meets the moved trace again (or gets a short 45° link), so it stays joined. The section you hold moves sideways (only at right angles to itself, in grid steps). Each neighbouring section keeps its direction and is lengthened or shortened to meet the moved line, so a 45° section stays 45°. A section that starts on a pad cannot move its pad end; a 45° link is added there. The result is chamfered like any trace.
- **Trim:** the clicked section is removed; what is left either side becomes separate traces.
- **Split:** the trace is cut in two at the clicked point; no copper is removed.

Edited traces count as hand-drawn, so Auto-route leaves them alone.

Auto-place

Two placers (*Board > Auto-place algorithm*). **Annealing** is the one below. **Routability** (experimental) uses the same moves with two extra cost terms: a RUDY-style congestion estimate (each net's bounding-box half-perimeter times its trace pitch, spread over the box and summed on a 5 mm grid, against the room each cell has left once part bodies are taken out; the overflow is wire that will not fit) and a count of pads hemmed in by other parts on all four sides within 1.6 mm; its temperature comes from the spread of random moves and cools by the acceptance rate (VPR's schedule), with the move range shrunk to keep about 44 % of moves accepted, and it cools faster when the time limit would otherwise cut it off hot. In measurements it did not beat Annealing (DESIGN_NOTES.md).

Simulated annealing (random moves that are accepted if they help, or sometimes if they don't, less often as time goes on). It moves, rotates and swaps parts that are not locked. The score is the total length of the shortest ratsnest (minimum spanning tree per net) plus a penalty for each crossing of two airwires plus a large penalty for overlaps and parts off the board. It runs on a copy of the board in the background and you can cancel it.

Auto-route

Three drivers share the same grid, search and checks (*Board > Auto-route algorithm*):

- **Classic** is the sequence below, tried with up to six net orders.
- **Negotiated congestion** (PathFinder, McMurchie & Ebeling 1995, as in VPR): pass 1 routes every net with the search allowed to run through other nets' copper at no extra cost, so each takes its best path. After every pass the cells where two or more nets' copper (plus clearance) overlap are found; every such cell gets a history cost (+1 per pass), and the present-congestion price starts at 0.5 on pass 2 and is multiplied by 1.3 each pass (capped at 1000). On later passes only the nets in conflict are torn up and re-searched, paying price × (number of other nets in the cell) + history per cell. Once the price reaches 1, a jumper may be taken at five times its usual cost, so it is used only where contested copper costs more. The loop stops when no cell is shared (usually 3 to 15 passes), after 30 passes, or after 8 passes without improvement; converged nets are then checked against exact geometry, and anything still in conflict is handed to the classic sequence below (rip-up, jumpers) with the history costs cleared. Nets are visited in ascending bounding-box size, then widest first, which the net-ordering literature favours; the result does not depend much on that order.
- **Auto** runs negotiated once, and only if a connection is left or a jumper was needed runs the classic tries within the time budget; the best result (fewest unrouted, then fewest jumpers, then shortest) is kept.

The classic sequence:

1. Nets are ordered: wide nets first, then short nets first.
2. **Rigid pairs first.** Pairs marked rigid (Shift + Wire) are routed before anything else in their net, each as one pin-to-pin search, and those traces are flagged so the editor can mark them with a thin orange centre line.
3. Each remaining connection is searched with **A*** on a 0.635 mm grid. The search state is *position and heading*, and a turn costs extra. Moves are straight or diagonal, and a 90° turn costs as much as two 45° turns, so it prefers long straight runs and gentle bends instead of staircases.

4. Each width class has its own obstacle map, so a 5 mm trace keeps further away from other copper than a 0.8 mm one. A step under a part's body (away from its pins) costs a little extra, and a lot extra with *Keep traces out from under parts*.
5. A found path is **checked with exact geometry** (the real copper shapes, with the real clearance). If it fails, the offending grid cell is forbidden and the net is searched again.
6. If a net cannot be routed, a "soft" search is allowed to pass through other nets. Those nets are torn out (**rip-up**) and routed again after the blocked one. This is limited: at most 3 tear-outs per net, 40 in total per round, 3 rounds.
7. If a net still fails, a **jumper pass** looks for an orthogonal wire link of 5.08 to 15.24 mm (2.54 mm steps) that lands on free pads and doesn't pass over a part's body or another jumper.
8. Finally, corners on wide traces are enlarged to the chamfer rule, and every trace is tidied as described above.

Traces you drew yourself are kept. This is **not** a full PathFinder router: it has no per-cell "history cost" that grows over repeated rounds. The research notes explain the idea and why it was left for later.

The in-browser version (nothing to install)

`docs/app/` on GitHub Pages is the same web page with the server taken away. `tools/build_web_app.py` copies the page from `src/solotrace/web/static/` and packs the editor's Python (everything in `src/solotrace` except the tkinter window) into `solotrace-py.zip`. When you open the page, `worker.js` starts a Web Worker, which downloads Pyodide 0.29.5 (CPython compiled to WebAssembly) from `cdn.jsdelivr.net`, unpacks the zip into Pyodide's in-memory file system and calls `solotrace/web/api.py`, the same function the local server calls for every request. The page sends its requests to the worker instead of over HTTP and gets the same answers. In the browser there are no threads, so long jobs (auto-route, auto-place) run straight through and send progress messages as they go; the page cannot cancel them. The folder where the desktop and server versions keep `config.json`, the autosave and your own parts is mounted on the browser's IndexedDB, so they survive a reload. Nothing about your board leaves the page: there is no server to send it to. The only downloads are Pyodide (once, then cached by the browser) and the page's own files from GitHub Pages.

Moving a part over laid traces

Traces do not follow a part: they are separate copper that happens to end on its pads. So moving, turning, nudging or mirroring a part after routing can put one of its pads on another net's trace, which is a short. While you drag, the editor tries the part at the mouse position (`probe_part` in the web UI, the move itself in the window) and draws a red ring on every pad or trace closer than the clearance; when you let go, the status bar says how many collisions there are and what to do (`after_part_edit` in `drc.py`). Check finds the same things, plus one more: a part mounted under the board (mirrored) and soldered flat is pressed against the copper, so its body over a trace of another net is a warning; on header pins it is only a note.

The wire lock

With the Wire tool the editor works out, at every mouse move, which pin a click would connect to (`Board.pin_lock`): the pad whose copper edge is nearest the cursor, within about 14 screen pixels (clamped to 1.4 to 3 mm on the board). It draws a solid green circle there and snaps the rubber band

to it. If that pin is the one you started from, the mark is a grey dashed ring and a click does nothing. The web UI gets the answer from the server (the `hover` reply carries a `lock`), because the board lives in Python.

Hole strips

A hole strip is a single oval pad that lists several holes (`holes` in the part file). To the rest of the program it is one pin: one net, one place to wire to. Every hole is drilled (the Gerber and laser exports use `Board.drills()`), and the whole bar is flat copper you can solder to. The breakout boards use a strip on each module pin, overlapping the module's pad so the two are one piece of copper.

Antennas, keep-outs and coils

An antenna or coil part carries its own copper (`strokes` in the part file: a polyline and a width) and one or more keep-out boxes (`keepouts`). The copper is ordinary copper: it is in the Gerber, the laser job and the checks, and a stroke named after a pin is on that pin's net. The keep-out is a box where no other copper may be: Check reports any (kind `keepout`), the ground fill and the back-side plane skip it, and the autorouter will not run through it. Some parts also list `ties`: pins joined by a wire you solder by hand (the coil's inner end and its landing pad); those pins count as one connection and nothing is routed between them.

The antenna lengths come from the quarter-wave formula and the coil inductance from a sum of straight-bar inductances; both are estimates, explained in docs/research/pcb-antennas.md.

DRC

The checker measures the real copper shapes: clearance between different nets, edge clearance, width, corner angle, ring size, jumper length and crossings, and unrouted pins.

Exports

- **Gerber:** RS-274X, coordinates in millimetres with 4.6 format. Traces are drawn with round apertures (so the capsule ends are right), pads and polygons as regions. The copper layer is labelled *bottom copper* and uses the same coordinates as the design view, which is how a fab expects a single-sided board whose copper is on the back. Files: copper, board outline, silkscreen (component outlines), and an Excellon drill file. There is no soldermask layer.
- **1:1 PDF/SVG:** the page is exactly the board size. For toner transfer use the mirrored version.
- **BOM:** parts grouped by name and value.

Laser G-code

The exporter works out, for each scan line, the exact stretches that must be burned, instead of converting an image. That gives sharp edges and lets the **trace adjust** and **gap** settings be exact:

- *Negative:* burn the whole board area **except** copper, shrunk or grown by *trace adjust*, and keep a border. Paint is left only where copper is wanted.

- *Isolation:* burn only a band *gap* mm wide around the copper (copper grown by *trace adjust* first).
- *Positive:* burn the copper itself.

Negative adjust is done by "eroding" the copper shapes through their edges, which is exact for the capsule and polygon shapes used here. Scan lines run along X, every *line interval* mm, optionally alternating direction and optionally repeated at 90° (cross-hatch) or several passes. G-code uses `G21` `G90`, a rapid `G0` to the start of each run and `G1 X.. S.. F..` along it. With **M4** (GRBL "dynamic power") the controller scales power with speed; with **M3** the exporter adds a zero-power run-in and run-out (*overscan*). Drill centres get a short dot. The time estimate is distance divided by speed plus rapids, and is not a promise.

What is assumed, not known: that the Phecda behaves like a GRBL machine (the manual names LaserGRBL and LightBurn), that `$30` matches the S max you enter, and that laser mode (`$32=1`) is on. No file has been run on a real Phecda. See docs/research/phecda-laser-notes.md.

Breakout boards for the ESP32 "postage stamp" modules (0.9.0)

What was asked: a breakout for the ESP32-S3 and ESP32-C3 stamp-sized boards, with a few reachable + and - rails, power in through a 2-pin screw terminal *or* a 2-pin header, and every GPIO with **four** possible connections, "a hybrid of deadbug and through hole".

Everything below is from the repository's own data (the module notes in mcu-modules.md and the parts' code), checked by tests. Nothing here has been built on a real board.

Which modules

Module (part id)	Pins	Power pins	Source of the pin order
Seeed XIAO ESP32-S3 and ESP32-C3 (`MOD_XIAO_S3`, `MOD_XIAO_C3`)	2 × 7, rows 15.24 mm apart	right column, top: 5V, GND, 3V3	Seeed's KiCad footprint; both chips share it (mcu-modules.md)
ESP32-C3 SuperMini (`MOD_ESP32C3_SUPERMINI`)	2 × 8	right column, top: 5V, GND, 3V3	two independent footprints; one maker's file disagrees (see the part note)
Waveshare ESP32-S3-Zero (`MOD_ESP32S3_ZERO`)	2 × 9	**left** column, top: 5V, GND, 3V3	two community footprints; Waveshare publishes the pinout only as a picture

The XIAO ESP32-S3 and ESP32-C3 have the same footprint, so one breakout (built on `MOD_XIAO_S3`) serves both; the board and its pin labels are the same. The two other modules are added so the same generator covers the stamp-size boards people actually buy.

The idea: a hole strip

A normal breakout is a row of single holes, one per pin. Here each module pin gets a **hole strip**: one bar of copper (1.8 mm wide, rounded ends) with holes every 2.54 mm. Because it is one piece of copper, every hole of a strip is the same net, so a pin has:

1. the module's own pad (the module sits in the middle, through-hole or flat on the copper),
2. and **four holes** on its strip (`BRK_STRIP4_LEAD`).

Each hole is a through-hole (pin header, jumper wire, component lead) *and* flat copper (a wire or SMD part soldered flat on the bar, deadbug style), which is the "hybrid of deadbug and through hole". That makes 4 places per GPIO besides the module's own pad. The bar starts 0.3 mm past the module's pad, so the pad and the bar overlap and are one piece of copper: no trace is needed between them (SoloTrace joins touching copper of the same net).

Strips are not in the BOM (they are copper and holes, not parts), always drilled and always flat copper: the *Pads of this part* choice does not apply to them.

Also added: plain strips of 2, 3, 4, 5, 6, 8, 10, 12, 14 and 16 holes (`BRK_STRIP<n>`) under *Board bits* > *Hole strips (breakout)*, for rails and your own breakouts.

The rails and power in

- Three vertical rails of 6 holes: **3V3, GND and 5V**, on the side of the module where its power pins are. They are stepped (the 3V3 rail is nearest and shortest to reach, 5V the farthest) so each power row reaches its rail with one short straight trace and no rail is in another's way.
- **Power in:** a 2-pin screw terminal (`TERM_5.08_2`, 5.08 mm pitch) and a 2-pin header (`HDR_1x2`) are both wired to 5V and GND. Fit one of them. Pin 1 is 5V and pin 2 is GND on both (silk/pin labels show it).
- 3V3 has a rail but no connector: it comes from the module.

Not verified here: what happens if you power the module from the screw terminal and USB at the same time. Seeed's symbol calls the XIAO's 5V pad VBUS (mcu-modules.md), which suggests it is the USB 5 V line, but whether it can be back-fed safely was not checked in a datasheet. Read your module's documentation before feeding 5V to it while USB is plugged in. 3V3 is the module's own regulator output; its current limit was not looked up either.

No copper under the module

Many of these modules carry pads or a battery connector on the *back* (the part notes say the pads under the board are not included in the footprint). So no copper and no trace goes under the module body: the strips stand clear to the left and right of it, and the power connectors sit below it. A test checks that no trace point lies inside the module's body rectangle.

Layout sizes

Generated by `demos.breakout()` from the module's own pad table; sizes at the time of writing (mm): XIAO 52 × 30, SuperMini 52 × 31.5, S3-Zero 52 × 32.5. All three routed with no jumpers and no DRC problems; see `tests/test_breakout.py`.

What was tried first

Four full-height rails routed badly (5V failed, traces wandered along the top, then through the middle under the module). Three short stepped rails fixed that, and moving the connectors under the module fixed the rest.

Limits and open points

- Never etched, never soldered, no module plugged in. The strip hole spacing (2.54 mm) and 1.0 mm drills follow the rest of the library.
- The three breakouts are generated with the module's own pads *and* hole strips, so a module soldered flat (deadbug) lands on the strips' copper where they overlap its pads; a module on header pins stands in the module's own holes.
- A footprint saved from the custom-footprint text editor does not keep a pad's extra holes (the `holes` list is read and written only in files and the built-in parts).
- The 3V3 and 5V rails are plain copper bars: their current rating was not worked out. Power class traces use the usual 2 mm width.

PCB antennas and the coil generator (0.10.0)

Notes behind the antenna parts (`src/solotrace/antennas.py`), the keep-out boxes and the coil generator (`src/solotrace/coil.py`). Written down so that nobody has to take the numbers on trust. **None of this has been built, etched or measured.** Everything here is a starting point for a person with a NanoVNA or an LCR meter.

What was added

Part	What it is
`ANT_MEANDER_315`, `_433`, `_868`, `_915`, `_2G4`	A meander monopole: one 1 mm trace, a stub up from the feed pad, then runs back and forth. Pin 1 = feed, pins 2 and 3 = ground, either side of the feed. A hatched keep-out box round it.
`ANT_WIRE_315` … `_2G4`	A straight quarter-wave wire (a normal part in the parts list): its length is in the name, pin 1 = the wire, pin 2 = ground.
`ANT_KEEPOUT_10x10`, `20x10`, `20x20`, `30x15`, `40x20`	A bare keep-out rectangle with no pins: mark where a module's own antenna sits.
Coil from *Parts > PCB coil generator…*	A rectangular spiral sized by area plus inductance, or area plus turns.

Bands: 315 MHz, 433.92 MHz, 868 MHz, 915 MHz and 2.45 GHz (the common licence-exempt bands for cheap remote-control modules, LoRa modules and 2.4 GHz radios). The list is common knowledge of which bands such modules use; it is not a list of what any given module uses, so check your module's datasheet.

The formula

The trace is a quarter-wave monopole. The length of copper is

```
L = c / (4 f sqrt(eps_eff))        eps_eff = (eps_r + 1) / 2 = 2.7   (FR4, eps_r taken as 4.4)
```

`(eps_r + 1) / 2` is a rule of thumb for a trace on a board with air above it and nothing underneath. It is not a field solution. Real boards differ (laminate, thickness, copper, case, hand, battery). So each meander gets **10 % extra copper** past the formula length, with a silk tick every 2 mm back from the tip (the longer tick is the formula length). The idea is to measure the antenna and scratch copper back from the tip. If the resonance is too high, you need copper added, not removed.

Band	Formula length (mm)	Copper drawn (+10 %)
315 MHz	144.8	159.3
433.92 MHz	105.1	115.6
868 MHz	52.5	57.8
915 MHz	49.8	54.8
2450 MHz	18.6	20.5

(The test `tests/test_antenna.py` checks the drawn length against the formula.)

The wire whip is the free-space quarter wave, `c / (4 f)` (172.8 mm at 433.92 MHz). The part's note says a common rule of thumb is to cut about 5 % shorter and trim while measuring.

The keep-out

Every antenna part carries a keep-out box. Inside it, SoloTrace:

- reports **any copper** (a trace, a pad, a jumper, a via, an island) as a DRC error, kind `keepout`;
- cuts the **ground fill** and the **back-side ground plane** away from it;
- keeps the **autorouter** out (the box is a hard obstacle, so routes go round it, never through).

Why: antenna manuals from chip makers tell you to keep ground planes and other copper out from under and round a printed antenna. For example, the TI application notes that were consulted while planning this (see below) say no ground plane or traces beneath the antenna. The box sizes here are my own: the length of the meander plus a 5 mm margin below 1 GHz and 3 mm at 2.4 GHz. They are not copied from any datasheet. **Ground copper beside the feed is still needed** (a monopole works against ground), so pins 2 and 3 are ground pads either side of the feed, and the ground fill in the rest of the board does the job. Keep ground out of the box and beside the feed.

What was left out, and why

- **Other people's layouts are not copied.** The TI documents that describe tested antenna layouts (AN043 meander inverted-F, 15.2 × 5.7 mm; DN007 inverted-F, 25.7 × 7.5 mm on a 1 mm board; DN024 meander monopole, 38 × 25 mm with a 2.0 mm trace on 1.6 mm FR4, in 868/915 MHz and dual-band versions; AN058 with a table of sizes for several antenna types) are listed here **for comparison only**. Those numbers were read from search results and web summaries: the TI PDFs themselves could not be downloaded here, so nothing was traced from them, and their layouts are the maker's copyright. Do not treat the sizes above as verified. Notice that those designs are also much more compact than the textbook quarter-wave length would suggest; the generated antennas are plain meanders, they are **not** as small.
- **No matching network.** Real antennas need a matching network (a pi network of small SMD parts, or a series part) tuned with a VNA. SoloTrace has no such part for this; the note on the part says to feed it through any matching parts. Hand-soldering a pi network onto the feed is a normal approach.
- **Not included:** inverted-F antennas, chip antennas, GPS patch antennas, 125 kHz coils, loop antennas. Say if you want any of them.
- **ESP32 / Wi-Fi module keep-outs.** Many modules need a clear area past the module's antenna end. The size is in each module's datasheet. I did not enter numbers from memory. Use the bare keep-out box and size it from your datasheet.
- **No antenna performance claims.** No gain, bandwidth or range is claimed anywhere.
- **Nothing has been simulated or measured**, not even with a field solver.

The coil generator

Choose the **outer size** and either **the inductance wanted** (the generator finds the number of turns and the pitch that come closest) or **the number of turns** (it works out the inductance, spreading the turns over the area). Track width, the smallest gap between turns (default 0.4 mm), copper thickness

(default 35 um) and an optional frequency (it then prints the parallel capacitor that would resonate the coil) are the other settings.

How the inductance is worked out

The spiral is a chain of straight rectangular bars.

- Each bar has the self inductance of a straight flat conductor (Rosa; Grover, *Inductance Calculations*): `L = mu0/(2 pi) * l * [ln(2l/(w+t)) + 0.50049 + (w+t)/(3l)]`.
- Every pair of **parallel** bars has a mutual inductance (Grover), summed with `+` when the currents run the same way and `-` when they run opposite ways. Bars at right angles do not couple.
- The total is the sum of the self terms plus twice the sum of the mutual terms.

That is a textbook method that needs no fitted coefficient. **Cross-check:** `tests/test_coil.py` compares the result for three square coils with the square-spiral current-sheet formula of Mohan, Hershenson, Boyd and Lee (IEEE Journal of Solid-State Circuits 34, 1999): `L = mu0 n^2 d_avg (c1/2) [ln(c2/rho) + c3 rho + c4 rho^2]` with the square values `c1 = 1.27`, `c2 = 2.07`, `c3 = 0.18`, `c4 = 0.13`. The test accepts a difference up to 8 %; the three cases come out +1.3 %, +1.4 % and +2.5 % above the formula. That agreement shows the two calculations are consistent, it does not show the real coil will measure that.

What the estimate leaves out: the coil's own capacitance and self-resonance, skin effect and proximity effect (the DC resistance is shown, the AC one is not), a ground fill or plane under or near it, ferrite, metal, a case, and the tolerances of a home-etched board. It is a single-layer coil in air. Measure the real coil and trim.

The single-sided problem

On a one-sided board the inner end of a spiral has to cross the turns to come out. The generated part therefore has three pins:

- **pin 1**: the outer end;
- **pin 2**: the inner end, in the middle of the hollow;
- **pin 3**: a landing pad outside the coil.

You solder an **insulated wire** (enamelled or sleeved, so it cannot touch the turns) from pin 2 to pin 3. The part ties 2 and 3 into one net and counts them as already joined, so the router does not try to run a trace across the coil and nothing is reported as unrouted. Wire your circuit to pins 1 and 3. A 2 mm copper-free keep-out box surrounds the coil.

Not provided

No reader-IC matching (for NFC or RFID), no tuning-capacitor selection beyond the single parallel value, no multi-layer or round coils.

Sources

- F. W. Grover, *Inductance Calculations*, 1946 (reprinted by Dover): self and mutual inductance of straight bars.
- S. S. Mohan, M. del Mar Hershenson, S. P. Boyd, T. H. Lee, "Simple accurate expressions for planar spiral inductances", IEEE JSSC 34 (10), 1999.

- The rule-of-thumb effective permittivity `(eps_r + 1) / 2` for microstrip is textbook material; it is a rule of thumb here, not a result.
- Texas Instruments application notes AN043, AN058, DN007, DN024 (names only, see "What was left out").

Changelog

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN

0.10.1-wip (2026-11-10)

- **Resizable side panels.** The web UI has grey drag handles between the parts panel and the board, between the board and the selection panel, and between the parts list and its preview (double-click a handle to reset it). The long part description under the preview now scrolls inside its own box instead of squeezing the list. Sizes are remembered in the browser. The desktop window has drag handles beside both side panels too.
- **The coil generator is easy to find.** *New footprint...* and *PCB coil generator...* buttons now sit under the parts list in the web UI (before, the coil generator was only in the *Parts* menu).

0.10.0-wip (2026-11-10)

- **PCB trace antennas with copper keep-outs.** A new group *Board bits > Antennas and keep-outs* (15 parts): a **meander monopole** for 315, 433.92, 868, 915 and 2450 MHz (one 1 mm trace, quarter-wave length from `c / (4 f sqrt(eps_eff))` with `eps_eff = 2.7` for FR4, plus 10 % to trim, silk ticks every 2 mm from the tip), a **wire whip** (a straight quarter-wave wire, in the parts list) for the same five bands, and five bare **keep-out boxes** (10×10 to 40×20 mm) for the antenna end of a module. The formulas are rules of thumb: nothing was etched or measured, and the parts say so. See docs/research/pcb-antennas.md.
- **Keep-outs are real.** A part can carry copper strokes (`strokes`), keep-out boxes (`keepouts`) and pins that are joined by a wire (`ties`), all saved in `.solotrace` files and custom parts. Copper of any kind inside a keep-out is a DRC error (kind `keepout`, also shown while moving a part), the ground fill and the back-side ground plane are cut away from it, the autorouter treats it as a hard obstacle and auto-tidy keeps traces out. It is drawn as an orange hatched box with a label in the desktop window and the web UI.
- **A parametric PCB coil generator** (*Parts > PCB coil generator...* in the desktop app, *Parts > PCB coil generator...* in the web menu). Give the outer size and EITHER the inductance wanted (it finds the turns and the track pitch) OR the number of turns (it works out the inductance), plus the track width, smallest gap, copper thickness and an optional frequency (it prints the parallel capacitor that resonates the coil). It saves the coil as a part under *My parts* and arms it for placement. The inductance is a straight-bar method (Rosa self inductance, Grover mutual inductance), cross-checked against the square-spiral formula of Mohan et al. (1999): within 3 % on three test coils. It is an estimate for a single-layer coil in air, **not measured**. On a one-sided board the inner end is carried out by an insulated wire between pins 2 and 3 of the part; the part counts those pins as joined.
- Two new examples (*File > Open example*, also in `examples/`): a 433.92 MHz meander antenna test board with ground fill, and a 30 × 20 mm 1.5 uH coil with a 13.56 MHz tuning capacitor footprint.
- Fix: the library-size test now leaves out the antennas and the breakout hole strips. 22 new tests (antenna parts and formula, keep-outs in DRC / fill / plane / router, save and load, web dialog, coil engine and both design modes).
- Not done: inverted-F, chip, patch and loop antennas, matching-network parts, NFC reader matching, round or multi-layer coils, and keep-out sizes for particular Wi-Fi modules (read them

from the module's datasheet and use a keep-out box). The custom-footprint text editor drops strokes, keep-outs and ties of a part it re-saves.

0.9.0-wip (2026-11-07)

- **A green lock circle when wiring.** With the Wire tool the pin your click would land on is now marked before you click: a solid green circle with the pin name (and its net, in the web UI) appears on it, the rubber band snaps to it, and a grey dashed ring marks the pin you started from. The lock reaches about 14 screen pixels past the pad edge (clamped to 1.4 to 3 mm on the board, so it stays easy to hit when zoomed out and does not grab the wrong pin when zoomed in) and takes the nearest pad by distance to its edge, not to its centre. One shared function, `Board.pin_lock`, serves the tkinter window and the web UI, and the web UI asks the server after every command while the Wire tool is active. Tests: `tests/test_wire_lock.py`.
- **Hole strips and breakout boards.** New part kind `strip`: one bar of copper with several holes, one pin. A pad may now list its holes (`PadDef.holes`, saved in `.solotrace` files and custom parts), and `Board.drills()` (Gerber, laser, preview, web) includes all of them. Library: `BRK_STRIP2` to `BRK_STRIP16` (2 to 16 holes) and `BRK_STRIP4_LEAD` (4 holes, starting 0.3 mm past a module pad), under *Board bits > Hole strips (breakout)*. Strips are always through-hole and flat copper, never in the BOM, skipped by auto-place and by the ring check.
- **Three generated breakout boards** (*File > Open example*): XIAO ESP32-S3 / C3, ESP32-C3 SuperMini and ESP32-S3-Zero. Every module pin gets a 4-hole strip touching its pad (four extra connections per GPIO, through-hole and deadbug in one), three 6-hole rails (3V3, GND, 5V), and a 2-pin screw terminal and a 2-pin header for 5V / GND in. No copper under the module. They route with no jumpers and no DRC problems, and are exported to `examples/breakout-*` (Gerber, SVG, PDF, BOM, laser). Never etched or built; the XIAO 5V / USB question is left open on purpose (see docs/research/breakout-boards.md). Tests: `tests/test_breakout.py`.
- **PDF guide with every release.** `tools/make_pdf.py` builds `docs/SoloTrace-guide.pdf` from the README, changelog and notes, and the release workflow attaches it.

0.8.0-wip (2026-11-06)

- **A web app: nothing to install.** The whole editor now also runs inside a web page on the project's GitHub Pages site (`https://the-dorkknight.github.io/solotrace/app/`): Pyodide 0.29.5 (Python in WebAssembly, from the jsDelivr CDN) runs the same SoloTrace code in a Web Worker, so the page stays responsive. Nothing is uploaded: *Save* downloads, *Open* reads a file you pick, settings, favourites, your parts and the autosave stay in the browser's storage. Slower than the download and auto-route has no Cancel; tested in Chromium with a local copy of Pyodide, **not yet loaded from the CDN** (the build machine cannot reach it). Built by `tools/build_web_app.py` into `docs/app/`; `tests/test_web_app.py` checks it is up to date, that its Python runs without tkinter, threads or sockets, and (with Playwright and a local Pyodide) loads, routes and saves a board in Chromium.
- **The browser is the default.** `python3 src/main.py` and the three launchers always open the web UI now, so tkinter is no longer needed; `python3 src/main.py desktop` opens the old window (and falls back to the browser if tkinter is missing). The launchers only check for Python 3.8+.
- **Collisions after moving a part over laid traces.** Traces do not follow a part, and moving, turning, nudging, mirroring, duplicating or placing a part over existing copper said nothing until the next *Check*. Now, in both UIs, the pads and traces it collides with are ringed in red while you

drag (the web UI shows a red ghost), and the status bar says how many collisions there are when you let go, with what to do (drag it clear, or *Route this part's nets*); auto-place reports them too. *Check* itself was measured and never missed one (64 random moves, compared with a brute-force check of every pair of copper shapes). New in *Check*: a module mounted under the board (mirrored) and soldered flat is pressed on the copper, so its body over another net's trace is a warning (a note on header pins).
- Fix (web UI): pressing on a trace that has branches joined to it got no answer from the server (its drag description held Python objects that cannot be sent as JSON), so such a trace could not be selected or dragged in the browser. Found while taking the 0.8.0 screenshots; test added.
- **Auto-route: measured against the first versions, made repeatable, and a polish pass.** The question was whether auto-route had got lazier since the first releases. Measured on 12 boards (the four oldest examples, each as built and with two shuffled placements), routed by 0.3.4 (the oldest version in the repository), 0.4.0 to 0.7.0 and this one: 0.3.4 used 64 jumpers in all and left 3 connections unrouted; 0.7.0 used 44 and left none, with the same total copper (4 515 mm against 4 509 mm). On the small boards it sometimes takes a longer path to save a jumper (it always prefers fewer jumpers first). What was wrong: **the same board did not route the same way twice.** The order in which ripped-up nets were rerouted came from a Python set, whose order changes every time Python starts, so one board could come out with 2 jumpers on one run and 4 on the next (measured). Fixed: the order is now sorted, so a board always routes the same way. New: after the best try is picked, a **polish pass** reroutes each net once against all the others and keeps the new copper only if it has fewer jumpers or is at least 1 mm shorter (it never makes a board worse, and Cancel during it puts back what it was trying): on the same 12 boards 42 jumpers and 4 498 mm of copper, up to 4 % less copper on the logic boards. `tests/test_router_quality.py` checks the repeatability across hash seeds, that polish never adds jumpers or copper, and that cancelling it loses nothing.

0.7.0-wip (2026-11-06)

- **Modules & displays: 23 new parts** in a new group of the through-hole list, every pin labelled as printed on the module (and in *Pin labels* as *As printed on the module*): Seeed XIAO ESP32-C3 / ESP32-S3 / RP2040, ESP32-C3 SuperMini, Waveshare ESP32-S3-Zero and RP2040-Zero, Raspberry Pi Pico and Pico W, ESP32 DOIT DevKit V1 (30 pins) and Espressif DevKitC (38 pins), NodeMCU v2 Amica and v3 LoLin, WEMOS D1 mini, ESP-01, Arduino Nano, Pro Mini (with and without the FTDI header) and Pro Micro, 0.96 inch I2C and SPI OLEDs, the 0.91 inch I2C OLED, and the two common SPI micro SD modules. Each works as through-hole or deadbug (castellated boards can be soldered flat on the copper). Pin orders, row spacings and board sizes come from makers' KiCad and Eagle files, datasheets and vendor pages, never from memory; where the sources were thin or disagreed the part's note says so. Notes: docs/research/mcu-modules.md. Tests: `tests/test_modules.py` checks the row spacings and pin orders against those notes and that every module is DRC-clean as through-hole and as deadbug.
- **Mirror a part (Shift+M)**: mount it under the board (or an SMD part on top). It flips about pin 1, its outline goes to the bottom silkscreen (Gerber `Silkscreen_Bottom.gbo`), the BOM gains a *Mounted on* column, and *Check* lists it as a note. Mirrored parts are drawn with a thick dashed pink frame, a pink *MIRRORED* band, pink outlines and a pink panel note, in both the desktop window and the web UI; while placing, the ghost turns pink. Also in the right-click menu, the Edit menu and the part panel.

- New example: *XIAO ESP32-C3 logger*, with the 0.96 inch OLED mirrored under the board, the SD module, a button and a sensor header.
- Fix: an SMD part moved to the component side kept its outline on the bottom silkscreen; the silkscreen side now follows where the part really sits.

0.6.0-wip (2026-11-05)

- **Web UI: the same editor in your browser when tkinter is missing.** Python's window toolkit is often absent or broken (a Linux without `python3-tk`, a Windows install without "tcl/tk", the Apple Python with its old Tk), and until now that ruined the first start. `python3 src/main.py` (and the three launchers) now notice and start a tiny local server (`127.0.0.1` only, standard library, nothing installed) with the editor as a web page in your default browser; `python3 src/main.py web` asks for it on purpose. The page has the tool bar (Select, Place, Wire, Trace, Trim, Cut, Jumper, Island, Via), the parts list with search, favourites and the SMD / through-hole button, the selection panel with the width slider, the right-click menus, the Board / Export / Parts / Help menus, the dialogs (board rules, ground fill, back-side plane, laser export with its preview and test cards, new footprint with KiCad import), Check with a clickable list, undo/redo, keyboard shortcuts, the first-start notice, autosave and recovery. *Save* downloads the `.solotrace` file, *Open* reads one from your computer, exports download. The editing logic is shared with the desktop window: `src/solotrace/web/session.py` is the editor with the window taken out (the mouse handlers of app.py ported line for line), `server.py` is the server, `static/` the page. The server stops with *File > Quit* or by itself about 90 s after the page has gone. Tests: `tests/test_web.py` (session, server, a Chromium test when Playwright is installed). Not yet used by a person on a real design; see Known limitations.
- **SMD parts button instead of the Pro tick box.** The parts list is now one kind at a time: a whole-width button under the search reads **SMD parts** with a SOT-23 picture; click it and the list shows only the SMD parts and the button becomes **Through-hole parts** with a TO-92 picture; click again for the normal list. The choice is remembered (`parts_mode` in the config; the old `pro` key is read once and dropped). The group is called *SMD* (was *SMD (Pro)*).
- Launchers only require Python 3.8+ now; they say when tkinter is missing and the browser will open instead.
- Fixes found while fuzzing the headless editor: splitting a trace at its very end no longer leaves a one-point stub; the tidy step ignores such stubs if a file has them.

0.5.0-wip (2026-11-05)

Four things the author asked for, each researched first (the notes are in `docs/research/`):

- **Pro button: SMD parts.** A *Pro* toggle beside the parts search adds an *SMD (Pro)* group of 41 common, hand-solderable surface-mount footprints: chips 0603 to 2512, SMA/SMB/SMC, SOD-123/123F, MELF and MiniMELF diodes, SOT-23 (3, 5, 6 pins), SOT-223, DPAK (TO-252), D2PAK (TO-263), SOIC-8/14/16 narrow, SOIC-16 to 28 wide, SOP-8 5.3 mm, tantalum A to D, SMD electrolytics 4 to 10 mm, two shielded inductors. Every pad size comes from a maker's land-pattern drawing (TI, Diodes Inc, Vishay, KEMET, Nichicon, Sumida; the roomier wave-solder / level-A patterns where there was a choice) and every pad-to-pad gap is at least 0.3 mm, so nothing needs a solder mask. Packages finer than that (SOT-363, TSSOP, TQFP) are left out on purpose. SMD parts sit on the **copper side**: they are mirrored in the component-side view (press M to see them as you solder), their outlines go to a bottom silkscreen Gerber, and

they have no holes. SoloTrace stays a through-hole program first: Pro is off by default and only adds parts. Pin-label presets for SOT-23, SOT-223, DPAK and the SOICs. New example *SMD regulator (Pro parts)*.

- **Thermal reliefs on ground pads** (ground fill and back plane): a ring of bare board round each ground pad bridged by spokes, so the iron does not have to heat the whole board. Defaults follow KiCad (four spokes, 0.5 mm ring) but the spokes are 0.8 mm because ground carries the return current: four of them are good for about 8 A at a 10 C rise (IPC-2221). *Ground fill settings* has the spoke count, width and ring, an **"or size spokes for N amps"** box that works the width out from the IPC-2221 formula, and shows the current the relief carries. Round pads get radial spokes at 45 degrees, ovals and rectangles (deadbug pads, copper islands) get theirs off the flat sides, big islands get spokes scaled to their size. Each part can be set to **heavy** (spokes twice as wide) or **solid** (no relief: high current, RF decoupling) in its panel. Check warns when the ground net's trace class is good for more current than its reliefs carry. A spoke that would touch another net is left out; a pad that would lose all its spokes stays solid.
- **Back-side ground plane for a double-clad board** (*Board > Back-side ground plane*). All traces stay on the copper side; the component side is left as one sheet of ground copper. Ground leads that pass through the board are joined by the plane and soldered on both sides; a new **Via tool (V)** adds vias (a hole with a bit of wire soldered both sides) for deadbug / SMD ground pins and for stitching; every other hole gets a bare ring in the plane (default hole + 1.1 mm: 0.25 mm electrical plus a 0.3 mm registration allowance). Buttons add a via at every ground pin without a hole and a stitching grid; the dialog says up to what frequency a spacing keeps the lambda/20 rule and explains the "big LC circuit" worry in two sentences (two copper sheets joined at one point resonate near 700 MHz on a 100 mm board; joined at every ground pin plus a grid they are a low-impedance mesh). Exports: a top-copper Gerber (plane with clearance holes and reliefs), 1:1 back-plane PDF/SVG, a back-plane laser job (not mirrored), and a **countersink list** for the registration-free way: leave the back solid, drill, and clear the copper round each non-ground hole with a 2 to 2.5 mm drill tip. Check reports vias cut by a clearance ring, ground leads trapped in one, a plane nothing joins, and how far the stitching is good for. New example *RF oscillator (back plane)*.
- **Auto-route algorithm (Board menu):** *Auto* (default), *Negotiated congestion* and *Classic*. Negotiated congestion is the PathFinder method used by FPGA tools: every net is routed at once, overlaps allowed, then the price of every contested cell rises pass after pass until the nets have sorted themselves out; jumpers join in as a priced resource once contested copper is dear enough. It does not depend on the net order, so it needs no retries. On eleven test boards it routed everything (classic left 2 connections), used the same number of jumpers (55), 7 to 9 % less copper, and took about two-thirds of the time; on the logic demo 5 jumpers instead of 7. *Auto* runs it first and falls back to the classic tries only if something is left or a jumper was needed, so it is never worse than before. Details and numbers in DESIGN_NOTES.md.
- **Auto-place algorithm (Board menu):** *Annealing* (default, unchanged) and *Routability* (experimental): the same moves plus a RUDY-style estimate of where copper will not fit and a penalty for pads boxed in by other parts, with VPR's adaptive cooling. **Measured no better than Annealing** on the test boards (sometimes worse), so it is off by default; it is there to try and to improve.
- The DRC window now also shows notes (severity "info"), such as the stitching-spacing guide; they do not count as problems.
- Command line: `route --algorithm auto|classic|negotiated`, `place --placer anneal|routability`.

0.4.1-wip (2026-11-02)

Bug and edge-case sweep of 0.4.0: a randomised test of about 400 boards (random parts, islands and ground-fill settings, routed and exported in every mode) and about 4 000 random editor actions (clicks, drags, Shift + clicks, right-click menu items, keys, undo/redo, view flips, ground fill on/off), plus targeted checks. Fixed:

- **Ground fill could take seconds per edit on a big board** (about 1 s on a 160 × 100 mm, 30-part test board, more on a slow laptop). The editor now draws the fill at once and removes floating copper about 0.4 s after you stop changing things (0.05 to 0.09 s per edit on that board). Check and every export always use the finished fill.
- **A thin ground trace through a narrow gap between islands did not join the two pieces of fill** it physically joins: a false "not connected" rubber band, or a connected piece of fill removed as floating. Ground copper now counts as part of the fill.
- **Renaming the ground net moved the fill to another net.** The fill now follows the new name.
- **A net called V- (or V+, VCC, 12V...) could be guessed as ground.** Supply-like names are never guessed; with no GND-like name, the biggest other net is used.
- **Esc while dragging out an island still made the island.** It now cancels it.
- **An island completely hidden under a part body could not be selected** with the Select tool. Click the part, then click it again (not a double-click) to pick the island under it; the Island tool picks islands directly too.
- **The Trace tool could not start on an island**, and clicking an island of the same net did not finish the trace there (it does for pins). Both work now.
- **Auto-place on a board with copper islands** moved parts off their islands (in a test, 8 shorts). It now asks first, and the command line prints a warning.
- The laser "trace adjust would close the gap" warning used the gap as typed, not the real one (never less than the copper clearance).
- Islands are checked whenever they are made or changed, from a file, the editor or a script: sizes are kept between 1 and 400 mm (was 60), an unknown shape becomes a rectangle and a negative corner cut becomes 0.
- If a mouse press never got its release (the window lost focus mid-drag), the fill check now still runs once the mouse stops moving.

0.4.0-wip (2026-10-31)

For quick and dirty RF and fast analogue boards, built deadbug / Manhattan style:

- **Copper islands (Island tool, `I`).** Free-standing round or rectangular copper pads. Click to drop one, drag on the board to draw one any size, Shift + click to switch between round and rectangular. With *Select*, drag an island to move it and drag its white corner squares to resize it (the opposite corner stays put); the right panel has exact width, height and corner cut, square/circle size buttons, *Turn 90*, *Duplicate* and *Delete*, and the right-click menu has the same. An island has no net of its own: it joins the net of any pin, trace or jumper pad that touches it, and islands that touch each other share one net. Pins of two different nets on one island are a short, and *Check* says so. A board built from islands needs no traces at all.
- **Ground fill (Board > Ground fill).** The whole board becomes ground copper, with a gap (default 0.6 mm, never less than the copper clearance) cut round every other net. The ground net is guessed (GND, 0V, VSS..., else the net with the most pins) or chosen; it is joined by the fill, so Auto-route leaves it alone and removes its old automatic traces. Other nets' copper can be

fattened (default 1 mm, only where it keeps the gap; islands you placed keep their size). Pieces of fill that no ground pin sits on (floating copper) are cut away; a ground pin on a piece of fill that is cut off from the rest gets a yellow rubber band. Turning it on offers to make every pad a deadbug pad (no holes to drill).
- The fill is in every output: the editor, Gerber (with clear polarity, read back by the independent gerbonara library in testing), PDF and SVG prints, and the laser G-code (negative mode then burns only the gaps; isolation mode burns the same as negative on such a board).
- New examples: *Manhattan RF amplifier (copper islands)*, built from islands with no traces, and *RF oscillator (ground fill, routed)*.
- Command line: `solotrace islands board.solotrace out.solotrace [--net GND] [--gap 0.6] [--grow 1] [--deadbug] [--keep-floating] [--off]` turns the ground fill on or off.
- The coloured tool-bar buttons are only as wide as their text, so the bar fits a 1280 px window with the new tool.
- A redraw could be left scheduled when a window closed ("invalid command name ...redraw" in the test output); fixed.

0.3.5-wip (2026-10-29)

Rigid-wire refinements, after the author found rigid trunks "still separate, not a true union":

- **One colour.** Routed rigid traces are drawn in the normal copper colour, like the branches joined to them, with a thin dashed orange line down the middle to mark them. (They were always ordinary copper in the exports; only the editor drew them in a different colour on top.)
- **Clean T-joints.** A branch that ends on another trace stops on its centre line, and is pulled back a little when it is the wider one, so its round end no longer pokes out of the far side of a narrow trunk. It always stays inside the trunk's copper. Done after routing, tidying and dragging.
- **Branches follow a dragged trunk.** Dragging a trace sideways now moves the ends of the branches joined to it, so they stay joined (each slides along its own direction, or gets a short 45 degree link).

0.3.4-wip (2026-10-29)

- **Right-click never deletes anything by itself.** Right-clicking a trace only opens its menu. While the mouse rests on *Cut this piece*, the piece that would go is shown in red; it is removed only when you choose that item. On a joint the item is greyed out and says why.

0.3.3-wip (2026-10-29)

- **Cut (C) is one click now:** click a trace and the piece under the mouse is removed, from the joint before it to the joint after it (a pad, a place where another trace meets it, a jumper pad). The rest of the trace and the net stay. The piece turns red while you hover, before you click. Clicking right on a joint does nothing. The old two-point cut is still there as Shift + click (twice), or right-click > *Cut between two points*.
- **Right-click a trace > Cut this piece** does the same as one click with the Cut tool.
- **Shift + click while placing turns the part** 90 degrees instead of placing it; click without Shift to place.
- **Favourites & recent:** a drop-down at the top of the parts panel lists your starred parts and the last ten you placed. Star a part with the button under the preview, or right-click it in the list.

- The preview of a trace being drawn (Trace tool) and of a cut were drawn with a stipple pattern that some X11 setups do not show; they are solid now.
- The status bar text no longer pushes the version stamp off the window.

0.3.2-wip (2026-10-29)

From the next real board: "the second a rigid line appears the other traces form weird, as if they are repelled":

- **Branches no longer come loose from a trunk.** After routing, and whenever a width changes, wide traces are tidied (straight pad exits, centred ends, bigger chamfers). That tidy could move a rigid trunk away from the branches joined to it, leaving stubs that stop short, overshoot, or a pad cut off (in one rebuild a capacitor pad lost its connection). Now a trace is only reshaped if everything it touched still touches it and every branch end sits at least as deep in it as before; otherwise only its end is moved onto the pad centre, or nothing changes. The corner smoothing obeys the same rule.
- **DRC warns about hanging trace ends** ("dangling"): a trace end that is not joined to a pad, trace or jumper of its net.
- **Board > Keep traces out from under parts** (off by default): Auto-route then goes round part bodies (between DIP rows, under transistors) unless there is no other way. The areas round each pin stay free. On a cramped board this can cost jumpers. Without it, the retries now also count copper under part bodies against a result, so a short route through a chip only wins when going round is much longer.

0.3.1-wip (2026-10-29)

Fixes after routing a real board with 0.3.0 gave overlapping tracks and forced jumpers:

- **Overlapping / disappearing tracks fixed.** When a net was only partly routed and then routed again (after a rip-up), the copper it already had was not counted as part of it, so new traces ran straight over the old ones or crossed them. Now every trace already on the board joins its net's group, and new connections end where they meet that copper. This bug was older than 0.3.0; rigid wires made it show up much more often.
- **Retries:** Auto-route now tries up to six net orders (within about 12 seconds) and keeps the best result: fewest unrouted connections, then fewest jumpers, then the shortest copper. The progress bar says "Try n of 6". On seven test boards (the demos, two random boards and a rebuild of the board from the bug report) this took the unrouted connections from 2 to 0 with jumpers allowed.
- **Jumpers are optional:** untick **Jumpers** in the tool bar (or *Board > Auto-route may add jumpers*) and Auto-route never adds wire links; what it cannot route stays as rubber bands. Each net can also refuse jumpers (net panel, or right-click > *Allow jumpers on this net*). Saved with the board.
- **Rigid wires are not stiff:** a rigid (Shift + Wire) pair still gets its own pin-to-pin trace, but the path bends round things like any other trace, and traces now cost a little more under part bodies, so a trunk is less likely to cut through the middle of a chip.
- **Cut tool** (`C`, or right-click a trace > *Cut from here*): click two points on a trace and only the copper between them is removed; the rest of the trace and the net stay. (*Trim*, `X`, still removes a whole section.)

0.3.0-wip (2026-10-29)

From the first real design made with SoloTrace (a 555 power oscillator):

- **Rigid wires.** Hold **Shift** while wiring (click or drag from pin to pin) and the pair becomes a rigid link: the router joins those two pins with their own direct trace first, and the net's other pins branch off it. Use it for a power trunk, e.g. a transistor emitter straight to the supply terminal, instead of being chained through a capacitor. Rigid links show as thicker orange rubber bands, the routed trunk is drawn in orange copper, and both can be set or cleared from the right-click menu and the net panel.
- **More visible rubber bands:** thicker yellow lines with a dark edge; the wire being drawn is a dashed line that turns orange while Shift is held.
- **Pin labels.** Common pin layouts per form factor, chosen from a list (right panel or right-click > *Pin labels*): TO-92 E B C / C B E / B C E / D G S / G D S / S G D / small regulators; TO-220 power transistor B C E, MOSFET G D S, IGBT G C E, regulator IN GND OUT, adjustable regulator ADJ OUT IN, negative regulator GND IN OUT, triac, dual diode; TO-247; optocouplers (DIP-4, DIP-6); 555-style timer, single and dual op-amp, dual optocoupler (DIP-8); logic GND/VCC and quad op-amp (DIP-14/16/20); pots; 2- and 3-pin connectors. Or type your own. Diodes and LEDs show K A and electrolytics + - by themselves. Labels are drawn beside the pads.
- **Value / type** is now easier to find: bold field at the top of the part panel with examples, *Set value / type...* on the right-click menu, and double-clicking a part jumps to it. Part names and values sit just above small bodies so they no longer cover the pads.
- **TO-92 parts are D-shaped** (flat face and round back) instead of a circle with a line.
- New example and demo: *555 power oscillator* (pin labels, standing resistors, a rigid power trunk).

0.2.1-wip (2026-10-29)

- **Smoother drawing.** Moving the mouse while drawing a wire, a trace, a jumper or placing a part used to redraw the whole board on every move. Now only the moving layer (rubber band, ghost part, hover highlight) is redrawn. On a 40-part test board that is about 6 ms per mouse move instead of about 97 ms; a full redraw went from about 97 ms to about 29 ms (grid points are one item each and only the visible ones are drawn; rubber-band lines are cached until the board changes). Measured in the development sandbox, not on your laptop.

0.2.0-wip (2026-10-29)

Changes after trying 0.1.0-wip on a real screen. 0.1.0-wip was packaged but never published, so this is the first version on GitHub.

- **Wide traces fixed:** a trace of 1.2 mm or more now leaves each pad with a straight vertical or horizontal run before it turns and ends exactly on the pad centre (before, a widened trace left a TO-92 pad at an angle and ended beside the resistor pad as a blob). Runs after auto-routing and, by default, whenever a width changes; *Tidy* and *Tidy all traces* do it on demand. Chamfers can no longer eat those straight exits.
- **Parts panel by form factor:** chips, transistors & regulators, resistors / diodes / inductors, capacitors, LEDs, pots / switches / crystals, connectors, board bits and your own parts, each

with expand arrows for sub-types and a small drawn icon, plus a live drawing of the chosen part in the sidebar (no pop-up). Search matches all words.
- **New parts (90 in total):** standing (vertical) resistors 1/4 W at 2.54 and 5.08 mm, 1/2 W and 1 W at 5.08 mm; standing DO-35 and DO-41 diodes; axial and radial inductors; a 5.08 mm inline pot and a triangle trimmer.
- **Right-click menus** for traces, pins, parts, rubber-band lines, jumpers and the empty board (trim, split, tidy, widths, rotate either way, duplicate, lock, pad styles, wire or trace from a pin, route one net, delete, undo/redo). Right-drag still pans.
- **Trim tool** (`X`) and *Split trace here*.
- **Drag a trace section** sideways with the Select tool; the neighbouring sections follow at their own angle.
- **Look:** coloured tool bar and action buttons, rounded editor corners, a board shadow, status badge (all routed / N to route), net highlight on hover.
- **Ease of use:** "fits up to N mm" hint for the selected trace, arrow-key nudge, `Ctrl+D` duplicate, `Shift+R` rotate back, toolbar hints, *File > Open recent*, autosave every 2 minutes with *File > Recover autosave*, *Route this part's nets*.
- The examples were rebuilt; the logic example now uses auto-place seed 3 (seed 2 left one net unrouted).

0.1.0-wip (2026-10-29)

First work-in-progress build (packaged, not published).

- Tkinter editor: place, wire, trace and jumper tools; undo/redo; pan and zoom; flip view (component side / copper side).
- Library of 76 through-hole parts (90 from 0.2.0) (resistors, capacitors, diodes, transistors, LEDs, TO-220/TO-247, DIP-4 to DIP-40, headers, terminals, JST-XH, mounting holes and pads, wire links).
- Footprint wizard (pad table, click-to-add canvas, DIP and header presets) and KiCad `.kicad_mod` import.
- Trace width slider 0.4 to 10 mm for a segment, a whole trace or a whole net, with a current-capacity readout.
- Pads: through-hole, deadbug (no hole, extended copper for surface soldering) or both, per part or per pad.
- All trace corners chamfered by construction; DRC flags anything over 46 degrees.
- Auto-placer (simulated annealing) and auto-router (heading-aware A* on a 0.635 mm grid, 0/45/90 moves, rip-up and reroute, wire-link jumper fallback).
- Exports: Gerber RS-274X + Excellon drill (zip), 1:1 PDF and SVG (mirrored for toner transfer or not), BOM CSV, netlist text.
- Laser G-code for the Elegoo Phecda (GRBL-style): negative, isolation and positive modes, adjustable trace compensation and gap, line interval, speed, power, passes, crosshatch, frame file, speed/power and trace/gap test cards, time estimate.
- First-start safety and work-in-progress notice.
- Click-and-launch scripts for Linux, Windows and macOS.

Not yet done or not yet tested: see "Known limitations" in the README.

Design notes

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN. These notes are for anyone who wants to know *why* SoloTrace is built this way, what was measured, and what has not been tested. Plain-English behaviour is in HOW_IT_WORKS.md. The original research and plan (written before the code) is in docs/research/pcb-app-research-and-plan.md and the laser-machine facts are in docs/research/phecda-laser-notes.md.

Goals

1. A small, readable library of **real full-size parts** (30 to 100) instead of thousands of SMD codes.
2. **Easy custom parts** without learning a vector editor.
3. **Trace width 0.4 to 10 mm** by clicking a trace and moving a slider.
4. **Deadbug pads** (no drill, extra copper for surface soldering).
5. An **autorouter whose output does not look like tangled string**.
6. **Never a pure 90° trace corner.**
7. **Laser output for a gantry diode laser (Elegoo Phecda)** with a negative option and adjustable gap.
8. Runs on a **7th-generation Core i3** with nothing to install beyond Python.

Real use

- **2026-10-29, the author:** designed his childhood LM555 astable power oscillator (it drives a high-voltage transformer) in about **5 minutes**; he estimates the same board would have taken about **2 hours** in other PCB programs. Tested on an i5 laptop with 8 GB of RAM.
- **Same day, 0.3.0 on his oscillator board:** "routing got far worse", with tracks that disappear and overlap and forced jumpers. Rebuilt from his photo in `tests/test_edit.py` (`test_branches_join_existing_copper_instead_of_running_over_it`). Cause: a net that was partly routed and then re-routed after a rip-up did not count its existing copper, so new traces ran over it (an old bug that rigid wires triggered more often). Fixed in 0.3.1, together with retries, optional jumpers, a small cost for routing under part bodies, and the Cut tool he asked for. He also explained that "rigid" was meant as "always join A to B directly", not "a stiff straight line"; the docs now say so.
- **Evening, 0.3.1:** "much better", but next to the rigid trunks branches looked repelled and malformed: stubs stopping short of a trunk, one overshooting it, a loose blob on a capacitor pad, and a trunk running through the middle of the 555. Cause: the tidy pass (and corner smoothing) reshaped a wide trunk after its branches had been joined to it. Fixed in 0.3.2 with a "keep joints" rule, a DRC "dangling" warning and the keep-out option; regression tests in `tests/test_edit.py` (`KeepJoints`, `Dangling`).
- **Night, 0.3.4:** a board saved in 0.3.3 reopened fine and routed "much better, completely workable"; the remaining wish was for rigid trunks to merge with their branches ("not a true bool union"). Copper was already merged in the exports; the editor drew rigid copper in its own colour on top, and wide branches poked out of narrow trunks. 0.3.5: one colour plus a thin marker line, branch ends settled inside the trunk, branches follow a dragged trunk (`TrunkAffinity` tests).
- **2026-10-31, request for 0.4.0:** "a deadbug island mode for quick and dirty RF designs". A first take built a ground fill with every net cut out as an island; he then said he was "thinking more

just round / rectangular / circular pads which are resizable and draggable", and asked to keep both. So 0.4.0 has copper islands (the pads he meant) and the ground fill (renamed from "deadbug islands" so the two are not confused).
- **2026-11-06, request for 0.7.0:** "add common esp 8xx, esp32, rp2040 and arduino components both for through hole and deadbug, especially the postage stamp size esp c and s3; also the spi sd card reader and the 0.9x inch oled displays; add a mirror function for when we place for instance the display under the board. Mirrored must be highlighted to stand out." Built as 23 module footprints from sourced data (below) and Shift+M with a pink frame and tag.
- **2026-11-05, request for 0.6.0:** "please give this app a web ui when tk doesnt install properly it ruins users experience", plus: rename Pro to SMD with an SMD transistor icon, make the button a whole rectangle in the sidebar, and have it swap the whole list (SMD only, the button then reading Through-hole with a plastic transistor icon). Built as a headless session + local server + canvas page (below); the SMD / through-hole button in both UIs.
- **2026-11-05, request for 0.5.0:** "App is amazing but I have tokens to burn": research other autoroute/autoplace options and add anything better as a selectable option; deadbug ground pads with partial cut-outs that are easy to solder yet carry a fair current; vias / through-hole jumpers to use the back of a double-clad board as a ground plane "without making the back a big LC circuit"; and a "pro" button for the common large SMD parts, with the app staying "first and foremost a single layer non-SMD app with SMD options". Mid-way he added that the first reliefs "look a bit skinny" and the contact should be adjustable for high current "as ground usually takes heavy current": hence the 0.8 mm default, the "size for N amps" box, and the heavy / solid per-part choice. Four research notes were written first (docs/research/), with sources; what was built from them is below.
- Problems he found, and what changed: drawing a wire felt slightly slow (0.2.1: only the moving layer is redrawn); auto-route chained the ground net Q1 emitter → C1 → J1 instead of a thick trunk from Q1 to J1 with a branch to the capacitor (0.3.0: rigid wires, his idea of Shift + wire); the rubber bands were hard to see (thicker, edged lines); pins could not be labelled and values were hard to find (pin-label layouts, a bold value field); TO-92 outlines looked odd (D shape).

Findings (with sources)

- The closest existing tool is a paid Windows program (Sprint-Layout); the free ones are multi-layer, SMD-heavy and weak at single-layer jumpers. Sources and the comparison table are in the research plan.
- "Tangled string" comes from routers that commit copper before choosing topology and route nets one after another. The literature offers rip-up and reroute, negotiated congestion (PathFinder), topological/rubber-band routing and planarisation; see the plan for links.
- Elegoo's manual names LaserGRBL and LightBurn, lists X 25 000 / Y 18 000 mm/min and spot sizes of 0.06 × 0.06 mm (10 W) and 0.07 × 0.13 mm (20 W), and says to set an origin; it does not mention homing, limit switches, GRBL or G-code. Details and links: phecda-laser-notes.md.

Decisions, and what was rejected

Decision	Chosen	Rejected, and why
Module data (0.7)	**Only text sources**: makers' and well-checked community KiCad/Eagle/Fritzing files, datasheets and vendor pages, each number with a confidence word in	Pinout pictures (cannot be checked by machine), memory (one wrong row spacing and the module will not plug in). Where only a footprint author's outline exists, it is used and the note says so; mounting holes

Decision	Chosen	Rejected, and why
	docs/research/mcu-modules.md; pins labelled from the files' own pin names	that the sources disagree on are left out rather than guessed
Mirroring (0.7)	**The existing `flip` flag** (mirror about pin 1), with "mirrored" meaning *not on the part's usual side*; drawn in pink, a colour nothing else on the board uses	A separate "bottom" flag (two flags that could contradict each other); highlighting every copper-side part (every SMD part would be pink and the warning would mean nothing)
Web UI (0.6)	**A local HTTP server from the standard library plus a plain HTML/JS page**, the editor logic in a headless `Session` shared with nothing but the window removed, panels and menus sent to the page as data	A second GUI toolkit (Qt, wx): another install that can fail, the very problem being solved. Rewriting the editor in JavaScript: two copies of every rule (snapping, 45 degree traces, cut/trim/tidy, the fill) that would drift apart; so the page draws and the server decides. A third-party web framework: nothing may be installed. WebSockets: hand-written framing for a page that needs a few requests per second; plain POSTs and a short poll while a job runs are enough
Language	**Pure Python 3 + tkinter**, standard library only	C++ core with pybind11 (the original plan): needs compilers on three systems, which breaks "double-click and run". The router is the part that suffers (see measurements). A C++ accelerator can come later behind the same interface
Custom parts	Footprint wizard (pad table, click-to-add, presets) + KiCad `.kicad_mod` import	Full vector editor: much more code, more ways to make a footprint that does not fit
Copper model	Capsules and polygons, unioned only when exporting	Raster or polygon-clipping library: heavy, and capsules make exact clearance and exact laser scanlines easy
Corners	`Board.add_trace` always chamfers; DRC flags turns over 46°	Allow 90° and warn: the brief said MUST
Router	Heading-aware A* on a 0.635 mm grid, 0/45/90 moves, rip-up and reroute, jumper pass	Plain Lee/maze (staircases), full PathFinder, topological router (see below)
Jumpers	Wire links on the component side in 2.54 mm steps	0 Ω resistors as jumpers: same effect, but harder to see and count
Width UI	A slider in the right panel plus presets, scope = section / trace / net; preset widths also on the right-click menu	A floating pop-up beside the cursor (the plan): more fiddly in tkinter, deferred
Wide-trace shape (0.2)	A tidy pass after routing and after width changes: centred ends, straight axis-aligned pad exits of about pad radius + width, chamfers that may not eat those exits	Routing wide nets with a "leave pads straight" cost inside A*: the grid route is found at the routing width, but the bug report was about traces widened* afterwards*, so a geometric pass that also runs after slider changes covers both
Trace dragging (0.2)	Move one section parallel to itself; neighbours keep their direction and stretch (line intersection); pad ends get a 45° link; then chamfer	Free vertex dragging: easy to make non-45° segments. Push-and-shove of other traces: much more code, left for later
Trim (0.2)	Remove one section; the pieces either side stay as separate hand-drawn traces	Trim to the nearest junction or pad: harder to predict what disappears
Parts panel (0.2)	`ttk.Treeview` grouped by form factor with sub-groups and drawn icons, plus a canvas preview in the sidebar	Thumbnail grid: needs much more room; pop-up preview: the request was "no pop-ups"

Decision	Chosen	Rejected, and why
Keeping joints (0.3.2)	Reshape a trace only if its touch set (pads, traces, jumper pads of its net) is kept and branch ends stay as deep; fall back to centring the end	Re-running the router after tidying, or moving the branches along with the trunk: more code, and moving copper the user did not touch is what caused the complaint
Better results (0.3.1)	Best of up to six tries with shuffled net orders and two under-part costs, scored by unrouted count, then jumpers, then length	A real negotiated-congestion router (PathFinder): still the long-term plan, but retries fixed the boards we have now at the cost of time
Power trunks (0.3)	Explicit rigid pairs routed first, then the rest of the net branches off them (pins connect to the nearest node of the trunk, which the router already allows)	A Steiner-tree router for wide nets, or always picking the two farthest pins as the trunk: guesses what the designer means; the explicit pair is predictable
Pin labels (0.3)	Per-form-factor lists of common layouts plus free text, stored per placed part; defaults only where every part of that shape agrees (diode K A, electrolytic + -)	A database of real part numbers and their pinouts: big, licence questions, and wrong for the many parts that share a package but not a pinout
Look (0.2)	ttk "clam" theme with a dark tool bar, copper / teal / blue / red action buttons, a status badge, and a canvas whose corners are painted over in the panel colour (fake 12 px fillet)	A custom-drawn widget set: lots of code for the same effect
Copper islands (0.4)	A new board object (round, oval or chamfered rectangle) that takes the net of whatever copper touches it, so it needs no net assignment UI and DRC finds shorts for free	Islands as parts in the library (fixed sizes, a "pin" to wire); or asking the user for each island's net: more clicks, and a wrong choice silently disagrees with the copper
Ground fill (0.4)	A paint order instead of polygon clipping: fill rectangle, then every non-ground shape grown by its growth + gap in bare board, then the fattened copper and all copper. Gerber does the same with clear polarity (`%LPC*%`), the laser engine with interval arithmetic per scan line, the canvas, SVG and PDF by drawing order	A polygon-clipping library (pyclipper or shapely): not in the standard library, which breaks double-click and run; writing a robust clipper: a lot of code for the same pictures
Fattening (0.4)	Per shape: grow g = min(wanted, edge room, distance to ground copper - gap, (distance to other nets - gap) / 2), so two growing shapes can never close the gap	One growth for the whole board limited by the tightest spot: a single cramped pin pair would stop every other net from growing
Floating fill (0.4)	Sample the fill on lines 0.05 mm apart, join overlapping intervals into pieces (union-find), cut away pieces no ground pin sits on with one polygon per chain of rows, then repeat across the other axis for slivers thinner than a row	Leave floating copper (it couples noise into RF circuits); one rectangle per row (hundreds of shapes and hairline seams in prints)
Thermal reliefs (0.5)	Ring + spokes generated in the fill/plane paint order; KiCad's geometry (4 spokes, 0.5 mm ring, 45° on round pads, sides on ovals), spokes 0.8 mm by default and sizable for a current by the inverse IPC-2221 formula; per-part heavy / solid	Two-spoke reliefs (higher inductance, less current); per-pad relief settings (more UI than the problem needs); Eagle's "half the drill" rule alone (too thin for a ground that carries the return current)
Back-side plane (0.5)	A second paint-order layer: plane, antipads, reliefs; vias as points that are ground pads with a hole; connectivity by "drilled ground	Routing on the back (two-layer routing is a different program); plane as explicit polygons (clipping library

Decision	Chosen	Rejected, and why
	pins and vias are on the plane"; exports as top copper, non-mirrored prints and laser, plus a countersink list	needed); a via as a part in the library (a via is not a part and must take the ground net automatically)
LC / resonance worry (0.5)	A stitching grid with a frequency guide f = c/(20 s sqrt(er)) and the two-sentence explanation in the dialog; a DRC note on how far the existing joins are good for	Ignoring it (he asked), or a split ground (the EMC literature is unanimous: one plane, many connections)
SMD library (0.5)	Only packages with maker-sourced, labelled land patterns and 0.3 mm pad gaps; roomier wave-solder / level-A patterns; SMD parts flip to the copper side on placement; bottom silkscreen Gerber	Generating pads from IPC-7351 formulas (the standard itself was not accessible; no numbers invented); fine-pitch parts (SOT-363, TSSOP, TQFP: gaps under 0.3 mm, unmaskable); exposed-pad parts (cannot be soldered from the copper side)
Router option (0.5)	Negotiated congestion (PathFinder) on the existing grid and search: overlaps allowed, rising present and history costs, jumpers as a priced resource; "Auto" = negotiated then classic tries	Expansion-room / shove routers (FreeRouting: far too much code), rubber-band topological routing (right idea for one layer, no small open implementation of the geometric stage), keeping classic only
Placer option (0.5)	RUDY congestion + boxed-pad terms with VPR's adaptive schedule, kept as an *experimental* option because it measured no better	Force-directed or min-cut placement (built for scale this program does not have; needs a legaliser that would itself be an annealer)
Laser	Exact scanline intervals computed from the shapes	Rendering a bitmap and tracing it: blurry edges, no exact "trace adjust"
Gerber	Round-aperture strokes + region fills, RS-274X, X2 attributes	Aperture macros for capsules: more to go wrong in viewers
Netlist	Wire pin to pin, no schematic	A schematic editor: far outside "easy"
Library	90 parts, generated from a few parameters	Importing a big library: licences and obscure part codes

What the plan said and what was built

Plan	Built
About 60 parts, C++ core	90 parts, pure Python
PathFinder negotiated congestion	Rip-up and reroute with per-net limits; **no history costs**
Planarisation step	Folded into the placer (crossings in the cost) and the jumper fallback
Topological routing (v2)	Not started
Floating slider pop-up, mouse-wheel nudge, Shift for free value	Right-panel slider + presets + right-click presets
`.solo` file	`.solotrace` JSON
Drill template STL, Specctra DSN export	Not built
Part packs through drop-in plug-ins	Not built (user parts folder instead)
(not in the plan)	Laser G-code, test cards, frame file, auto-placer, netlist text, KiCad footprint import; in 0.2: trace drag, trim, split, tidy, right-click menus, autosave, recent files

0.8: a web app, and collisions after moving a part

- **The browser is the default.** tkinter is the part of Python most often missing or broken, so `main.py` now always opens the web UI; the window is kept behind `main.py desktop`. Rejected: dropping the window (people use it, and it moves parts live).
- **No-install version with Pyodide, in a Web Worker.** The editor stays Python: one code base for the window, the local server and the browser. Rejected: rewriting the editor in JavaScript (two code bases to keep in step), Brython/PyScript on the main thread (the page would freeze during auto-route), shipping Pyodide in the repository (about 12 MB of someone else's build in every clone; the CDN version is pinned instead). `web/api.py` is the request handler shared by `server.py` and the worker, so the browser version answers exactly what the server answers; `Session(sync_mode=True)` runs jobs without threads and reports progress through a hook.
- **Collision finding.** Check itself never missed an overlap: on 64 random part moves over the routed examples it agreed with a brute-force comparison of every pair of copper shapes (`tests/test_collisions.py`). What was missing was any check *at the moment of the move*: dragging, turning, nudging, mirroring or dropping a part over laid copper said nothing until the next Check, and the part body of a mirrored module pressed flat on traces was never checked at all. Now both UIs mark collisions live while dragging and report them on drop, and Check has the body test.

0.9: wire lock, hole strips, breakouts

- **Wire lock.** The old pin pick used the distance to the pad *centre*, which is why a long strip or a big pad was hard to hit. Now the pick is the distance to the pad's copper edge (nearest wins, ties broken by the centre), with a radius of about 14 screen pixels clamped to 1.4 to 3 mm on the board. One function (`Board.pin_lock`) serves both UIs. Rejected: a fixed pixel radius only (too small on a zoomed-out board, too large when zoomed in).
- **A strip is one pad with several holes**, not several pads. Rejected: N pads joined by traces or by overlap (N pins to wire, N nets to keep equal, and a pin to wire to would be ambiguous); a new "bus" object (a second kind of copper to draw, drill, check and export). One oval pad whose `holes` tuple lists every drill reuses the connectivity, the DRC, the Gerber and laser code, and the web renderer. The pad's first hole is its main drill; `pad_extra_drills` returns the rest.
- **Strip and module are joined by touching**, not by a trace, because touching same-net copper is already one piece in `Board.components`. The lead-in strip starts 0.3 mm past the module's pad.
- **Generated, not hand-drawn.** `demos.breakout()` builds the board from the module's pad table and routes with the normal router, so the same code serves any module with two pin columns and 5V / GND / 3V3 pins. A first layout with four full-height rails failed to route 5V and ran traces under the module; three short stepped rails fixed it.
- **No copper under the module** (many of these modules have pads or a battery connector on the back).

0.10: antennas, keep-outs, coil generator

- **A part may carry copper of its own.** Antennas and coils are not pads, so `PartDef` gained `strokes` (a named polyline with a width, drawn as round-ended segments; a stroke named after a pad belongs to that pad and is on its net) and `keepouts` (boxes). A stroke named `*` is "body copper": it has its own net `~REF.*`, so the pads it touches keep theirs. Rejected: long runs of tiny pads (a pin to wire to per bit) and a new copper layer type.
- **One keep-out, four consumers.** `Board.keepouts()` is read by DRC (`check_keepout`, which also runs while moving a part), by the ground fill and the back-side plane (an extra bare cut in their paint order), by the router (stamped HARD, so even the net feeding the antenna cannot cross it;

an early version let the RF net run through it) and by auto-tidy. Rejected: only a DRC check (the router would draw the error for you and the fill would flood the box).

- **Antenna length is a formula, not a layout.** `L = c / (4 f sqrt(eps_eff))`, `eps_eff = (4.4 + 1) / 2`, +10 % to trim, silk ticks every 2 mm. Rejected: copying TI's tested layouts (could not be downloaded here; their layouts are the maker's work; a layout copied from a figure I could not read would be an invention).
- **The coil engine is a sum of straight bars.** Rosa self inductance per bar, Grover mutual inductance per parallel pair. It was chosen over a fitted formula (Mohan 1999, Wheeler) because it needs no coefficient table and handles non-square spirals; the Mohan formula is the cross-check in the tests (+1.3 %, +1.4 %, +2.5 % on three square coils). The search tries turns 1..79 and 40 pitches each, keeps the gap above the limit and the hollow big enough for the inner pad, and prefers fewer turns and a wider gap among near-equal answers.
- **The inner end is a wire, and the part says so.** A one-sided spiral cannot bring its centre out. `PartDef.ties` lists pad pairs joined by a wire; `Board.components` unions them, so airwires and the router treat them as one piece. The first version left the pins separate and the router could not make a jumper from a pad inside a hard keep-out (2 connections stayed unrouted).

Measurements

0.8: has auto-route got lazier? (12 boards, 12 s budget, development machine)

The four examples that exist in every version (555 flasher, 5 V regulator, 555 power oscillator, logic mix), each as built and with two seeded random placements, saved by 0.3.4 and routed by each version with its default settings.

Version	Jumpers (all 12)	Unrouted	Copper, mm
0.3.4 (oldest in git)	64	3	4 515
0.7.0 (= 0.8.0 before the fixes)	44	0	4 509
0.8.0 (sorted rip-up order + polish)	42	0	4 498

So no, by these numbers: fewer jumpers, nothing left unrouted, the same copper. Per board the newer router is sometimes longer on the small boards (5 V regulator moved: 323 mm with 3 jumpers in 0.3.4, 366 mm with 2 now) because the best-of score puts fewer jumpers before shorter copper. A weighted score (a jumper worth about 30 mm of copper) was considered and not done: it would pick more jumpers on exactly those boards, and jumpers are what people want fewer of. Versions before 0.3.4 are not in the repository and were not measured.

Found on the way: results changed from run to run. The rip-up step rerouted its victims in the order of a Python `set` of net names, which depends on `PYTHONHASHSEED` (random for every start of Python): the moved regulator board routed as 2 jumpers / 366 mm, 3 / 351 mm and 4 / 269 mm under three seeds. The victims are now sorted; three seeds give identical copper (tested). The polish pass (each net rerouted once against the finished board, kept only if strictly better, at most two passes) runs only on the winning try, so it does not use up the time for more tries; on these boards it removed 2 jumpers and shortened the logic boards by 2 to 4 %.

All on the cloud sandbox CPU, **not** on a 7th-gen Core i3; expect it to be slower there, by an amount nobody has measured.

Board	Auto-place	Auto-route	Result
555 flasher, 70 × 50 mm, 8 parts	4.1 s	0.1 s	all 7 nets, 0 jumpers, DRC clean
5 V regulator, 70 x 45 mm, 6 parts	2.1 s	< 0.1 s	all 4 nets, 2 jumpers, DRC clean
Logic mix, 80 × 60 mm, 8 parts + header	about 7 s	3.5 s	depends on the placement seed: seed 3 (the shipped example) routes everything with 10 jumpers, DRC clean; seed 2 leaves net G unrouted, seeds 4 and 5 leave net D unrouted
Random board, 12 parts / 74 pins	6.4 s	2.9 s	all 18 nets, 5 jumpers, DRC clean
Random board, 25 parts / 118 pins	7.3 s	6.4 s	all 29 nets, 6 jumpers, DRC clean
Random stress board, 40 parts / 178 pins	12.7 s	33.1 s	2 nets left unrouted, 25 jumpers

With the 0.3.1 retries (best of up to six tries, sandbox CPU): 555 flasher 0.1 s (first try was perfect), 5 V regulator 6.9 s (2 jumpers), power oscillator 0.2 s, logic mix 16 s (7 jumpers instead of 10), the rebuild of the bug-report board about 8 s (1 jumper, nothing unrouted; with jumpers switched off, 1 connection is left for hand routing). On seven test boards the unrouted connections went from 2 to 0.

Laser (default settings, `examples/`): 555 board 625 scan lines, negative about 27 min and isolation about 4.5 min by the program's own estimate; 5 V board 24 min / 3 min; logic board 36 min / 8 min. Estimates are distance over feed plus rapids and are not measured on a machine.

Tests: 48 pass (`python3 src/main.py selftest`), including a test that re-reads the exported Gerber and drill files with the independent `gerbonara` library (skipped if it is not installed), chamfer and 45° rules, DRC detection, DRC cleanliness of the routed 555 and regulator demos, pad clearance of all 90 parts in all three pad styles, laser mode geometry (negative burns more than isolation, trace adjust widens gaps, crosshatch doubles the work), G-code sanity, test cards, scripted GUI sessions (place, wire, route, click a trace, move the slider, deadbug toggle, open the dialogs, the parts tree and preview, the right-click menu, dragging a trace with the mouse, trimming), and the 0.2 trace edits: the board from the bug report gets straight pad exits and centred ends at 2 mm, tidying never adds clearance problems with nets it was not already touching (1.5, 3 and 5 mm), dragging keeps every segment at 0/45/90°, trim and split keep the right pieces, and tidy is idempotent.

Router and placer options (0.5.0)

Eleven boards, each placed once and saved, then routed by each algorithm from the same placement, on the development machine with nothing else running (`tools/` has no benchmark script; the boards were the five demos plus six random boards of 12 to 30 parts). Classic = best of 6 net orders within its 12 s budget; negotiated = one run; auto = negotiated, then the classic tries only if needed.

Algorithm	Unrouted connections	Jumpers	Copper length	Time
Classic	3	53	4993 mm	78 s
Negotiated	0	55	4402 mm	52 s
Auto	0	54	4471 mm	99 s

Per board the negotiated router was better or equal on nine of the eleven (logic demo 5 jumpers instead of 7 and 383 mm instead of 466; power oscillator 237 mm instead of 296); it was worse on the one genuinely non-planar random board (19 jumpers against 14 to 16), and *auto* does not recover that because the negotiated run alone uses up the time budget there. Earlier variants of the jumper pricing (jumpers allowed from the first pass; allowed only once the price reached 2, 3 or 4) gave 71, 54 (3 unrouted), 53 (2 unrouted) and 61 jumpers; the shipped setting (allowed once the price reaches 1, at five times the usual cost) was the only one with nothing unrouted and no jumper penalty. The classic router's numbers move by a jumper or two between runs because its time budget decides how many tries happen; in one run on the non-planar board it also produced a layout with one DRC clearance error that could not be reproduced afterwards.

Placer, same method (place for 15 s, route with classic, 11 boards): annealing 73 ratsnest crossings, 1 unrouted, 69 jumpers, 55 s; routability 119 crossings, 3 unrouted, 57 jumpers, 133 s. Fewer jumpers in total but more crossings and more unrouted, and twice the time: not better, so it stays experimental and off by default. Weight tuning (RUDY 0.5 / boxed 5, RUDY only, boxed only) on three boards did not change the picture.

Drawing speed (0.2.1)

A slowdown while drawing a wire on an i5 laptop with 8 GB was reported. Profiling a 40-part board showed the cause was not the speed of Python but the drawing strategy: every mouse move deleted and recreated about 6 600 canvas items (most of them grid points drawn as two little lines each), about 97 ms per move. The fix keeps the board as a static layer and redraws only a small "dynamic" layer (rubber band, ghost part, trace preview, hovered net) on mouse moves; grid points became one item each and only the visible ones are drawn; rubber-band lines are cached until the board changes.

Measured in the sandbox, 40-part random board	Before	After
Mouse move while drawing a wire	about 97 ms	about 6 ms
Mouse move with the Select tool (hover highlight)	not measured (it did a full redraw whenever the hovered net changed)	about 5 ms
Full redraw (after an edit, zoom or pan)	about 97 ms	about 29 ms

Moving the drawing to C would not have helped: the time was spent inside Tk creating canvas items, which is already C. C would help the router and the placer; see Ideas.

Design details worth knowing

- **Heading in the router state.** The A *state is position* and* direction, with a turn cost, so a 90° bend becomes two 45° turns and long straight runs win over staircases.
- **Exact check after the grid search.** A grid path is only accepted after it has been checked against the real copper shapes and clearances; if it fails, the cell or edge responsible is forbidden and the net is searched again. This is slower than trusting the grid but stops wide traces clipping neighbours.
- **Deadbug pads grow outward.** The extra copper (about 2.2 mm) extends away from the part and is clamped by neighbouring pads; rows of three or more fold perpendicular to the row. The library test checks every part in all three styles keeps at least 0.25 mm between pads.

- **Width of an unrouted net.** Clicking the yellow rubber-band line creates a net class named after the width (for example "2.5 mm"), so the router uses it.

- **Ground fill with clear polarity.** The Gerber copper layer of a filled board is: a dark region (the fill), `%LPC*%`, the cut shapes, `%LPD*%`, then the copper. The independent gerbonara library read this back with 63 clear and 120 dark objects for the RF example and drew it as expected; no fab CAM has seen it yet.

The web UI (0.6)

- **The page is a thin client.** `scene()` sends every copper shape in paint order (capsules and polygons in mm, Y up, with net and tag), the plane and fill plans, outlines, pin-label anchors, rubber bands, vias, marks and the tool state; the page draws it with canvas 2D and the same colours as theme.py. Hit testing, snapping, trace finishing, cut/trim/tidy, dialogs and menus all run on the server, so a rule changed in board.py or tidy.py changes both UIs. The page keeps only what has to follow the mouse without a round trip: pan and zoom, the ghost of a dragged part / island / via / trace section, the next trace leg (`snap_dir` is the one function ported to JavaScript, 12 lines), the wire rubber band and the placing ghost (the armed part's shapes at the origin, translated).
- **Panels, menus and dialogs as data.** The selection panel is a list of widgets (`head`, `label`, `entry`, `select`, `check`, `radio`, `slider`, `button`, `row`); the right-click menu a list of `{label, cmd, args, sub, checked, disabled}`; a dialog a list of fields plus the command that applies them. The page renders them generically, so the desktop panels and the web panels show the same choices without a second copy of the logic.
- **Drag = ghost + one command.** A press answers with what was picked up (`{"ref": ...}`, `{"trace": ..., "seg": ...}`, `{"island": ..., "resize": ...}`); the page draws a ghost while the mouse moves and sends `drag_to` once when it is let go. The desktop window moves things live, which is nicer for traces (the branches follow); the ghost keeps the server traffic to two requests per drag. The mouse can move on before the press has been answered, so the page counts that movement too (a fast drag used to end as a click).
- **The ground fill's second pass.** The desktop window computes the full fill (floating copper removed) 0.4 s after the last edit; the session does the same in a background thread while the page polls `fill_ready`, so a quick plan is drawn at once and the final one a moment later, as in the window.
- **Jobs.** Auto-place and auto-route run in a thread on a copy of the board, exactly as in the window; the page polls every 0.4 s for the progress text and the result is applied on the next request after the job ends.
- **Hover is a request too** (`hover`, throttled to about 15 per second and never queued more than one deep); it returns the net under the mouse and what the hit test found, so the page can highlight the net and pick the cursor. On a ground-fill board with 500 copper shapes it takes a few milliseconds.
- **Fuzzing the session** (`cmd` with random names and random, often wrong, arguments on all eight example boards, 400 steps each, several seeds) found: negative indexes reaching a list (Python counts from the end), a split at the very end of a trace leaving a one-point trace that then broke tidy, a pending wire pin whose part had been deleted, and a dialog name that was not one. All fixed; the server also catches anything a command raises and shows it to the page rather than dying.
- **Security.** The server binds 127.0.0.1 and refuses any request whose `Host` is not the local address (a web page elsewhere cannot make the browser talk to it by name), serves only files

inside `static/`, and reads only the JSON body it is given. There is no authentication beyond that; another program on the same computer could talk to it, as it could to the window.

Edge-case sweep (0.4.1)

`tools/` has no fuzzer in the repo; the sweep scripts were throwaway. What they did, so it can be repeated: (1) about 400 random boards (3 to 9 random parts, 0 to 8 random islands including off-board and sub-minimum ones, random fill settings and ground net including a non-existent one), each checked before and after auto-route that fattening never closes a gap and that a floating cut never covers a ground pin, then exported to Gerber, SVG, PDF, BOM and G-code in all three laser modes (with crosshatch and trace adjust), and round-tripped through the file format; (2) about 4 000 random editor actions on the four demo boards with the Tk error hook recording any exception. Neither found a crash after the fixes; the bugs listed in the changelog came from the targeted checks and from reading the code. Timing of the fill check, 160 × 100 mm board with 574 copper shapes: 0.95 s full, 0.02 s without the floating check; a 0.1 mm sampling step would have saved only a quarter, so the check was moved off the editing path instead.

Tried and dropped (0.2)

- **"No worse" by count.** The first tidy version accepted a reshaped trace if it touched no *more* shapes than before. On a board where a 5 mm trace already overlapped a thin one, the thin trace was then re-pathed straight across the wide one: the same count, but a new short. Now a reshaped trace may only touch copper of nets it was already too close to, and thin traces are only allowed to move their end onto the pad centre, never to shortcut.
- **Comparing by shape tag** in that rule also failed in a test, because trace numbers change when other traces are tidied; the test now compares by net.
- **Tool button labels with the shortcut key** ("Select S") made the tool bar wider than a 1280 px window; the keys moved into the hover hints.

Found while making the power-oscillator demo (0.3): a 2.5 mm power net could not be routed from the middle pin of a TO-220, because at 2.54 mm pitch a trace that wide plus 0.3 mm clearance does not fit between the neighbouring pads (it misses by 0.01 mm at 2.5 mm). The demo uses 2 mm; a "neck-down" that narrows the first millimetres near a pad is on the ideas list.

Not tested (be honest)

- **0.9: the hole strips and the three breakout boards** were checked by unit tests, DRC, an independent Gerber read and the autorouter only: never etched, never soldered, no module plugged in. The strip drill (1.0 mm) and 2.54 mm hole pitch are the library's usual values. The XIAO 5V pad is the USB 5 V line according to Seeed's symbol; back-feeding it was not looked up. The wire lock was tested with unit tests and scripted events; no person has used it. The custom-footprint text editor drops a pad's `holes`.

- **0.10: the antennas and the coil** come from formulas only. Nothing was etched, measured with a NanoVNA or LCR meter, or simulated with a field solver. The effective permittivity is a rule of thumb. Whether the meander gives a usable match without a matching network is unknown. The coil estimate ignores self-capacitance, skin effect and anything near the coil. The wire between coil pins 2 and 3 is not drawn on the board. The custom-footprint text editor drops strokes, keep-outs and ties.

- **The in-browser version (0.8)** was tested in headless Chromium with a local copy of the npm package `pyodide@0.29.5` (the same files the CDN serves), not with the CDN itself: the build machine's network does not reach `cdn.jsdelivr.net`. Firefox, Safari and phones have not been tried. Speed on a slow laptop has not been measured.

- **Module footprints (0.7)** have not been etched and had a real module plugged in. The SuperMini's position along its board (about 1 mm of disagreement between sources), the Waveshare Zero outlines, the DOIT outline, the 0.96 inch SPI OLED header position and the SD modules' pin orders are single-source or disputed; the part notes say so.

- **The web UI (0.6)** has been driven by unit tests, a headless-Chromium test and the screenshot script only; no person has designed a board in it, and no browser other than Chromium has shown it (Firefox and Safari should be fine: canvas 2D, fetch and plain CSS, no libraries, but it has not been looked at). The no-tkinter fall-back was exercised with a Python 3.13 that really has no tkinter and with tkinter blocked in the tests; the launchers' "tkinter missing" branches were not run on Windows or macOS.

- **Real laser, real Phecda, real etching.** All laser defaults (1 500 mm/min, 70 %, 0.08 mm interval, 0.6 mm gap) are starting guesses.
- That the Phecda's controller behaves like GRBL (`$30`, `$32`, M4 dynamic power).
- Windows and macOS launchers and the macOS Tk; GitHub workflows (never run).
- Gerber files in a fab's own CAM or a second viewer.
- 7th-gen Core i3 speed; large boards (above about 40 parts).
- The Trace and Jumper tools have had light manual testing only. Trace dragging, trim and the right-click menus have been tested with scripted mouse events, not by a person.
- The new generic footprints (standing resistors and diodes, inductors, 5.08 mm pot, triangle trimmer) are not measured against real parts.
- Tooltips and the right-click menu on Windows and macOS (Tk draws menus natively there, so colours will differ).
- Part outlines (silkscreen) are approximate; check against the real part.
- Thermal reliefs, the back-side plane, vias and the SMD parts (0.5): tests, examples and screenshots only. Nobody has soldered a relieved pad, drilled a via, countersunk a plane or placed an SMD part from this library. The relief current figures are the IPC-2221 formula, conservative for short spokes, not measurements. The SMD pad sizes are the makers' recommendations; two inductor patterns and the SMD electrolytics were read off drawings whose labels did not survive extraction (the research note says which).
- The negotiated router was measured on eleven boards on the development machine only; the classic router's results vary a little between runs because its 12 s time budget decides how many of the six tries happen.
- Copper islands and the ground fill (0.4): only tests, the examples, screenshots and gerbonara. Nobody has etched, lasered or soldered such a board yet, and the 0.05 mm sampling leaves slivers of fill below about 0.01 mm² (too small to etch).

Ideas

- Native accelerator for the router and placer (a small C library loaded with `ctypes`, with the pure-Python code kept as the fallback so double-click still works without a compiler). Pre-built libraries for Linux, Windows and macOS would come from the GitHub release workflow. The A* search, the obstacle maps and the annealing loop are the hot spots.

- A topological (rubber-band) router for a hand-drawn look; the negotiated router (0.5) covers the PathFinder half.
- Routability placer: tune the weights against routed jumper counts rather than crossings, or drop the RUDY term and keep only boxed pads; a force-directed first placement to seed the annealer.
- Back plane: a second routing layer is out of scope, but a few *back-side jumper traces* (short straight runs on the plane with their own clearance) would remove most jumpers on a double-clad board.
- SMD: pin-1 markers in the bottom silkscreen export; an "exposed pad" footprint once there is a way to solder it (a through-hole slug via).
- Floating slider pop-up; copper pours; soldermask layer; drill-guide STL for 3D printers.
- Push-and-shove when dragging; dragging a single corner; teardrops where wide traces meet small pads.
- Neck-down: let a wide trace narrow automatically where it leaves a fine-pitch pad.
- Pin labels and values in the silkscreen exports (needs a small stroke font for Gerber).
- Let auto-route choose straight pad exits itself for wide nets (a cost in the search) so the tidy pass has less to do.
- Part packs as plug-ins; flip parts to the copper side for deadbug builds.
- Islands: more shapes (L-shapes by joining rectangles already work), "make an island under every pin of this net", and a stitched fill (several pieces joined by jumpers automatically).
- Laser: exposure-test presets per paint type once real data exists; air-assist and focus notes.

How to test a change

```
python3 src/main.py selftest                        # or: xvfb-run -a python3 -W ignore -m
unittest discover -s tests
python3 tools/make_examples.py                      # rebuilds examples/ from the demos;
look at the DRC lines
xvfb-run -a -s "-screen 0 1280x840x24" python3 tools/take_screenshots.py   # needs
ImageMagick; rebuilds docs/images/gui-*.webp
python3 tools/take_web_screenshots.py                                      # needs
Playwright + Chromium; rebuilds docs/images/web-ui*.webp
```

The GUI tests need a display (`xvfb-run` on a headless machine) and a Python with tkinter.